

Manual

MES Development Suite MDS-RPB 8.1

Version 1.1.14621

Last changed on: 09.11.2018

Copyright

©Copyright 2018 All rights reserved.

SAP® and R/3® are registered trademarks of SAP AG.

WINDOWS® is a registered trademark of Microsoft Corporation.

MPDV® and HYDRA® are registered trademarks of MPDV Mikrolab GmbH.

ORACLE® is a registered trademark of ORACLE Corporation, California, USA.

Copying and distribution of this documentation or any part thereof, for any purpose or in any form, is prohibited without prior written permission from MPDV Mikrolab GmbH.

The information contained in this documentation is subject to change without prior notice.

Contents

1	Overview	9
2	MES Development Suite	10
2.1	Activating the MES Development Suite	10
2.2	Applications in MOC	10
2.3	Meaning of Customizing	11
3	MOC Configuration Settings	14
3.1	Configuration settings and configuration levels	14
3.2	Activation of a configuration level	15
3.3	Storage locations for configuration settings	15
3.3.1	User data	16
3.3.2	System-wide (local) changes	16
3.3.3	Modifications by MPDV	17
3.3.4	Notes on application configurations	17
3.4	Distribution of configuration settings	18
3.5	Configure syntactic types	19
3.6	Change the MOC logging	25
3.6.1	Change the storage location of the log file	25
3.6.2	Change the log level	26
4	Update Packages for the Maintenance Manager	27
4.1	Overview	27
4.2	Black list for MOC updates using Maintenance Manager 2	28
4.3	Structure of MOC Client Package	29
4.4	Structure of Java Server Package	31
4.5	Structure of Server Package	34
5	Reports on the basis of Crystal Reports	37
6	The Repository	41
6.1	Overview	41
6.2	Domain	41
6.3	Service	41

6.3.1	Name.....	42
6.3.2	Function	42
6.3.3	ServiceType	42
6.3.4	ListMode.....	43
6.3.5	DLG.....	43
6.3.6	SystemCall	43
6.4	ServiceGui.....	43
6.4.1	Name.....	43
6.4.2	Package	43
6.4.3	Extended	43
6.4.4	AdditionalDataLogics.....	44
6.4.5	ApplicationID	44
6.4.6	ApplicationTitle	44
6.4.7	ApplicationHelpFile.....	44
6.4.8	ApplicationHelpIndex.....	44
6.4.9	Description	44
6.5	ServiceParameter.....	45
6.5.1	Acronym.....	45
6.5.2	ResultSet.....	45
6.5.3	WebServiceType	45
6.5.4	DefaultValue.....	45
6.5.5	IsResult	45
6.5.6	IsDynamicResult	45
6.5.7	InputAsArray.....	46
6.5.8	IsSpecialParameter	46
6.5.9	IsFilterParameter	46
6.5.10	IsMandatory.....	46
6.5.11	Can* (Filter) Operators	46
6.5.12	HydraAcronym.....	47
6.5.13	HydraResultAcronym.....	47
6.5.14	TransferEmptyValuesToHydra.....	48
6.5.15	HydraShiftPart.....	48
6.5.16	Reference.....	48
6.5.17	TransformationType	48
6.5.18	PlugName	49
6.5.19	DBField	49

6.5.20	DBAlias	50
6.5.21	DBTable	50
6.5.22	DBFieldAlternative.....	50
6.5.23	DataObjectName	50
6.5.24	ConditionalFieldKey.....	50
6.5.25	Constraints	51
6.6	ServiceParameterGui	51
6.6.1	Acronym	52
6.6.2	ResultSet.....	52
6.6.3	Label	52
6.6.4	Tooltip	52
6.6.5	FormatType	52
6.6.6	ClientDefaultValue.....	53
6.6.7	IsKey	55
6.6.8	ShowInGrid.....	55
6.6.9	ShowInDetail	55
6.6.10	ShowInSearch	55
6.6.11	ColumnCategory	55
6.6.12	Category1, Category2, Category3	56
6.6.13	TabOrder.....	56
6.6.14	ColmnOrder.....	56
6.6.15	ShowSecondControllnSearch.....	56
6.6.16	SearchTabOrder.....	56
6.6.17	SearchCategory1, SearchCategory2	57
6.6.18	ControlType.....	57
6.6.19	ControlTypeMode	57
6.6.20	ControlParameter	59
6.6.21	ControlDataSource	59
6.6.22	ControlDataSourceMode	59
6.6.23	ControlDataSourceParameter	59
6.6.24	ControlDataSourceResult.....	59
6.6.25	VisibleCondition.....	60
6.6.26	EditableCondition	60
6.6.27	ScriptId	61
6.7	Property	61
6.7.1	Acronym.....	61

6.7.2	WebServiceType	61
6.7.3	NETType	62
6.7.4	SemanticType	62
6.7.5	SyntacticType	62
6.7.6	Label	63
6.7.7	DefaultTooltip	63
6.7.8	UnitLabel	63
6.7.9	OutputFormat	63
6.7.10	InputFormat	63
6.7.11	Length	64
6.7.12	Rules for the input/output formatting	64
6.7.13	FillChar	68
6.7.14	Calculation	68
6.7.15	Further fields see ServiceParameterGui	68
6.8	ControlDataSource	68
6.8.1	Name	69
6.8.2	Source	69
6.8.3	Parameter	69
6.8.4	Columns	69
6.8.5	Result	70
6.9	ReferenceData	70
6.9.1	ref_data_key	70
6.9.2	Type	70
6.9.3	db_key	71
6.9.4	is_default	71
6.9.5	Designation	71
6.9.6	sort_key	71
6.10	Authorization	71
6.10.1	Authorization type	71
6.10.2	Authorization Context	71
6.10.3	Authorization ID	72
6.10.4	Authorization key	72
6.10.5	Authorization Designation	72
7	Using the MDS Repository as Development Tool	73

7.1	Use of Transformation Types for the Dynamic Conversion of DB or BAPI Values	73
7.1.1	Summary	73
7.1.2	Adding a New Transformation Function	74
7.1.3	Standard Transformation Functions	76
7.2	Checklist: Repository Data	86
7.3	Entry of Fixed Values for Wrappers	89
8	Repository Client	91
8.1	Quick start	91
8.2	Start and exit Repository Client	93
8.3	The application window	94
8.4	Grids/table views	95
8.5	The application menu	97
8.6	Workset	100
8.7	Relations	103
8.8	References	104
9	Using the MDS Repository Client as development tool	106
9.1	How to create new contents	106
9.2	Context menu of the table view/grid	108
9.3	Export	113
9.4	Validation	113
10	Interpreted Java Service2	114
1.1	Introduction	114
10.1	Availability	114
10.2	Definition	114
10.3	Storage in a server	114
10.4	Available Special Parameters	114
10.5	Repository data	115
10.5.1	Tab Services	115
10.5.2	Tab ServiceParameter	116
10.5.3	Tab Dataobjects	118
10.6	Exits	122
10.6.1	Available user exits	122
10.6.2	Available program exits	123

10.6.3	Specifications for the implementation class of the exit	123
10.6.4	Interfaces	124
11	Interpreted Java Service	134
11.1	Introduction	134
11.2	Definition	134
11.3	Storage in a server	134
11.4	Available Special Parameters	135
11.5	Repository data	136
11.5.1	Tab Services	136
11.5.2	Tab ServiceParameter	136
11.5.3	Tab Dataobjects	138
11.6	Exits	143
11.6.1	Available user exits	143
11.6.2	Available program exits	143
11.6.3	Specifications for the implementation class	144
11.6.4	Interfaces	145

1 Overview

The MES Development Suite offers functions for customizing the HYDRA Client MES Operation Center to your particular requirements.

This document initially supplies background information on the MES Development Suite and in particular on the significance of configurations for customization; subsequently, it describes the functions provided by the product MES Development Suite - Report Builder (MDS-RPB).

2 MES Development Suite

The MES Development Suite offers functions for customizing the HYDRA Client MES Operation Center to your particular requirements. Background information generally required for handling customizing and other modifications is provided below.

2.1 Activating the MES Development Suite

The MES Development Suite is (de)activated in the main menu via [Extras --> MES Development Suite](#).

In order that the MES Development Suite may be activated (and/or in order to activate the menu entry), licenses which have to be loaded in the relevant system are required.

These licenses are

- MDS-BAS: MES Development Business Applications & Services
- MDS-RPD: MES Development Suite Report Design

Either of these licenses is sufficient to activate the menu entry, however only MDS-BAP will enable you to activate all functions available.

2.2 Applications in MOC

MOC offers a variety of different functions made available via applications. Applications may offer very different functions, but their structure is consistently the same - it applies equally for a complex application for evaluations/reports such as the "workplace overview" and for a simple editing dialog such as that for maintaining "units".

The basic elements of an application are:

- Ribbon or toolbar with buttons to activate functions
- Selection area or selection panel to parameterize data queries
- Area for detail applications where one or any number of detail application(s) may be presented.
- Data sources or DataControllers responsible for providing detail applications with data, i.e. for calling (web) services on the server providing the relevant data.

From a technical point of view, editing dialogs for creating, editing and copying of data records are also applications. However, they normally do not provide detail applications but only an individual (selection) area to enter parameters used to call the corresponding editing function (i.e. the relevant web service).

Please note: Maintenance applications are normally generated using the application generator (included in the product "MES Development Suite – Business Applications").

2.3 Meaning of Customizing

MOC allows for the creation of new and the customization of existing functions via customizing. This means that the control of available functions, in particular as regards applications, is not effected by programming but by maintenance of customizing files (.config) usually implemented by operating specially developed customizing dialogs. These dialogs are integrated into the software and are enabled by activating the MES Development Suite as required.

General background information on customizing settings and in particular their distribution to the clients is provided in a separate chapter. Some special features of special customizing settings, e.g. those of applications or menu entries, are described below.

The specific changes which are possible in these customizing settings, each, depend on the rights of the relevant user and/or the available licenses. In general, however, an ordinary user may also perform changes in these files, e.g. by storing a table layout customized for own demands.

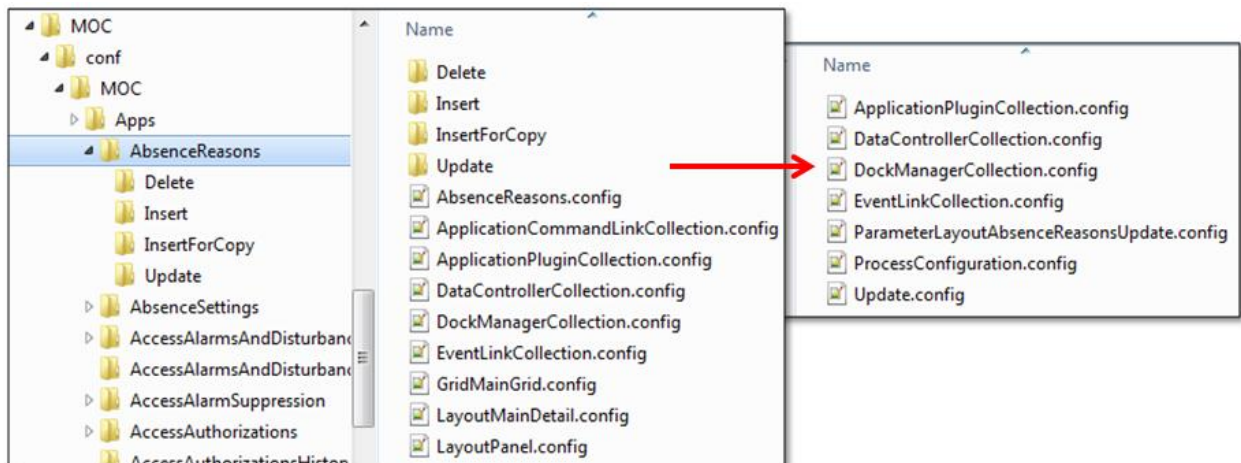
Customizing files for applications

There are (main) applications for presenting data and applications used for the maintenance of data. For each main application, the directory %scope%\conf\Moc\Apps contains a directory with a clear (English) name of the application. Maintenance applications are always allocated to a main application and are therefore found in a subfolder of this main application.

Example: The "Absence reasons" application is filed as a customizing file in the "applications" folder within the sub-folder with the name of the related application ID, in this case AbsenceReasons. The maintenance dialogs required for maintaining absence reasons are managed in the subfolders "Delete", "Insert" and "Update".

Main application

The application directory contains all customizing files required for the customizing of an application. The type and number of files depends on the relevant application, among other things on the number of detail applications (tables, graphs, etc.) included in the application.



Content of an application directory including maintenance applications.

The list below provides an overview of the most important files:

- <Id of the application>.config → Customizing data for the application (title, help file, ...)
- LayoutPanel.config → Customizing the selection area
- DockManagerCollection.config → Customizing the layout of detail applications
- DataControllerCollection.config → Customizing the data sources of the application
- ApplicationPluginCollection.config → Customizing the detail applications
- EventLinkCollection.config → Customizing the relations between data sources and detail applications
- ApplicationCommandLinkCollection → Customizing the toolbar.

In addition to this, the main application includes a separate customizing file for each detail application. These customizing files can usually be identified by their file prefixes. For example

- Grid*.config → detail application with table
- *Chart*.config → detail application with chart
- Pivot*.config → detail application with pivot
- Layout*.config → detail application for detail views

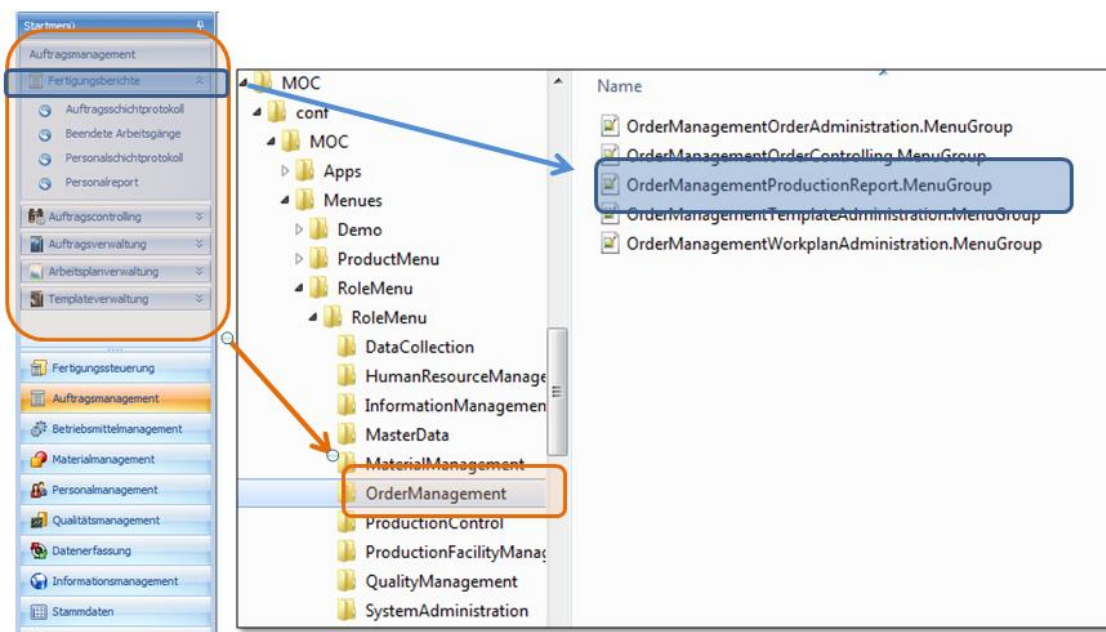
By further development or special plug-ins, other files, which are not listed here, may also be included in the application directory.

Maintenance applications

Maintenance applications are special applications (see above) the customizing files of which are managed in subfolders of a main application. In addition to the files described above, the application directory of the main application contains an additional directory for each maintenance application including the customizing files. Each maintenance application includes an additional file `ProcessConfiguration.config` that includes information on the process of calling this maintenance application. Moreover, the main configuration file also includes additional, specific information.

Customizing files for menus

As many menus as required may be created and managed in MOC using [Extras --> Menu editor](#). Each menu includes (main) menu items including sub items storing the actual functions. The customizing files of different menus are found in the `%scope%\conf\Moc\Menues` directory. Each main menu is managed in a separate subfolder. The submenus are represented by a separate file, each, containing the functions.



Example for a menu structure: The folder of the menu “RoleMenu” includes subfolders, such as the “OrderManagement” folder that manages the structure of the “order management” menu. The submenu “production reports” including its functions is managed in the file “OrderManagementProductionReport.MenuGroup”.

3 MOC Configuration Settings

Overview

Menu	System menu: Help → Configuration help
Transaction code	
Function authorization	

→ Layout and functions of the MES Operation Center (MOC) are determined by numerous configuration settings: this includes, for example, the current window size or the used language or the number and order of columns in a table of a specific application. This section provides background information on the management of MOC configuration settings and describes, on the basis of this, how individual configuration settings can be shared by the entire company.

3.1 Configuration settings and configuration levels

Subject to the currently valid configuration level (scope), every configuration setting may have up to four different values. MOC has the following configuration levels:

- The “standard” configuration level includes values that are provided for all users.
- The “custom” configuration level includes values that are provided for all users according to the users’ requirements.
- The “local” configuration level includes values that are provided locally for all users according to the users’ requirements (e.g. by an administrator or key user).
- The “user” configuration level includes the values that a user has created individually.

At runtime, MOC imports all values and respectively selects the most specific value. If there is a value in the “user” configuration level, this one will be used instead of the values from the levels “local”, “custom” or “standard”. This rule always works up to the respectively active configuration level, i.e. in case the “local” level is active, only values from the levels “local”, “custom” or “standard” are used, but not from the “user” level.



Configuration settings that are intended for being used throughout the entire system should be made in the “local” configuration level.

If you want to make changes to the “local” configuration level, activate the “local scope” at first. Then all changes, for example, to applications are written in the *scope* folder after saving, i.e. the subfolder *custom\conf* of the MOC program directory.

3.2 Activation of a configuration level

Normally, MOC is operated with the “user” configuration level.

Use the system option “DefaultSettingScope” or the function “MES Development Suite”, “System Information Center” to set the configuration level. (Note: the latter function is only available if corresponding licenses have been purchased.)

Set the system option “DefaultSettingScope” in the file MOC.ApplicationSettings.config or via a command line parameter.

The following row in MOC.ApplicationSettings.config specifies the option:

```
<add key="DefaultSettingScope" value="User" />.
```

Allowed values are “standard”, “local”, “custom” and “user”. Restart MOC, once changes have been made.

As an alternative, set the configuration level via the command line parameter DefaultSettingScope=<scope> (e.g. in a link to moc.exe). Example

```
C:\Programme\MOC.exe DefaultSettingScope=Local
```



Only MPDV staff is allowed to use the configuration levels “standard” and “custom”. In general, users are never required to change these configuration levels, as this would endanger system stability and, in particular, the system’s ability to be upgraded.

3.3 Storage locations for configuration settings

All configuration values are saved in files. One file normally summarizes a number of configuration values. The files of a configuration level are always summarized in one folder structure. In this context, the following paths correspond to the default settings:

- The configuration values of the “user” configuration level are filed in the subfolder *user\conf* of the Windows user folder of the MOC application.
You can find the folders here:
Windows 7: [LocalApplicationData]\MPDV\MOC\user\conf
- The configuration values of the “local” configuration level are filed in the subfolder *local\conf* of the MOC program directory.
- The configuration values of the “custom” configuration level are filed in the subfolder *custom\conf* of the MOC program directory.

- The configuration values of the “standard” configuration level are filed in the subfolder *conf* of the MOC program directory.

You can find the currently applicable storage locations in the MOC function “Help --> System information”, “system” tab, “configuration levels”.

You can change the storage locations for configuration levels by configuring the system options in MOC.ApplicationSettings.config. The corresponding options are described in the sections that follow.



Please note that the MOC update process only takes into account the standard values for storage locations. If you change the storage locations, you are responsible for making sure that software updates are also installed in the new folders.



If you change the storage location, please consider that the start of the application might be slowed down by loading files over the network.

3.3.1 User data

The system option “UserDataDirectory” specifies where user data and, as a result, the configuration values changed by the user are saved. You can use placeholders when you define paths.

Default value:

```
<add key="UserDataDirectory" value="$ApplicationData\user\" />
```

Note: \$ApplicationData refers to the data directory for the MOC application. In Windows 7 this is the folder C:\Users\<user>\AppData\Roaming\MPDV\MOC\.

Allowed placeholders are:

- %HYDRAUSER%: the name of the registered HYDRA user
- %WINDOWSUSER%: the name of the registered Windows user
- %HYDRASYSTEM%: the name of the system the user is logged on to.

Example:

```
<add key="UserDataDirectory" value="\\dataServer\moc\users\%hydrauser%" />
```

3.3.2 System-wide (local) changes

The system option “LocalConfigurationDirectory” determines where the configuration data of the “local” configuration level is saved. This configuration level includes individual or local changes applicable to the entire system.

Example:

```
<add key=" LocalConfigurationDirectory" value="\\dataServer\moc\local\" />
```

Note: The placeholders described in section 3.3.1 must not be used here!

3.3.3 Modifications by MPDV

The system option “CustomConfigurationDirectory” determines where the configuration data of the “custom” configuration level is saved. This configuration level includes the modifications provided by MPDV.

Example:

```
<add key=" CustomConfigurationDirectory" value="\\dataServer\moc\custom\" />
```

Note: The placeholders described in section 3.3.1 must not be used here!

3.3.4 Notes on application configurations

Each application has a separate subfolder for configuration files in the subfolder conf\MOC\Apps of the corresponding “scope folder”. The settings of the “workplaces/resources” application with the application ID “workplaceoverview”, for example, are stored in the below-mentioned folders:

Configuration level (scope)	Folder
User	C:\Users\<cbu>\AppData\Roaming\MPDV\MOC\user\conf\Moc\Apps\WorkplaceOverview
Local	C:\Programme\MPDV\MOC\local\conf\MOC\Apps\WorkplaceOverview
Custom	C:\Programme\MPDV\MOC\custom\conf\MOC\Apps\WorkplaceOverview
Standard	C:\Programme\MPDV\MOC \conf\MOC\Apps\WorkplaceOverview

The scope of a configuration value depends on the folder. Consequently, you can also transfer changed values to the “local scope” by copying files from the user directory to the local directory.

If you click the save function in an application, only the changes made are saved, i.e. the folder of the “local” configuration level does not include the entire application, but only deviations from the content of the “standard” configuration level.

If you load configurations, the folder that includes the settings file determines the configuration level of a configuration value. Consequently, you can also transfer changed values to the “local” configuration level by copying files from a user directory to the local directory.

3.4 Distribution of configuration settings

To make an application configuration changed in the “local scope” available in other MOC installations, for example, copy this configuration to the folder of the “local” configuration level of the required installations.

MOC clients are updated via the update function (see section **Fehler! Verweisquelle konnte nicht gefunden werden.**) and, usually, by downloading updates provided by the Maintenance Manager.

Using the “MOC Update Package Creator”, you can create update packages that may be imported to the Maintenance Manager. The tool is started via the function *Extras* → *Generate update package*. You have to enter the following:

- Directory that includes data for the update package. Usually, the program directory of the MOC client that was used to perform the changes.
- Directory where the update package is to be filed once it has been generated.
- Name and description of the update: this information is displayed in the Maintenance Manager, once the update package has been imported to the Maintenance Manager.

Please note the following:

The tool requires the function authorization “mupc”.

The update name must not include blank characters or umlauts.



The directory including the update data must contain a subfolder with the selected configuration level (local or custom).

The configuration level is set to the value “local” by default. The “custom” value is reserved for deliveries by MPDV.

Click the option “generate update package” to create the update package. Then import this update package to the Maintenance Manager. Then all MOC clients can install the changes by using the “search for updates” option in the system menu *Help*.

3.5 Configure syntactic types

Overview

Menu	-
Transaction code	syty
Function authorization	syty

Syntactic types make sure that data is presented in a uniform manner in the MOC. Consequently, you can use the syntactic type for quantities to specify the number of decimal places. This setting affects all quantities displayed in the MOC.

Most of the syntactic types can only be changed by MPDV or customers/partners if they use the MES Development Suite.

But the application "configure syntactic types" additionally provides the option to configure some important syntactic types directly on the customer's systems without needing a customization order or the MES Development Suite.

The application "configure syntactic types" is an expert application and only available in English. Usually, MPDV consultants use this application to adjust specific syntactic types of MOC data according to the customer's requirements.



The application "configure syntactic types" uses the methods of the MES Development Suite. The document "MES Development Business Applications & Services" (MDS-BAS) provides further basic information on the MES Development Suite you require when working with the application "configure syntactic types".

Acronym	Label	Unit Label	Output Format	Input Format	Length
> cycle	lkcycle	lkHrsPer 1000	{0:mpdv_cycetime}		10
quantity	lkquantity		n0	[0-9]{0,10}	10
quantity_negative				-?[0-9]{0,10}	0
st_iw_te	lkoperation.plan.te	lkHrsPer 1000	{0:mpdv_te}		10
st_iw_teb	lkoperation.plan.teb	lkHrsPer 1000	{0:mpdv_te}		10
st_iw_tr	lkoperation.plan.tr	lkHrs	{0:mpdv_te}		10
st_iw_trb	lkoperation.plan.trb	lkHrs	{0:mpdv_te}		10
st_mf_te	lkoperation.plan.te	lkHrsPer 1000	{0:mpdv_te}		10
st_mf_teb	lkoperation.plan.teb	lkHrsPer 1000	{0:mpdv_te}		10
st_mf_tr	lkoperation.plan.tr	lkHrs	{0:mpdv_te}		10
st_mf_trb	lkoperation.plan.trb	lkHrs	{0:mpdv_te}		10

This document illustrates the procedure step by step. Consequently, even inexperienced users should be in the position to make the most important changes independently.



You can configure the syntactic types included in the client file "ConfigurableSyntacticType.Properties.xml".

Definition of terms

This section briefly explains technical terms used in this application, the MES Development Suite and HYDRA system administration.

Scope

A scope is a level where you can configure and program HYDRA.

Standard

MPDV uses the *standard* scope to deliver standard products.

Custom

MPDV uses the *custom* scope to deliver customizations that complement or overwrite the standard.

Local

Customers or partners can use the *local* scope to make changes that complement or overwrite the *custom* and the *standard* scope.

Update package

An update package includes programs and configurations the Maintenance Manager first installs on the server. Then the MOC Updater distributes the MOC data from the server to the other MOC clients. Usually, this is an automatic process.

Update Package Creator

The Update Package Creator is a tool used in the MOC to pack locally created modifications in an update package. The application "configure syntactic types" uses a simplified and restricted version of the Update Package Creator.

Maintenance Manager

Web application used to install update packages on the HYDRA server. Usually, your IT department is familiar with the procedure as MPDV regularly sends updates that are installed via the Maintenance Manager.

Overview of the procedure

This section provides a brief overview of the steps you have to perform to change syntactic types. The sections that follow provide further details.

1. Load configuration: loads the existing configuration of syntactic types from the system.
2. Change table data: changes the configuration.
3. Save changes.
4. Create update package: creates an update package to distribute the new configuration.
5. Install update package: installs the update package via the Maintenance Manager.
6. Update MOC clients: updates MOC clients via the MOC Updater.

1) Load configuration

Load syntactic types from the system before you can change them. You can define the scope:

Local

Loads the syntactic types you have already changed. In case you have not changed syntactic types, the system loads the syntactic types provided by MPDV from the custom scope or the standard scope.

Custom

Loads the changed syntactic types provided by MPDV. In case MPDV has not changed syntactic types, the system loads the standard configurations. This process does not include syntactic types you have already changed.

Standard

Loads the syntactic types from the standard scope. This process does neither include the modifications provided by MPDV nor the customizations you made.

As a general rule, you should choose the following methods for loading syntactic types:

- Create or change modifications you made: load configurations from the *local* scope.
- Discard changes you made and reset configurations to the version delivered by MPDV: load configurations from the *custom* scope.

2) Change table data

You can make changes to the table. Usually, you only have to change the columns *OutputFormat*, *InputFormat* and *Length*.

Table columns

Acronym

Name of the syntactic type. You should not change this value.

Label

Labeling of input fields displayed in front of input fields. By default, "language keys" are used for the labels. These keys are translated depending on the language set in the MOC. If required, you can also enter customer-specific texts instead of "language keys". But these texts will not be translated. Customers using the product MES Development Suite (MDS-BAS) can define their own language keys. There are some rare places in the MOC that are not affected by changed labels. But the entire system will be affected if you use the MES Development Suite (MDS-BAS) to customize the *language key* of the label.

UnitLabel

The UnitLabel is the labeling displayed behind the input field. The same rules apply as to the label.

OutputFormat

Output format for data. A separate section describes expedient output formats.

InputFormat

Input format to check user input. Normally, the MOC automatically defines the appropriate input format that matches the output format. You should only indicate the InputFormat if additional checks are required. So-called "regular expressions" specify the InputFormat. Regular expressions are common standard in software development. You can find further information on regular expressions on the internet.

Length

Field length: number of characters.

Configuration of quantities

You can change the output and input formats for quantity fields:

Output format

n<x>: shows the number with thousands separator. <x> indicates the number of decimal places.
e. g. n1, n2

f<x>: shows the number without thousands separator. <x> indicates the number of decimal places.
e. g. f1, f2

Input format (examples)

[0-9]{0,10}

Integer with up to 10 digits. No minus sign allowed; only positive values supported.

-?[0-9]{0,10}

Integer with up to 10 digits and optional leading minus sign.

-?[0-9]{0,9}\R.?[0-9]{0,3}

Optional sign, up to 9 pre-decimal positions and optionally up to three decimal places.

Configuration of cycles

You can edit the label, UnitLabel, OutputFormat and length. Appropriate values are assigned by default to "UnitLabel" and "OutputFormat". The following configurations are useful:

Unit Label	Output Format
lkHrsPer1000	{0:mpdv_cycletime}
lkUnitsSecondsPerOne	{0:mpdv_cycletime_sec_cycle}
lkUnitPiecePerHour	{0:mpdv_cycletime_piece_hour}

Configuration of single piece specifications (te, teb)

- Syntactic types starting with "st_iw_" affect incentive pay applications.
- Syntactic types starting with "st_mf_" are used in all other applications.

You can edit the label, UnitLabel, OutputFormat and length. MOC automatically assigns reasonable values to the *InputFormat*.

You can use the format "mpdv_calc..." to perform calculations for the output format. The value existing in the HYDRA database is the basis for calculations. This value is stored in "seconds per 1000 pieces".

UnitLabel	OutputFormat
lkHrsPer1000 = [h/1000]	{0:mpdv_te}
lkUnitsSecondsPerOne = [sec/1]	mpdv_calc;MULT=1;DIV=1000;INVERSE=false;FORMAT=f3

lkUnitMinutesPerPiece = [min/1]	mpdv_calc;MULT=1;DIV=60000;INVERSE=false;FORMAT=f3
lkUnitPiecePerHour = [1/h]	mpdv_calc;MULT=1;DIV=3600000;INVERSE=true;FORMAT=f3
[1/min]	mpdv_calc;MULT=1;DIV=60000;INVERSE=true;FORMAT=f3
...	...

Output formats with calculation allow you to calculate the inverse by setting "INVERSE=true". Consequently, you can display data as "quantity per time" instead of "time per quantity".



First use the multiplier and divisor. Then calculate the inverse. If you want to display the *pieces per minute*, you have to divide the t_e in [sec/1000] by 60000 to get [*minutes/piece*]. Then calculate the inverse to indicate [*pieces/minute*].

Configuration of setup specifications (tr, trb)

- Syntactic types starting with "st_iw_" affect incentive pay applications.
- Syntactic types starting with "st_mf_" are used in all other applications.

You can edit the label, UnitLabel, OutputFormat and length. MOC automatically assigns reasonable values to the *InputFormat*.

You can use the format "mpdv_calc..." to perform calculations for the output format. The value existing in the HYDRA database is the basis for calculations. This value is always stored in "seconds".

UnitLabel	OutputFormat
lkHrs	{0:mpdv_te}
[min]	mpdv_calc;MULT=1;DIV=60;INVERSE=false;FORMAT=f3
...	...

3) Save changes

Click the "save" button to save changes to the local MOC client in the local scope. You should not change the file name.

4) Create update package

Click the "Create update package" button to start the Update Package Creator. The input fields are populated with default values. Enter the following settings:

Path (folder that includes the generated update)

Specifies the folder where the update package is stored. The folder must exist already.

Update name

We recommend appending a unique ID to the update name. You can add date and time, for example: "SyntacticTypes20170726_1342".

Version number

Optionally, you can indicate a version number that will be displayed in the Maintenance Manager.

5) Install update package

Like any other HYDRA update, the generated update package is installed via the Maintenance Manager.

6) Update MOC clients

Usually, the MOC Updater downloads the updates to the MOC clients. You can use the menu to search immediately for updates (Help --> Search for updates). Once all MOC clients have been updated, the changes to the syntactic types take effect.

3.6 Change the MOC logging

By default, the client log files are located in the user directory of the Windows user who runs the MOC. The log files are stored in the following directory, if they are not customized:

```
[LocalApplicationData]\MPDV\MOC\log\
```

3.6.1 Change the storage location of the log file

We recommend to separate the log entries of different MOC instances in order to facilitate the failure analysis. To change the storage location, create a new file named "NLog.user.config" or "NLog.local.config" with the following content in the main directory of the affected MOC installation (e.g. "C:\Program Files (x86)\MPDV\MOC"):

```
<?xml version="1.0" encoding="utf-8" ?>
<nlog xmlns="http://www.nlog-project.org/schemas/NLog.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" autoReload="true">
  <variable name="logpath" value="${specialfolder:folder=ApplicationData}\MPDV\MOC\log" />
</nlog>
```

Change the path highlighted in red and enter your required storage location (e.g. \${specialfolder:folder=ApplicationData}\MPDV\MOC\TEST\log). Ensure that the local user has write access to this folder.

Use the file NLog.**user**.config for modifications that only apply to this specific workstation (client). Whereas you should use the file NLog.**local**.config to distribute the modifications to all workstations (clients) via the update package.

3.6.2 Change the log level

In some rare cases, you might have to increase the MOC log level. To do so, create the file "NLog.user.config" with the following content as described in the previous section:

```
<?xml version="1.0" encoding="utf-8" ?>
<nlog xmlns="http://www.nlog-project.org/schemas/NLog.xsd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" autoReload="true" globalThreshold="Trace">
</nlog>
```

Once you have sent the log file generated with the increased log level, you should delete this file. As increasing the log level might have considerable negative impact on the performance.

4 Update Packages for the Maintenance Manager

4.1 Overview

Update Packages are used to distribute new features via the Maintenance Manager. The following chapter describes the structure of these files.

An update package is an archive file (zip) and can have any name. The file extension *upd* is set by default.

The package can include the following subfolders:

client: MOC client package in *.*upd* format (0-n)

→ You can also deploy these *.*upd* files individually.

java: java server package in *.*upd* format (0-n)

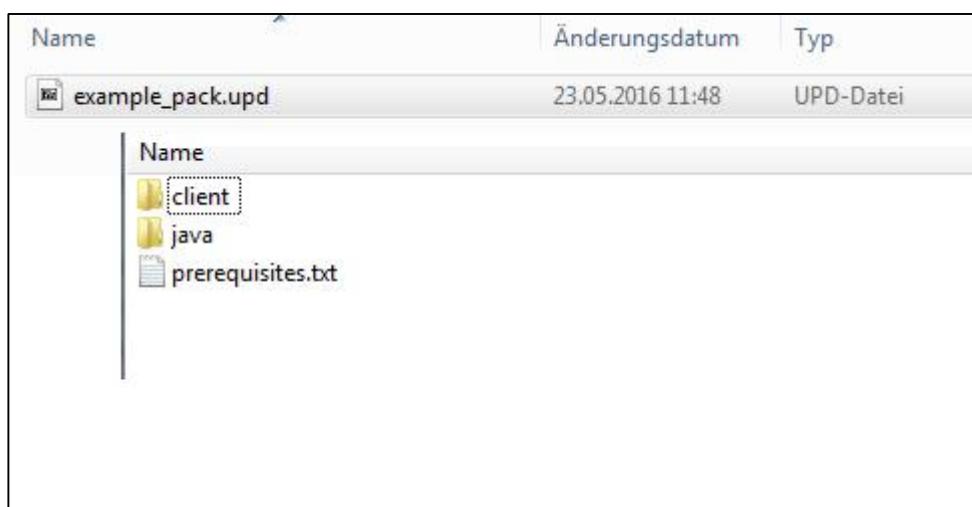
→ You can also deploy these *.*upd* files individually.

server: server packages in *.*upd* format (0-n)

→ You can also deploy these *.*upd* files individually.

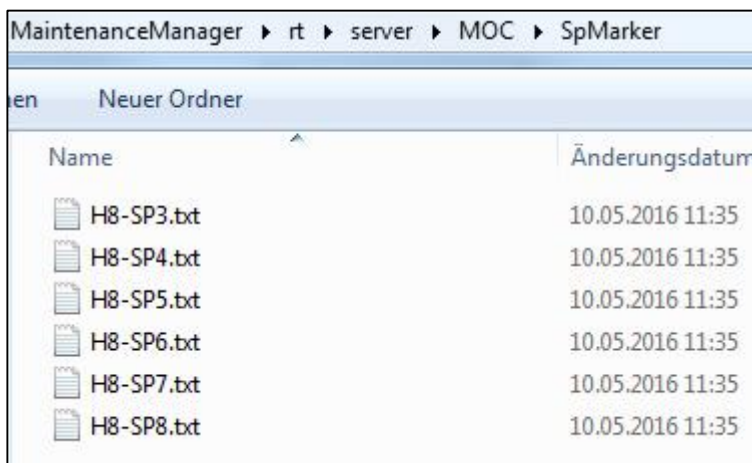
You may find a *prerequisites.txt* file in addition to the folders mentioned above. The *prerequisites.txt* file includes information on the required service pack or hotfix version.

You can install update packages via the *Package Deployment* in the Maintenance Manager.



prerequisites.txt

The file *prerequisites.txt* describes the requirements, i.e. which service pack is needed. You can check in *MaintenanceManager\rt\server\MOC\SpMarker* if the required file exists.



4.2 Black list for MOC updates using Maintenance Manager 2

The update process and update behavior of a MOC installation on a workstation PC have changed if you use Maintenance Manager 2 and the MOC Updater. In contrast to the previous approach to the MOC update, where files were only supplemented or updated, the new update process also deletes files.

During the update process, the local MOC installation is compared/synchronized with the reference version in *Maintenance Manager 2*. All files that do not correspond to the server's reference version are overwritten or deleted. This also applies to files created or modified as part of the development of customizations with the MES Development Suite.

To avoid data loss, you can exclude directories or files from the update process. For this purpose, enter the corresponding files or directories in a MOC black list. You can only enter files and directories that are located in the MOC main directory!

You can create the black list using any text editor. Save the file as "Blacklist.txt" in the home directory of the MOC Updater <MOC installation directory>\update\ in order for the MOC Updater to process the file.

The file structure must be in JSON format. Enclose each entry in quotation marks. Separate multiple entries with comma.

Example of a *Blacklist.txt* file:

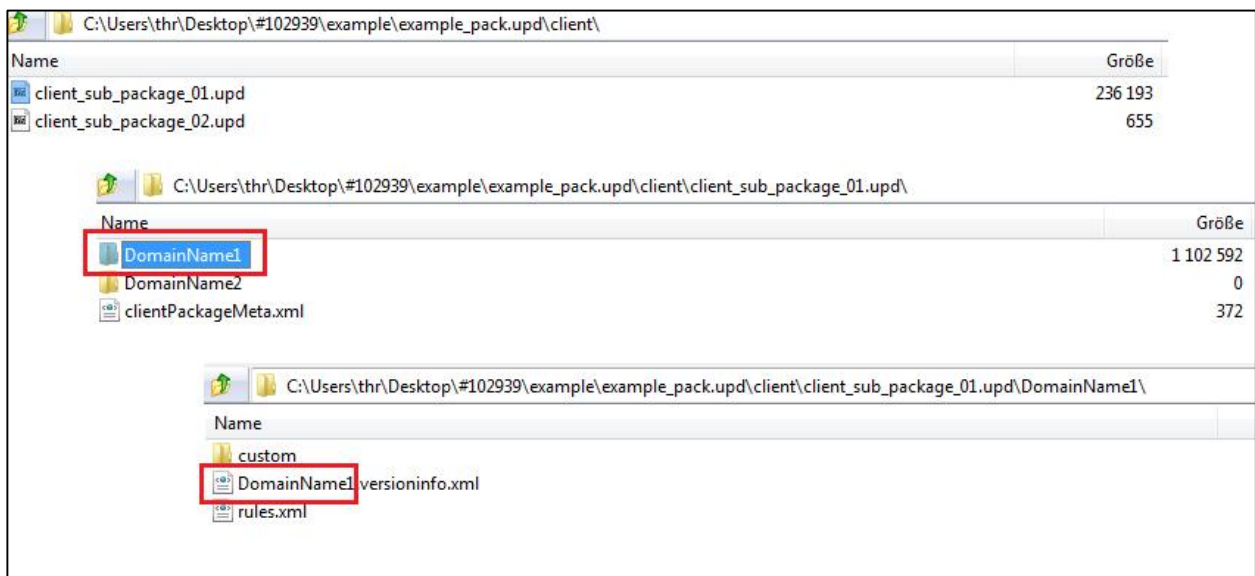
```
{
  "DirectoryBlacklist":
  ["local\\",
   "custom\\",
   "conf\\MOC\\Apps\\"],
  "FileBlacklist":
  ["conf\\DoNotDelete.txt",
   "conf\\DoalsoNotDelete.txt"]
}
```

Especially when developing your own applications in the local scope, the directory of the local scope has to be entered into the blacklist, so that the local proprietary developments are not deleted by the MOC Updater:

```
{
  "DirectoryBlacklist": ["local\\"]
}
```

4.3 Structure of MOC Client Package

An MOC update package is structured as follows:



clientPackageMeta.xml:

The *clientPackageMeta.xml* in the root directory of the *.upd folder includes information on the contents of the update package: name of the update package without file extension, description, date of creation, name of the application, 1-n domains.

```
<?xml version="1.0" encoding="utf-8"?>
<packageMeta>
  <name>client_sub_package_01</name>
  <description>Description package 01</description>
  <creationDate>2016-05-23T09:30:08</creationDate>
  <applicationName>MOC</applicationName>
  <domain version="1.0.CUST.00001">DomainName1</domain>
  <domain version="1.0.CUST.00002">DomainName2</domain>
</packageMeta>
```

#.versioninfo.xml

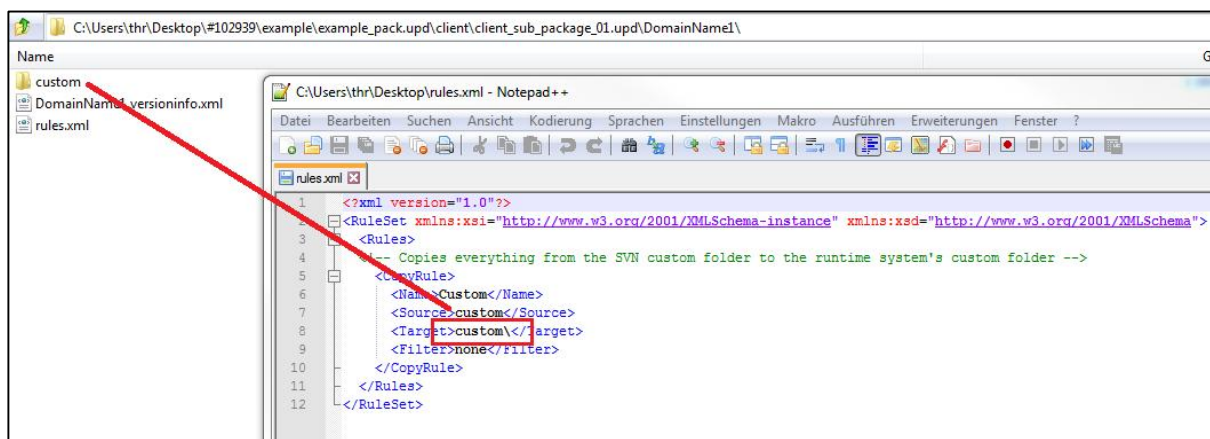
You can find the **#.versioninfo.xml** below the higher-level domain. Enter the domain name for the placeholder "#". Enter the correct customer ID and the domain as object ID in this file.

```
<?xml version="1.0"?>
<mpdvVersion xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <CustomizedVersion>0</CustomizedVersion>
  <MajorVersion>1</MajorVersion>
  <MinorVersion>0</MinorVersion>
  <Revision>45661</Revision>
  <CustomerId>CUST</CustomerId>
  <ObjectId>DomainName1</ObjectId>
</mpdvVersion>
```

rules.xml:

You can find the *rules.xml* below the higher-level domain. This file includes 1-n copy rules. These rules define which file / which directory (source) is filed in which target directory. Use the filter to select specific files. If you only want to copy *xml* files, enter the following filter: "<filter>*.xml</filter>". This example copies the complete contents of the *custom* folder into the MOC runtime directory. Use the placeholder **#SERVER#** in the target, to store the files directly in *JHYDRADIR* after activation in the Maintenance Manager.

You can find further copy rules in the description of the java server packages.



MaintenanceManager ▶ rt ▶ MOC ▶ custom ▶	
Neuer Ordner	
Name	Änderungsdatum
resources	09.05.2016 15:48
readme.txt	10.05.2016 11:37

4.4 Structure of Java Server Package

A server update package is structured as follows (the examples mentioned below sometimes include the placeholder #CUSTNAME#; replace this placeholder with the appropriate customer name):

C:\Users\thr\Desktop\#102939\example\example_pack.upd\java\	
Name	
server_sub_package_01-versions.lst	
server_sub_package_01.upd	
server_sub_package_02-versions.lst	
server_sub_package_02.upd	
server_sub_package_01-versions.lst	
1	TYPE DOMAIN VERSION
2	CUST_DOMAIN Cust#CUSTNAME#DomSvcU_#CUSTNAME#_server_sub_package_01 1.0.#CUSTNAME#.00001
3	

*.lst files are not relevant for the update package and are created for internal purposes only. This file is not mandatory.

C:\Users\thr\Desktop\#102939\example\example_pack.upd\java\server_sub_package_01	
Name	
Custom	
deploymentMeta.xml	
packageMeta.xml	

deploymentMeta.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
- <deploymentMeta>
  <warName>MocServices</warName>
  <confContextName>MOC</confContextName>
</deploymentMeta>
```

packageMeta.xml:

The *packageMeta.xml* in the root directory of the *.upd folder includes information on the contents of the update package: name of the update package without file extension, description, date of creation, 1-n domains including version, customer, type, path and name.

```
<?xml version="1.0" encoding="UTF-8"?>
- <packageMeta>
  <name>server_sub_package_01</name>
  <applicationName>MOC</applicationName>
  <description>Custom Service for sub package 01</description>
  <creationDate>2016-05-23T11:30:43</creationDate>
  <packageElement version="1.0.#CUSTNAME#.00001" customer="#CUSTNAME#" type="CUST_DOMAIN" path="Custom/Domains/Services/U_#CUSTNAME#_DomainName1"
    name="Cust#CUSTNAME#DomSvcU_#CUSTNAME#_DomainName1"/>
  <packageElement version="1.0.#CUSTNAME#.00002" customer="#CUSTNAME#" type="CUST_DOMAIN" path="Custom/Domains/Services/U_#CUSTNAME#_DomainName2"
    name="Cust#CUSTNAME#DomSvcU_#CUSTNAME#_DomainName2"/>
  <packageElement type="THIRD_PARTY_LIBS" path="ThirdPartyLibs" name="thirdpartylibs"/>
</packageMeta>
```

Name	Größe
ExtSvc	0
ExtSvcMapping	0
Interpreter	0
MpdvCust#CUSTNAME#DomSvcU_#CUSTNAME#_DomainName1.xml	132
rules.xml	3 680

MpdvCust#CUSTNAME#DomSvcU_#CUSTNAME#_DomainName1.xml:

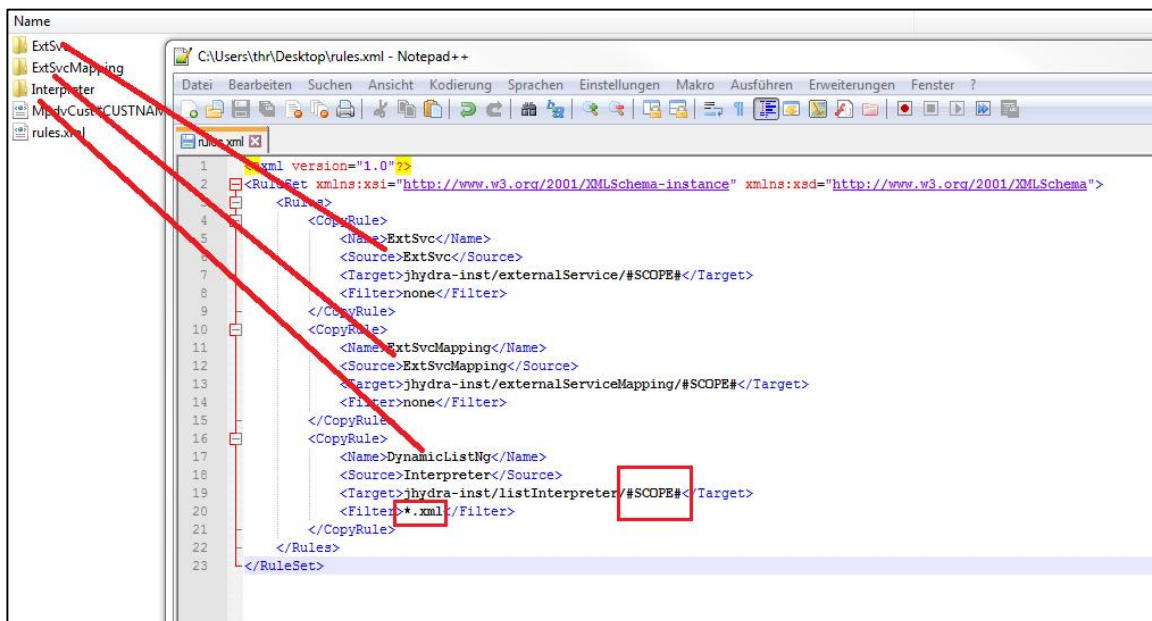
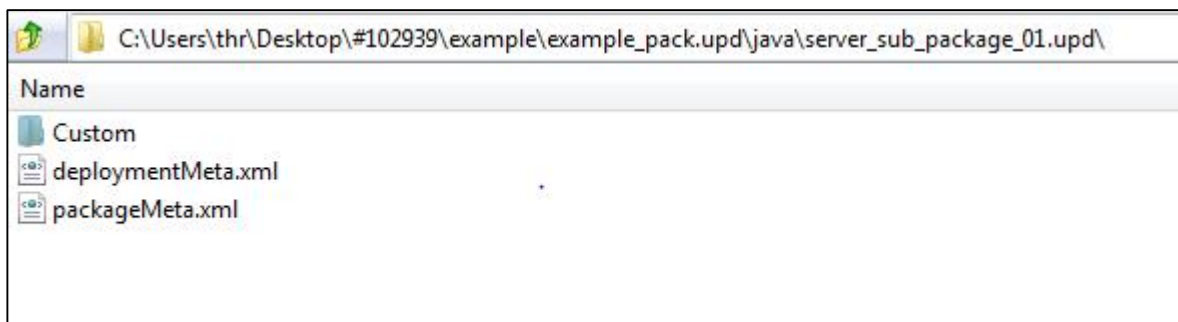
```
<?xml version="1.0" encoding="UTF-8"?>
- <projectMeta>
  <projectType>library</projectType>
  <useInWar>false</useInWar>
</projectMeta>
```

rules.xml:

You can find the *rules.xml* below the higher-level domain. This file includes 1-n copy rules. These rules define which file / which directory (source) is filed in which target directory. Use the filter to select specific files. If you only want to copy *xml* files, enter the following filter: "<filter>*.xml</filter>".

This example copies *ExtSvc*, *ExtSvcMapping* and the folder *Interpreter* to the *JHYDRADIR* (runtime directory of the Maintenance Manager) of the predefined subdirectory. The placeholder *#SCOPE#* is then replaced with the directory created in the root directory of the update package (e.g. custom, standard, local).

Use the placeholder *#CLIENT#* in the target to store the files directly in the runtime directory (MOC) after activation in the Maintenance Manager.

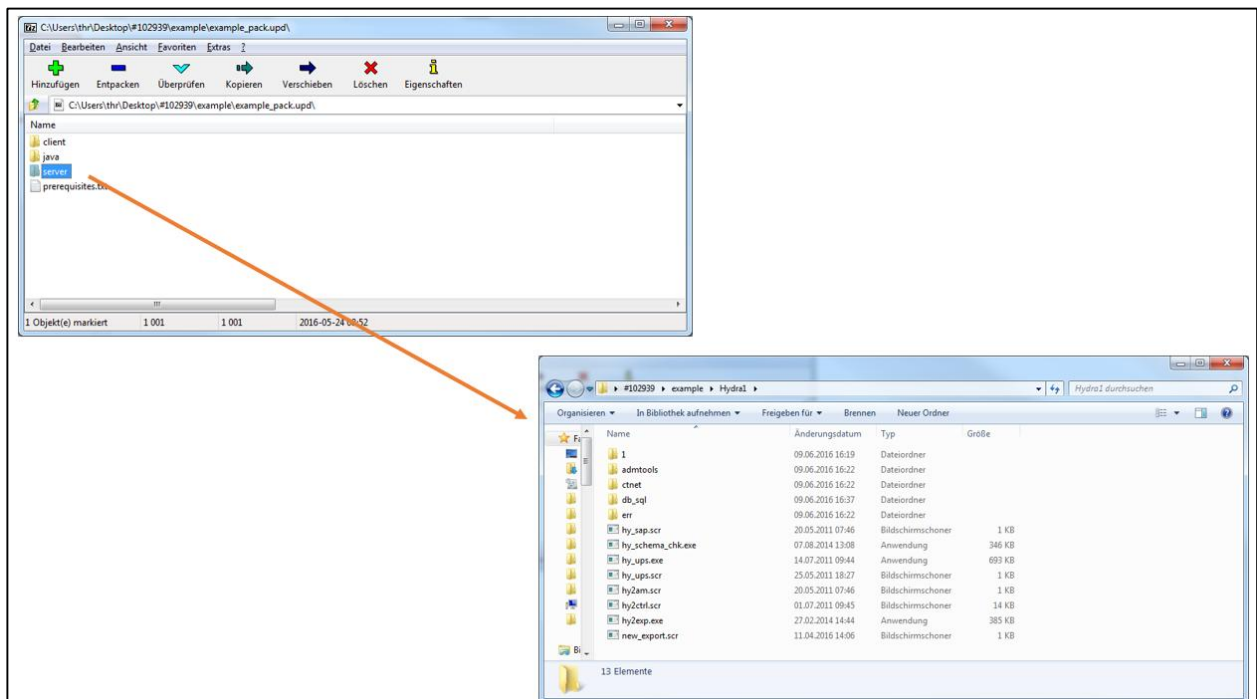


MaintenanceManager ▶ rt ▶ server ▶ MOC ▶ jhydra-inst ▶ listInterpreter ▶		
ür ▼ Brennen Neuer Ordner		
Name	Änderungsdatum	Typ
custom	12.05.2016 16:08	Dateiordner
standard	10.05.2016 11:36	Dateiordner

Interpreter copied with custom scope to the runtime directory.

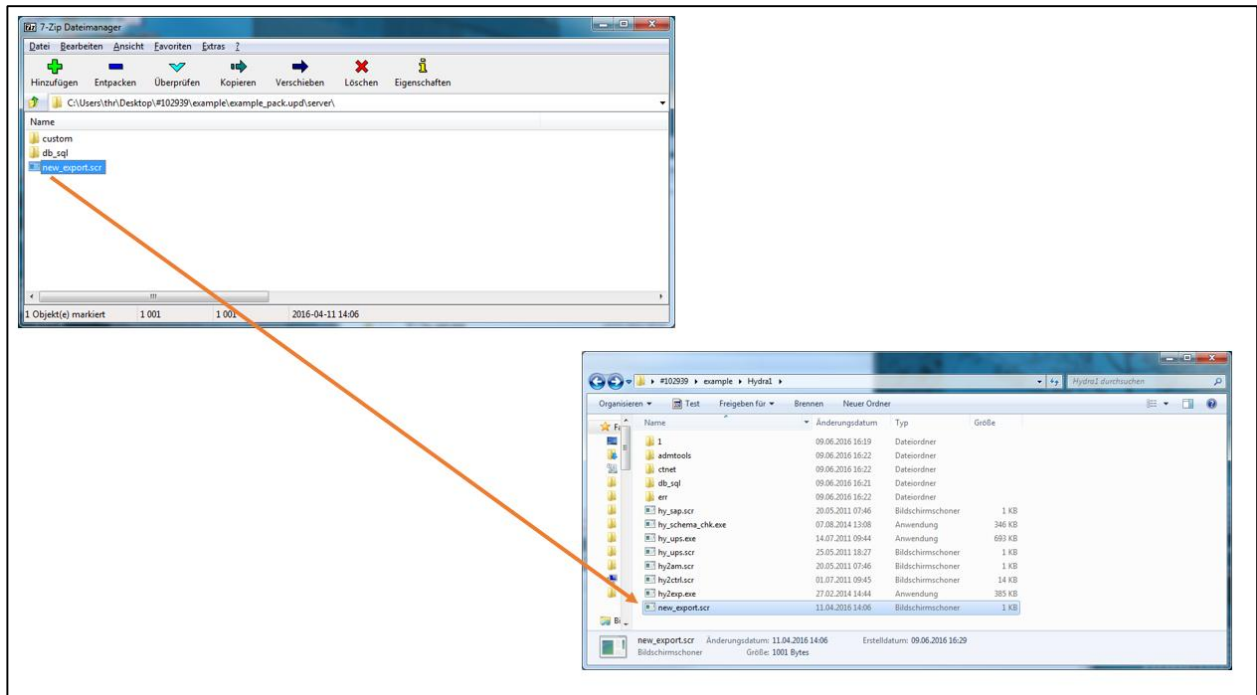
4.5 Structure of Server Package

The system copies the directory structure of the root directory of the update package one-to-one into the HYDRA directory (all subfolders of the *server* directory).

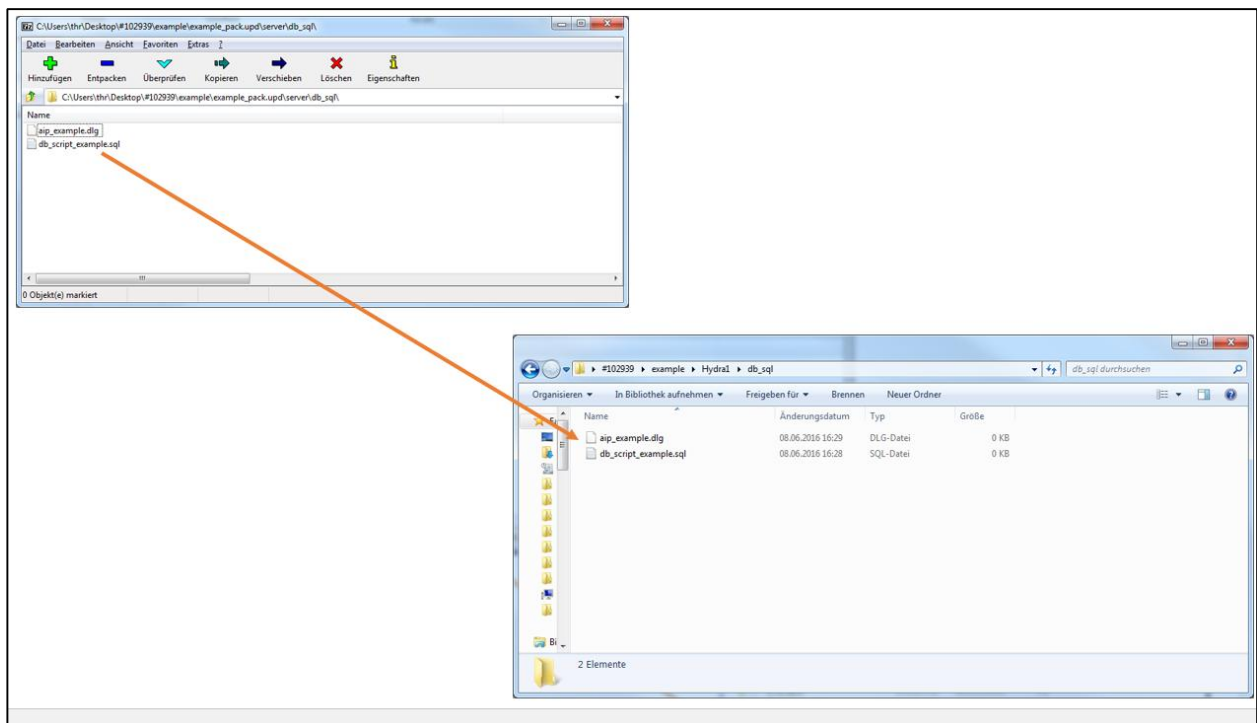


The following example shows a server update package:

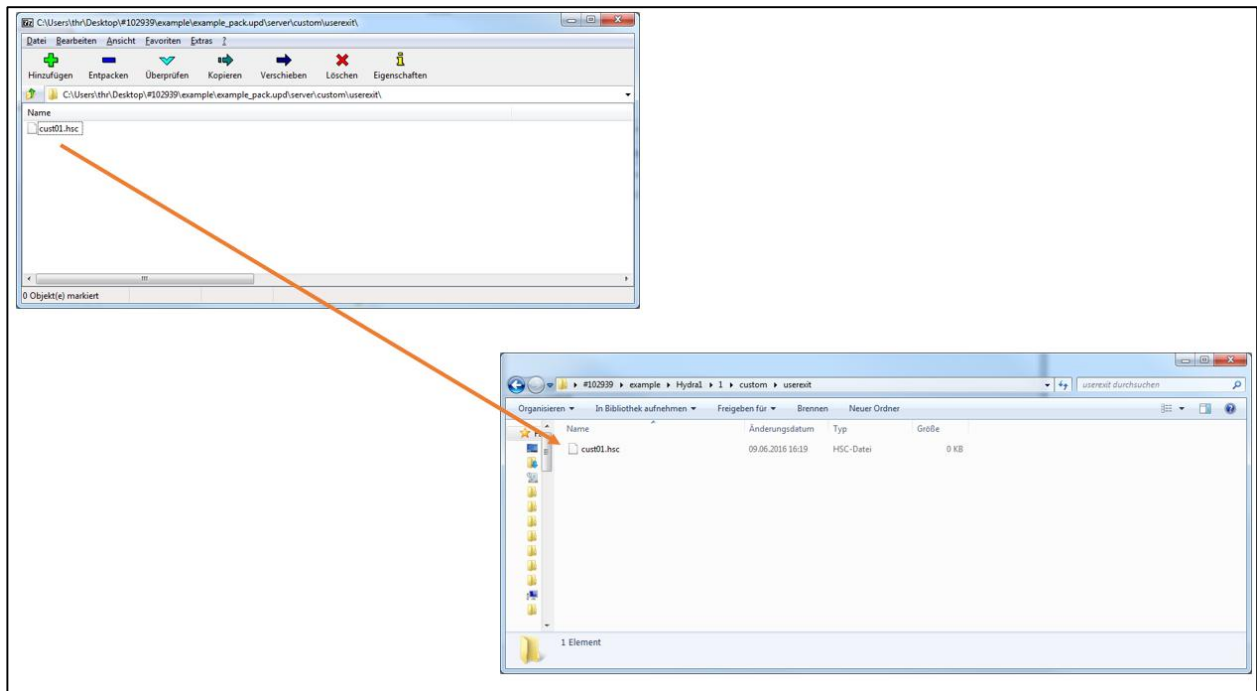
You should file server scripts (.scr), programs (.exe/.out), etc. directly in the root directory of the update package. These are filed one-to-one in the HYDRA root directory as described above.



DB patches, SQL scripts, SQL files, dialog files are filed in the subfolder *db_sql*. These are also filed one-to-one (including subfolders) in the HYDRA directory.



Customizations in the form of user exits (terminal scripts, server scripts, SVG files for the upload interface) are filed in the subfolder *custom/userexit*.



Further examples:

Label design: Reports are filed in *custom/reports* (.ll / .qr3 files).

Terminals: Customer-specific INI files are stored in *custom/aip* or *custom/aip2*.

Customer-specific language files are filed in *custom* (hycust.mld).

Note:

The directory structure in the update package must correspond to the HYDRA directory structure in the server without leading instance number (i.e. in the update package, the path is */custom/userexit* instead of *1/custom/userexit*).

5 Reports on the basis of Crystal Reports

For each application in MOC, any number of reports may be defined and created. A report is opened via the toolbar of an application and uses precisely one data source of this application. Data in this data source are then represented in the report.

This section describes the integration of reports created by the Crystal Reports Designer.



This section has only been kept for compatibility reasons. We recommend creating reports using the integrated Report Designer.

Data Source Export

In order to be able to create a report in Crystal Reports Designer, the general structure of the data to be displayed has to be known. Only then may the location and manner of display of data in the later report be defined.

For this purpose, the data of the requested data source are exported for use in Designer. This takes place from MOC during the run time. The Customizing Mode must be activated for this purpose.

At first, you open the application for which the report is to be created and request the relevant data. The data sources defined for the application are filled in this process (if data meeting the search criteria are available).

The data from a selected data source of the application may be exported by using the toolbar button *Export data sources*. A list of data sources defined for this application is opened.

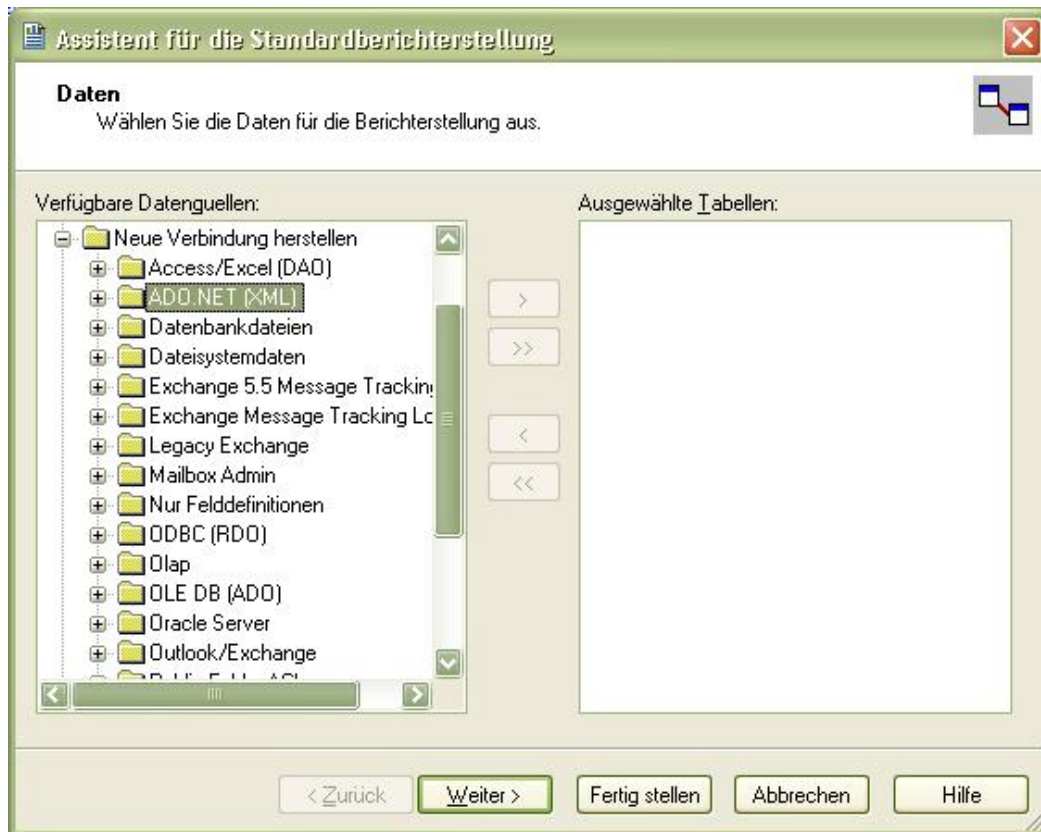
The information in brackets refers to the number of data records currently included in the data source. It is reasonable to export data sources containing a minimum of one data record, only.

By using *Export*, two files, each, are exported, which only serve as an aid to design the future report.

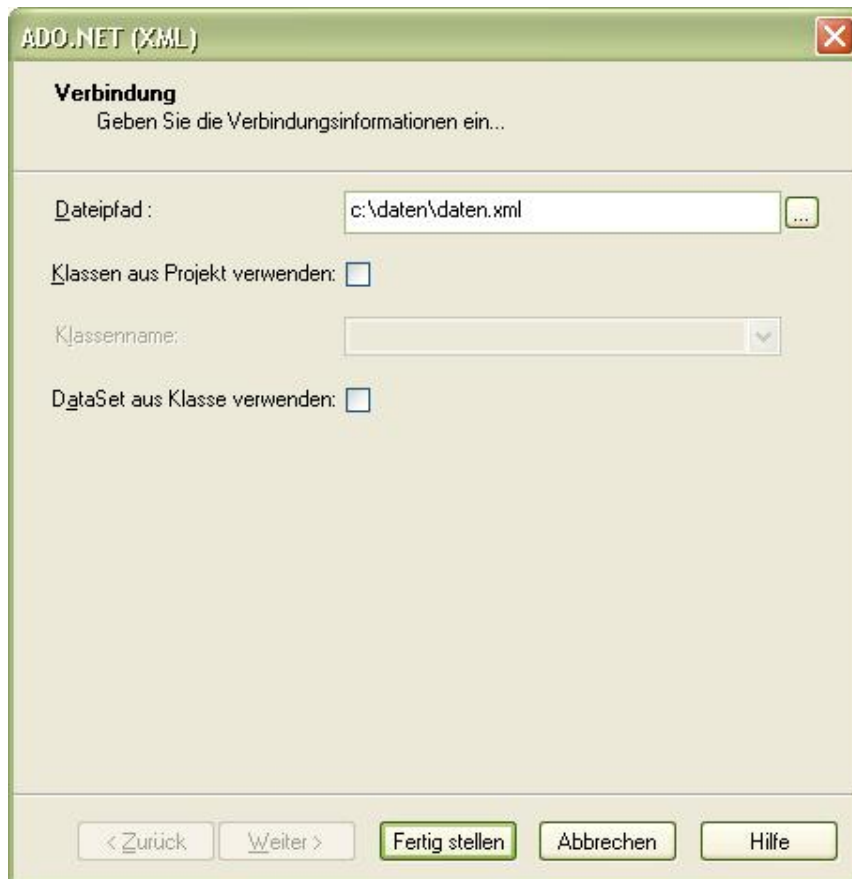
- 1 schema file (*.xsd): Data source structure
- 1 sml file (*.xml): Data source contents

Report Creation/Design

Reports are created or edited in Crystal Reports Designer. It is recommended to use the assistant when creating a new report (File - New - Standard report). The following dialog will open:



By *Establish new connection* → ADO.NET (XML), an exported file may be defined as data source for a new report.



It is recommended to use the xml file as data source. The report may be designed precisely by using data in the preview mode. At this point, however, it must absolutely be observed that Crystal reports will embed the data of the data source into the report document by default. If the report is to be transferred to customers, this default setting should be deactivated via *File - Report options - Embed data in report*. Non-customer-specific dummy data are to be used.

The actual design of the report is not the object of this documentation.

Add Report to Application

After opening the application into which the report is to be integrated, data must be requested first. Via *Edit reports*, the appropriate dialog where the list of already defined reports for this application is displayed may be opened.

The *Report file* may be selected freely. In *Template file*, the path to the report file to be integrated must be entered and/or selected in the file dialog.

By clicking the button *Analyze report file*, the report file is investigated in the background. This process may take several seconds. All main and secondary reports of the selected report are identified.

At this point, data sources must be allocated for the reports found in the Report file (main report and secondary reports, if any) (The XML file and/or XSD file is only used as an aid to create the report).

In the selection boxes, the data sources are allocated to the main report and the secondary reports, if any. The checkbox may be used to determine whether only marked data records are displayed in the report.

Create Report Call in Toolbar

The context menu of the toolbar of an application (right click on mouse - Configuration) may be used to open the link editor. Using *New* will add a new button to the toolbar.

"ShowReport" must be entered as *Function* of the new button. The configured name of the report from 0 (not the name of the report file) must be entered as *Parameter*. All remaining information may be selected freely.

6 The Repository

6.1 Overview

The repository defines and describes the interface between client and server. The repository describes the input parameters as well as the result sets of service requests.

For the server, the configuration of services in the repository defines not only the service interface in case of specific service types, but also the behavior and the processing of data. Only in exceptional cases, a real programming in the server is therefore required.

For the client, the repository defines how the data is displayed on the client and which GUI elements are used to enter data. It is also defined how the client checks the user input. A lot of applications in the client are therefore based on the repository configurations and an additional programming in the client is not required.

The repository data is grouped and structured using **domains**. A domain summarizes all data that logically belong to an application. For each application, a different domain is generated. You can globally use the defined services and client attributes at a later point in time. For example, a client application in its own domain can use a service of a different domain.

A domain contains hierarchically structured and typed data. A domain includes:

- services and service parameters including their appropriate customizing settings for the GUI,
- properties
- authorizations,
- ReferenceData and
- ControlDataSources.

In the following, the attributes of these data types are explained in detail.

6.2 Domain

Domains have properties and provide services within the domain context.

Name

Each domain has a unique name. For the name, you use the notation "UpperCamelCase".

6.3 Service

Services have transfer parameters and return values which directly correspond to domain properties in many cases.

6.3.1 Name

Name of a service. The service name usually consists of the domain name that includes the service and the **function**, separated by a dot.

6.3.2 Function

This field describes the requested service function. Typical functions are list, update, insert, delete, new, ...

6.3.3 ServiceType

There are several service types.

InterpretedJavaService2: Services of this type are used to display lists and evaluations. The services are interpreted at runtime using repository data. Contrary to the *InterpretedJavaService*, the services of type *InterpretedJavaService2* are prepared to stream data and provide more elegant options for Java user exits.

InterpretedJavaService (*obsolete*): Services of this type are interpreted at runtime using repository data. These services have been replaced with the service type *InterpretedJavaService2*.

InterpretedBAPIService: Services of this type are used to edit data. The services are interpreted at runtime using repository data.

ExternalJavaService: Services of this type are completely implemented in Java. Using these services, you can implement lists or editing functions. You use these services if the possibilities of the interpreting service types are not sufficient and the logic must be converted into Java programming.

InterpretedWrapper: Services of this type are interpreted at runtime using the repository data. The service is implemented as wrapper of an existing PDM dialog and is therefore subject to specific limitations, e.g. it does not support any dynamic Where.

Wrapper (*obsolete*): Services of this type are programmed and wrap an existing BAPI function. They are therefore subject to specific limitations, e.g. no dynamic Where.

JavaService (*obsolete*): Services of this type are completely implemented in Java.

Recommendation:

- The type *InterpretedJavaService2* is recommended for services that you use to read data.
- The type *InterpretedBAPIService* is recommended for services that you use to write data.

- If the interpreted service types cannot meet the requirements (or only with great effort) even if they include Java user exits, you should use the services implemented in Java of type *ExternalJavaService*.
- The other service types are older technologies and should not be used for new developments.

6.3.4 ListMode

For services of type *Wrapper* or *InterpretedWrapper*: This column must be completed for each service. The column specifies whether the requested PDM dialog supplies a file as result or whether it is only a return string. "Y" => The result is a file, otherwise only a string.

6.3.5 DLG

For all services of **ServiceType** *InterpretedWrapper* or *Wrapper*, you must fill in either **DLG** or **SystemCall**. You fill in DLG, if **ServiceType** is *Wrapper* or *InterpretedWrapper* and if the service requests a PDM dialog with the structure "DLG=<content in this column>|..."

6.3.6 SystemCall

For all services of **ServiceType** *InterpretedWrapper* or *Wrapper*, you must fill in either **DLG** or **SystemCall**. Fill in SystemCall, if the you want to run a program in the server. In the column, the name of the external program is specified. The result is a PDM dialog with the structure: "DLG=SYSTEM.CALL|PROG=<content of this column>|..."

6.4 ServiceGui

The ServiceGui data define the use and the presentation of the services on a client. You can clearly allocate the ServiceGui to a service via their name.

6.4.1 Name

The name of the service for which this data record provides presentation information.

6.4.2 Package

This field is obsolete and must be left empty.

6.4.3 Extended

This field is obsolete and must be left empty.

6.4.4 AdditionalDataLogics

This field is obsolete and must be left empty.

6.4.5 ApplicationID

Application ID used for generating applications in the client. In case of editing applications, the ApplicationID is edited with the main data source of the application that you want to generate.

6.4.6 ApplicationTitle

Language key for the title of the generated application. In case of editing applications, the ApplicationTitle is edited with the main data source of the application that you want to generate.

6.4.7 ApplicationHelpFile

File name of help file (including file extension) of the generated applications. In case of editing applications, the ApplicationHelpFile is edited with the main data source of the application that you want to generate.

The name of the help file should be independent of the technology of a used client. The client should therefore put a prefix in front of the file name. You can then design the help file displayed according to the client's technology.

Example for the client MOC: In ApplicationHelpFile, you enter "Article.pdf". The client MOC then loads the document "MOC_Article.pdf" as online help. The client automatically uses the prefix "MOC_".

6.4.8 ApplicationHelpIndex

Bookmark that is activated when Help is opened. In the main application it is usually "Overview". You must only edit this bookmark for the main data source of the application that you want to generate.

6.4.9 Description

6.4.9.1 General

Language key for short description of service.

You can show this description on the client when the selection of services is displayed.

6.4.9.2 Processing in the MOC client

The MOC shows the description if you add a data source while configuring an application.

6.5 ServiceParameter

ServiceParameters specify the parameters of a service. They provide information on the data source and value ranges.

The service parameters include selection criteria and the columns of the result set. A service parameter can be a selection criterion or be included in the result set. If a service parameter is used as selection criterion and/or is included in the result set, is specified via the attributes described in the following.

6.5.1 Acronym

Name of the parameter. The combination of **Acronym** and **ResultSet** must be unique for each service.

6.5.2 ResultSet

If the associated service returns more than one **ResultSet**, a name must be indicated here. This way, you can return results in parallel that have been calculated at the same time but have a different structure. The combination of **Acronym** and **ResultSet** must be unique for each service.

6.5.3 WebServiceType

Data type of the parameter (*decimal, integer, string, boolean, binary, datetime*). This value must be identical to the configured value of the property configuration. **IMPORTANT:** *binary* parameters are not supported by default. You can only use these parameters in user exits.

6.5.4 DefaultValue

Specifies a service-specific default value for a parameter.

6.5.5 IsResult

Specifies whether this service parameter is part of the ResultSet (return value). If you want to use the **DefaultValue**, do not set this field (IsResult).

In case of services of type *InterpretedWrapper*, you must only set the column IsResult to "Y" for UPDATE, LOCK, UNLOCK, DELETE, INSERT and COPY, if the BAPI actually returns a value, e.g. a new internal_id when you create new data records.

6.5.6 IsDynamicResult

Required for the generation of the Java function (for dynamic ResultSets, the column number must automatically be extended to the fixed number). Missing columns are added as empty columns (i.e. these columns are not computed).

6.5.7 InputAsArray

The client must transfer values in form of an array. **InputAsArray** is only reasonable in case of a quantity input parameter, i.e. if at least one of the two columns, **IsSpecialParameter** and **IsFilterParameter**, is set and a quantity operator such as BETWEEN or IN is possible.

Specify if a field is an array or not (with filters always yes except for Boolean type). If true and no array or empty, then exception. Is currently only verified in case of mandatory special parameters.

6.5.8 IsSpecialParameter

Specifies whether or not the parameter is a special type controlling the service functionality (i.e. is not a filter parameter). For the **ServiceType Wrapper**, this is the only possible parameter type. In case of the **ServiceType JavaService**, it represents a special parameter not directly included in the WHERE condition but with different "controlling" effects. If you want to use the *Default Value* on the server side, do not set this field. In addition to the defined special parameters of standard processing, you can also use other special parameters in user exits.

6.5.9 IsFilterParameter

Specifies whether it is a filter parameter. If you want to use the **DefaultValue** on the server side, do not set this field.

6.5.10 IsMandatory

Specifies whether it is a mandatory parameter for the service. If true and parameter is missing, an exception is thrown. Is currently only checked for special parameters.

6.5.11 Can* (Filter) Operators

This option specifies whether the service supports the relevant filter operator for this parameter. Set the "Can*" fields for filter parameters.

Available operators:

- **CanEqual**
- **CanLike**
- **CanBetween**
- **CanIn**
- **CanNotEqual**
- **CanLt (Can Less Than)**

- **CanLte (Can Less Than or Equal To)**
- **CanGt (Can Greater Than)**
- **CanGte (Can Greater Than or Equal To)**

For technical reasons, each operator has a second operator that you should select is a data record must be selected, if the operator is applicable or if the comparative value is NULL. The operator **CanEqual** will only return a data record in case of equal values, **CanEqualOrNull** in case of equal values or if the data record value is NULL. Accordingly, there are the following operators:

- **CanEqualOrNull**
- **CanLikeOrNull**
- **CanBetweenOrNull**
- **CanInOrNull**
- **CanNotEqualOrNull**
- **CanLtOrNull**
- **CanLteOrNull**
- **CanGtOrNull**
- **CanGteOrNull**

Especially with List Services you should make sure that generally all parameters support all operators in order to achieve the highest possible selectivity. In general, the framework supports this for Java services.

- You may only set **CanIn**, **CanBetween**, **CanBetweenOrNull** and **CanInOrNull**, if **InputAsArray** is also set.
- **CanLike** is only useful if the **WebServiceType** is *string*.
- With **WebServiceType** boolean, only **CanEqual** is useful.
- With **WebServiceType** string, all operators are possible.
- With all other types, all operators except for **CanLike** and **CanLikeOrNull** are useful.

Before you set wrappers, you must check which operators are actually supported by the PDM dialog or the system command.

6.5.12 HydraAcronym

With service type *InterpretedWrapper*, the HYDRA acronym is specified.

6.5.13 HydraResultAcronym

If the acronym of the selection criterion is different to the acronym in the result file, you can enter an acronym that is different to the HydraAcronym for the service type *InterpretedWrapper* and ListMode=Y.

6.5.14 TransferEmptyValuesToHydra

Specifies whether blank values, too, are to be transferred to HYDRA, or whether the ID is simply omitted. "Y" => blank values are transferred, otherwise => ID is completely omitted.

Note: You must set this field for Insert and Update (editing screens). Only then, you can enter blank values and/or overwrite existing values with blank values.

6.5.15 HydraShiftPart

Along with the **Reference** field, the following components are marked as related: Start of shift date, Start of shift time, End of shift, End of shift time stamp, Start of shift time stamp. The column "HydraShiftPart" can include the following values:

- beginDate
- beginTime
- beginDatetime
- endTime
- endDatetime

Important: The column can only be completed if the parameter is part of a group that includes the following five components:

- Start of shift date,
- Start of shift time,
- End of shift,
- End of shift time stamp,
- Start of shift time stamp.

You must not complete the column, if it is only a group of three components including Date, Time and Date + Time field. In this case, only complete the **Reference** column.

6.5.16 Reference

Is used to generate a DateTime data type from (Hydra) Date and (Hydra) Time data types and/or to identify the shift parameters.

6.5.17 TransformationType

Use this field to specify transformations for input and result parameters for List Services/wrappers (e.g. convert Bool to J/N and vice-versa or correct filtering for DateTime fields that consist of two fields in the database). For further details on this field, refer to section 6.10.

6.5.18 PlugName

Specifies whether the result parameter for this service is directly derived from the specified DataObject or whether it is added to the DataObject via plug.

Example:

Service A.List uses a plug of service B.List in the service parameter b. Consequently, the following configuration applies to service A.List:

ServiceParameter	DataObjectName	PlugName
a	A.List	
b	A.List	B.List

If the field *PlugName* includes a value, the Interpreter replaces the values of the ServiceParameter with those values of the plugged service when creating the SQL statement.

In the special case where an interpreted List Service does not use an own table but only plugs, and subsequently adds fields via user exit, these fields should state USEREXIT!



If you create new services, it is recommended to avoid plugs and to provide data directly via the DataObject via Join. Dependencies between several services are thus avoided.

6.5.19 DBField

Database field that you use to make a selection. Write the database field in lower case. You can either enter simply the field name or (for complex expressions) the expression with placeholders for the alias (e.g. `hydadm.get_datetime(%1$s.bearb_date,%1$s.bearb_time)` or `{fn substring(%1$s.field,2,1)}`).

Proceed as follows for joins to other tables:

Entry: `<ALIAS>.<DBfield>`

The entry will be described in more detail in the SD documentation.

Example:

DB field: `STA1.status_bez`

SD:

Acronym: `gage.status.designation`

Table: `caq_status (STA1)`

Conditions: status_typ = 'PMSTATUS', status_nr = status

6.5.20 DBAlias

The alias for the table that is used to select the value for the acronym.

6.5.21 DBTable

The table that is used to select the value for the acronym.

6.5.22 DBFieldAlternative

If you cannot use the **DBField** because the **ConditionalFieldKey** is not applicable, you use the **DBFieldAlternative**.

You can enter a number, 'null', 'string', {fn ...} or another field / subselect. If it is another field or subselect, you MUST enter %1\$s for the alias of the table.

If **DBFieldAlternative** is empty, but you require an alternative field, NULL is selected.

6.5.23 DataObjectName

If a service uses several data sources to identify its data, you can store the data source (= DataObject = DO) that issues the result parameter in this field. For example: A service includes the parameters a, b and c:

- a is computed,
- b is identified using data object (DO) F and
- c is identified using data object (DO) G.

For a: the field is blank. For b: the field contains F. For c: the field contains G. Is used as reference for the ...do.xml configuration.

6.5.24 ConditionalFieldKey

This field specifies if a DB field is only conditionally available. The ConfigurationManager checks the condition for the existence of the field. Enter the feature key of the Configuration Manager (**feature set**) in this repository field to enable the check.



If a parameter is a conditional field and the condition is not fulfilled, the entries for the MOC acronym are removed from the ComplexSelectMap and the SpecialFilterMap.

As a result, the changes in the Special Filter Map via user exits and transformation type are

also lost!

6.5.25 Constraints

Constraints are processing parameters that are used for **ServiceType** *InterpretedBAPIService*. Constraints are structured as keys with optional values. The separator between keys is the pipe character (|). You use a semicolon to separate various values. Key and value are separated from each other using the equal sign (=). The general structure is as follows:

```
Key1=Value;Value;Value|Key2|Key3=Value|
```

The following constraints are available:

Constraint Key	Constraint value(s)	Description
KEY	exactly one number between 1 and 5	Define field as key including key number for hyd_lock table
SERIAL	none	Field is a SERIAL (and/or auto-increment)
SEP_DATETIME	1st parameter refers to the date field 2nd parameter refers to the time field	Allows processing of separate date and time fields
BOOL	1st parameter is the value to be entered into the DB if true 2nd parameter is the value to be entered into the DB if false 3rd parameter is the value to be entered into the DB if null (null for null) 4th parameter is the type of DB field, e.g. BOOL=J;N;null string BOOL=1;0;null;integer	Use this to write Boolean values into a string or Integer Field.
MODIFY_TS	None	
MODIFY_BY	None	
CREATE_TS	None	
CREATE_BY	None	

6.6 ServiceParameterGui

The ServiceParameterGui define how ServiceParameters are displayed on the client. Use **Acronym** and **ResultSet** to clearly allocate ServiceParameterGui to a service parameter.

A number of settings exist in the `ServiceParameterGui` and in the properties. You normally use the properties to define how data is displayed on the client. Only if you want to display specific services on the client in a way that is different to the settings in the properties, the respective field in the `ServiceParameterGui` is completed. The `ServiceParameterGui` fields overwrite the property fields of the same name.

6.6.1 Acronym

Name of the parameter for which this data record provides presentation information. There must be a corresponding property for each **acronym** of a parameter.

6.6.2 ResultSet

See **ResultSet** with `ServiceParameter`.

6.6.3 Label

6.6.3.1 General

The label includes a language key. Using this language key, the parameter on the user interface is identified (by default), e.g. as label text of a column header. Overwrites the value from the property configuration.

6.6.3.2 Processing in the MOC client

The label is displayed as label text of a field or a column title.

6.6.4 Tooltip

Specifies a specific tooltip for the parameter in the service context. Entry as language key.

6.6.5 FormatType

Use this field to overwrite specific values of a property in relation to the service (currently **Label**, **Length**, **ControlType**, **ControlTypeMode**, **ControlDataSource**, **ControlDataSourceMode**, **ControlDataSourceResult**).

For example: If you enter *workplace.id* as **FormatType** for the parameter *resource.id*, you can define for the parameter to be a *resource.id* in this service, however its length, label and control properties are taken from *workplace.id*.

In this case (other than in case of semantic and syntactic types), the value from **FormatType** takes priority. For this reason, we have a new hierarchy:

- Value from **FormatType**
- Value from ServiceParameterGUI
- Value from Property
- Value from **SemanticType**
- Value from **SyntacticType**

6.6.6 ClientDefaultValue

Input fields have a **ClientDefaultValue** property. The value entered here is displayed as default value when the control is initialized. "From" and "to" values are separated by semicolons.

Set checkbox: If the value of this field is set to *true* during a CheckEdit, the checkbox is set after initializing.

Preallocation of text fields with "from" and "to" values (InputAsArray): set value1;value2 to prepopulate the 'from' and 'to' fields during a text edit.

Date fields: In case of date fields, the field can be preallocated with an offset. If you set default values for date fields, you definitely must indicate the type of offset. The following offsets are possible:

- h (hours)
- d (days)
- w (weeks)
- m (months)
- y (years)

The 'to' value is always **relative to the 'from' value**. The default value is always a DateTime object. The presentation depends on the output format of the relevant field.

You can put "[" and "]" in front and at the end of the relevant value to specify the start and end of a period of time. Consequently, e.g. "[0d;0d]" means that 0:00:00 is entered in the 'from' field today and 23:59:59 is entered in the 'to' field today. "[-1w;0w]" means from Monday last week up to Sunday last week.

Examples:

Current date:

0d

From today to the day after tomorrow:

0d;2d

From today to one week from today:

0d;1w

From yesterday to tomorrow:

-1d;2d

From one year ago today to one year from today:

-1y;2y

Year shortlists: You can configure a year shortlist by **ControlDataSource** = YearList and **ControlDataSourceMode** = Script, or even by standard "Service-ControlDataSource". In this case, you can use the following default values:

- Current year: 0y and/or currentyear
- Last year: -1y
- Following year: 1y
- 4 years ago: -4y
- Year that was current 10 months ago: y-10m → this is mostly the case when the relevant year field is used in combination with a month shortlist.

If you want to preallocate two fields (**ShowSecondControl**), you have to separate both values by semicolon, e.g. -1y;1y

Month shortlists: You can use the following default values for a month shortlist:

- Current month: 0m
- Last month: -1m
- Following month: 1m
- 4 months ago: -4m

If you want to preallocate two fields (**ShowSecondControl**), you have to separate both values by semicolon, e.g. -1y;1y.

6.6.7 IsKey

The IsKey column is very important and should be occupied for all key columns of a service, since otherwise data records cannot be clearly identified. Columns including the value 'null' may NOT be defined as keys. The IsKey columns should be identical for all services (insert, update, delete, lock, unlock, copy). These entries are important, so it is best to verify them twice.

This field specifies the positioning of the cursor after an editing operation. If the positioning option OnKeyValue is selected, the client should only request one new data row after editing. You also use values that are marked **IsKey** as selection criteria. **IsKey** must also be set for *delete*, since this data record must be deleted from the view.

IsKey must also be indicated for *list*. If no sorting is given in the *list*, sorting takes place according to **IsKey** fields.

Every parameter which is **IsKey** MUST always be **IsMandatory**. This rule has two exceptions:

- List service
- Wrappers with composed keys.

6.6.8 ShowInGrid

Specifies whether the parameter is to be displayed in tables by default.

6.6.9 ShowInDetail

Specifies whether the parameter is to be displayed in detail views by default.

6.6.10 ShowInSearch

Specifies if the parameter is to be used as selection criterion (i.e. in selection panels) by default.

6.6.11 ColumnCategory

6.6.11.1 General

In the tabular view, the client should provide the option to summarize the columns in the table to categories. You specify a language key that is displayed as title of the summarized columns.

6.6.11.2 Processing in the MOC client

The ColumnCategory is used to assign the parameter to a "strip" in the grid (table view).

6.6.12 Category1, Category2, Category3

6.6.12.1 General

The client processes the columns Category1, Category2, Category3 in order to group fields in applications. The grouping can be performed via tabs or frames for a group of fields. You specify a language key that is displayed as title or label text of the grouped elements.

6.6.12.2 Processing in the MOC client

Category1: Assigns the parameter to a tab in the detail view.

Category2: Grouping options for detail screens.

Category3: Currently not used.

6.6.13 TabOrder

You specify the order of tabs for detail views.

6.6.14 ColmnOrder

You specify the order of columns in tabular views.

6.6.15 ShowSecondControllnSearch

6.6.15.1 General

You specify if a second control is displayed (from/to). You normally use this setting with selection criteria that include a value range via the operator CanBetween, e.g. "date from/to".

6.6.15.2 Processing in the MOC client

The MOC provides two adjoining fields. The label text of the second field is automatically "to". If it is a field of "date" type, you can predefine a relative date for both fields.

6.6.16 SearchTabOrder

Specifies the tab sequence for the selection panel.

6.6.17 SearchCategory1, SearchCategory2

6.6.17.1 General

The client processes the columns SearchCategory1 and SearchCategory2 in order to group fields in selection panels. The grouping can be performed via tabs or frames for a group of fields. You specify a language key that is displayed as title or label text of the grouped elements.

6.6.17.2 Processing in the MOC client

SearchCategory1: You allocate the parameter to a tab in the selection panel.

SearchCategory2: Grouping options for the selection panel.

6.6.18 ControlType

Use the ControlType to specify which control should be used for the relevant parameter. The client will map the abstract type onto a specific control class. If you do not specify a type, the client uses the data type to decide on the ControlType. Possible values for the ControlType:

CheckEdit: Selects a Boolean value (true/false) or multiple values if a reference to a data source is given.

ColorEdit: Selects a color value.

ComboBoxEdit: Combobox with selection of values from web service or data reference.

DateTimeEdit: Enter a date and/or a time.

MemoEdit: Enter an arbitrary text.

RadioGroup: Selects a Boolean value (true/false) or one of multiple values if a reference to a data source is given.

TextEdit: Standard text input. You can add a button opening a search dialog to this control, if you add a reference to a service in **ControlDataSource**. If you enter the name of a DataLogic in **ControlParameter** and if a mapping is included in **ControlDataSourceResult**, data will be requested upon leaving the control and return values will be mapped appropriately.

6.6.19 ControlTypeMode

6.6.19.1 General

Allows for controlling the input control.

CheckEdit: DualState (default), TriState, J;N;J (checked;unchecked;tristate)

ColorEdit: none

ComboBoxEdit: SingleEdit, Single, Multiple (multiple selection)

DateTimeEdit: Date (date display), Time (time display), DateTime, RelativeDate, RelativeDateTime

MemoEdit: none .

RadioGroup: SingleColumn, SingleRow

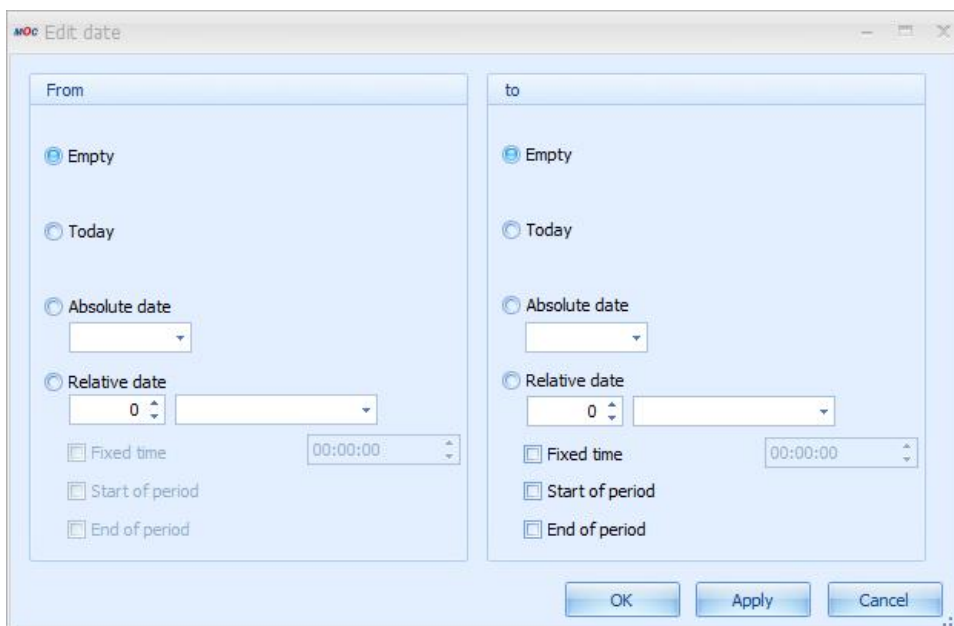
TextEdit:

- Empty: the search button is shown if a **ControlDataSource** is defined.
- "SearchButton": Search button is shown.
- "SearchButtonValidate": Search button is shown. If you enter an invalid value, an error is displayed.
- "OpenFileDialog": opens a file selection dialog.

6.6.19.2 Processing in the MOC client

If you use **DateTimeEdit** including the definition of a relative date (**ControlTypeMode**: RelativeDate or RelativeDateTime), you can enter a relative date.

If **ShowSecondControl** = true, you can predefine the complete relative value range. In this case, a button is displayed behind the second input control. You can use this button to open the following dialog:



Use this dialog to customize the values for **ClientDefaultValue** . The following entries are possible:

- **Empty:** no value is adopted
- **Today:** the current date is adopted
- **Absolute date:** you can select a fixed date value via a calendar control
- **Relative date:** you can select and adopt a date relative to the current date. In this context, "Start of period" means that you additionally go to the start of the selected period. Example: current date is 20-MAY-2010. If you select "- 1 month", 20-APR-2010 is adopted. If you also select "Start of period", the date is changed to 01-APR-2010. The same applies to "End of period". These settings are saved in the mpdvEdit or the selection profiles as **ClientDefaultValue**.

6.6.20 ControlParameter

See **ControlType** → **TextEdit**

6.6.21 ControlDataSource

Data source for the selection of values. The data source can be:

- **Web service** (configuration: **ControlType** = *ComboBoxEdit*, **ControlDataSourceMode** = *Lookup*, **ControlDataSource** = **Name** of a **ControlDataSource**. See also section "6.8 ControlDataSource")
- **ReferenceData** (configuration: **ControlType** = *ComboBoxEdit*, **ControlDataSourceMode** = *Reference*, **ControlDataSource** = **Type** of **ReferenceData**)
- **Search application** (configuration: **ControlType** = *TextEdit*, **ControlDataSourceMode** = *Lookup*, **ControlDataSource** = application name)
- **Script** (configuration: **ControlType** = *ComboBoxEdit*, **ControlDataSourceMode** = *Script*, **ControlDataSource** = Name of script)

6.6.22 ControlDataSourceMode

Data source mode (*Lookup*, *Reference* or *Script*).

6.6.23 ControlDataSourceParameter

Optional setting of parameters of a **ControlDataSource**. If you make settings here, these settings overwrite the settings in the **ControlDataSource**.

See also the description **ControlDataSource - Parameter**

6.6.24 ControlDataSourceResult

Optional setting of the result of a **ControlDataSource**. If you make settings here, these settings overwrite the settings in the **ControlDataSource**.

The settings in this field provide more options than the **Result** in the **ControlDataSource**:

Result columns are separated by semicolon. Field mapping:

- First entry: Value
- Second entry: Labeling
- Third entry: UnitLabel
- As of the fourth entry, the fields are mapped:
 - o Via acronym or semantic type.
 - o Field mapping: in **ControlDataSourceResult**, you can enter a mapping in the form "FieldName=ColumnFromResult" as from the fourth entry. For example, you can specify tool.id=resource.id in order to fill the field "tools.id" with the "resource.id" value from the search application. Several mappings are separated by ";" - spaces are not allowed.
 - o Asterisk mapping: Instead of mapping, you can also enter * . Subsequently, all return columns of the search application are mapped. The mapping is performed as usual via ID or semantic type.

6.6.25 VisibleCondition

Visibility condition. For customization, see EditableCondition.

6.6.26 EditableCondition

This value decides whether a parameter is editable. There are three possibilities:

- Boolean value: In case of *TRUE* or *FALSE*, the field is always editable / non-editable.
- Binary expression:
 - o Field name must be the name of a field that is also located in the ControlPanel.
 - o Valid operators: =, <, >, <=, >=, <>, !=
 - o The value is written as a string and interpreted depending on the comparative field value.
 - o Field, operator and value must be separated by a space!
- Concatenation of binary expressions:
 - o You can concatenate an arbitrary number of binary expressions.
 - o Links may be achieved by &&, AND, ||, OR.
 - o Here, too, all components of the conditions must be separated by a space.
 - o Priority of operators: AND or && are evaluated first, then OR and ||. Parentheses are not allowed.
 - o Example: resource.id = 12345 && resource.costcenter = 20 || resource.id = 60610

6.6.27 ScriptId

6.6.27.1 General

The ID of the script that is allocated to the parameter. If you set the ID, the relevant script is performed upon various events (at present EditValueChanged and Leave).

6.6.27.2 Processing in the MOC client

The method name of the script is ScriptId+EditValueChanged and/or ScriptId+Leave. The script can be included in any DLL that is read by the CodeManager.

6.7 Property

For the acronyms, properties include information on data types, input and output formats, display options, a name (that can be localized) and other settings specifying how ServiceParameters are displayed in the client. Each property has a system-wide unique acronym.

A number of settings exist in the ServiceParameterGui and in the properties. You normally use the properties to define how data is displayed on the client. Only if you want to display specific services on the client in a way that is different to the settings in the properties, the respective field in the ServiceParameterGui is completed. The ServiceParameterGui fields overwrite the property fields of the same name.

6.7.1 Acronym

Clear identification of the property across all domains.

6.7.2 WebServiceType

Describes the data type used to transfer the property between client and server. The currently supported WebServiceTypes are exclusively

- *binary*
- *boolean*
- *datetime*
- *decimal*
- *integer*
- *string*

Important: the types **date* and **time* are internal types which are not transferred.

6.7.3 NETType

The data type used by the client. If NETType is empty, the **WebServiceType** is used to automatically identify the data type used by the client. At present, NETType supports the following entries:

- **color:** Use *color* to convert the transferred integer into an RGB code. In this case, the conversion is implemented by the grid.
- **duration:** creates a duration from an integer.
- **image:** either creates an image from a transferred byte array or interprets a transferred string as image name. For example, the *maintenance.active.led* property is transferred as a string including the name of an icon.
- **preview:** Specifies that the contents in the client may be displayed as "preview" (similar to auto-preview outlook) (application e.g. in DevExpress grid).
- **timestamp:** Use *timestamp* to automatically create an additional column for date values in the client in order to process time and date separately.

6.7.4 SemanticType

Use semantic types to pass on semantic properties. The "order.id" is therefore used to identify orders (semantic meaning). The acronym *operation.order.id* includes such an order identification and therefore has the semantic type *order.id*. If an attribute of the property is not set (empty), the respective value from the semantic type is used for the processing in the client.

For example: You must set the semantic type if you want to adopt a value from a lookup screen in the field. For the *workplace* field, enter e.g. *resource.id* as semantic type in order to adopt the selected workplace from a search screen for workplaces. Refer to the description of the SyntacticType for further information on the priority used to specify the attributes of a Property, the SemanticType and the SyntacticType.

6.7.5 SyntacticType

You mainly use a syntactic type for a uniform presentation of the different properties. The syntactic type does not include any semantic content. For example: The properties *booking.begin_ts* and *booking.shift.start_ts* have different semantic meanings, but are presented in a uniform format that can be controlled centrally.

Syntactic types are consequently used to control different characteristics of a property such as length, input and output screen, tooltip, label, etc. To select the valid value for a characteristic, the client proceeds as follows:

- If the characteristic (e.g. length) is set in Property, the client uses this value.

- Or: If a semantic type is available and the characteristic is set, the client uses this value.
- Or: If a syntactic type is available and the characteristic is set, the client uses this value.

Notes:

- You must always enter a **description** for syntactic and semantic types.
- Syntactic types can refer to syntactic types in order to receive "inheritance hierarchies".
- Create syntactic types as property of the *SyntacticType* domain.
- Semantic types are usually "real" properties of a "normal" domain that are used as semantic type at other places.

6.7.6 Label

6.7.6.1 General

The label includes a language key. Using this language key, the parameter on the user interface is identified (by default), e.g. as label text of a column header. Overwrites the value from the property configuration.

6.7.6.2 Processing in the MOC client

The label is displayed as label text of a field or a column title.

6.7.7 DefaultTooltip

Specifies the default tooltip for the property as language key.

6.7.8 UnitLabel

Text key for unit. The unit is displayed to the right of the input field.

6.7.9 OutputFormat

This field specifies the format that is used to display a value (e.g. for date or quantity values). If you do not enter an **InputFormat** in the repository, the MOC tries to develop an appropriate format from the **OutputFormat**. Enter the value **InputFormat** in the repository only if special masking is required. Find further details in section "6.7.12 Rules for the input/output formatting".

6.7.10 InputFormat

Equivalent to OutputFormat. You can enter a valid regular expression in the field InputFormat. Other entries that are not regular expressions are not permissible. Find further details in section 6.7.12.

6.7.11 Length

The client shows the control for this acronym in the specified width (i.e. the specified number of characters). With Length=0, the control uses the entire width available. If a width is specified but the space available is not sufficient, the control is cut off.

This field also specifies the number of characters that you can enter in an input field with **ControlType = TextEdit**, if no other **InputFormat** is specified.

6.7.12 Rules for the input/output formatting

Overview

In the repository, you define the formatting of the data output and the input dialogs to edit data. The "Properties" of the different acronyms include an **OutputFormat** and an **InputFormat**. The **OutputFormat** defines formatting if you display a value.

Important: If you do not enter an **InputFormat** in the repository, the MOC uses the **OutputFormat** to generate an appropriate formatting. Enter the value InputFormat in the repository only if special masking is required.



In case of strings, you cannot enter the special characters asterisk (*) and pipe (|), if you have not defined any input format. As you use these two special characters as separator and control character, they can cause problems if they are written in the database.



With strings, the maximum number of characters that you can enter is defined by the attribute **Length**, if no other input format is defined.

Syntactic types

The Properties provide so-called "syntactic types" in order to make groups (similar to field types in Delphi). Syntactic types have the same properties as real properties. The real properties have a syntactic type. For example, if the **output format** of the syntactic type includes a value, this value is used wherever this syntactic type is entered.

Example: Industrial minutes

The syntactic type "Durations" has the format {0:mpdv_timespan}. In the individual properties showing durations, "Durations" is entered in the column **SyntacticType** and no entries are made in the columns "output format" and "input format". When the property is read - and if no **output format** is available in the property - the format of the syntactic type is used.

If a system displays industrial minutes and if the syntactic type "Durations" is specified, the **Outputformat** is automatically changed from {0:mpdv_timespan} to {0:mpdv_timespan_industrialMinutes}. As a result, all formats including the syntactic type "Durations" are shown in industrial time units.

Output formats

OutputFormat	Automatically created masking (input format)	Examples	Description
Numeric data			
f(number)	None	f3, f1	Numeric value without thousands separator. The number specifies the number of decimal places.
n(number)	None	n0, n2, n5	Numeric value with thousands separator. The number specifies the number of decimal places, even if the data type to be displayed is an integer type. In case of n0, no decimal places will be shown.
#, (##) , #.(0)	None	#,####, #.0000	Arbitrary format
{0:mpdv_timespan}	None	2:33:30	MPDV format provider. Conversion of seconds to hh:mm:ss and vice-versa.
{0:mpdv_timespan_short}	None	2:33	MPDV format provider. Conversion of seconds to hh:mm and vice-versa.
{0:mpdv_timespan_minutes}	None	45	MPDV format provider. Conversion of seconds to minutes and vice-versa.
{0:mpdv_cycletime}	None	1:30:00	MPDV format provider. Conversion into seconds
{0:mpdv_te}	None	2.00	MPDV format provider. Conversion into seconds
Strings			
empty	[^*]]*		Illegal characters begin with ^. In this example * and * No limitation in length
empty	[^*]]{0.10}		Illegal characters begin with ^. Max. length: 10 characters
empty	[0-9a-fA-F]		Allowed characters 0 through 9, a through f, A through F.
Special formats			
{0:mpdv_cycletime_sec_cycle}	None	29 sec/cycle	MPDV format provider. Conversion into seconds
{0:mpdv_IndustrialMinutes}	None	1.50	MPDV format provider. Conversion into seconds

Input formats

The following definitions are available for the input format:

- Leave empty: The input format is implicitly defined using the output format.
- Use of logical input formats
- Use of regular expressions

Logical input formats

To simplify the definition of input formats and limit the variety of entries in the repository, the logical input formats are provided. These input formats are permanently implemented in the client and can directly be used in the repository. Input formats are customized in the properties. But service parameters specify whether wildcards are allowed. For this reason, the input format actually used can vary depending on the allocated service.

In order to use logical input formats, define the name of the input format in the affected property in the repository. The following formats are currently available:

Name	Input format without wildcard	Input format with wildcard
CHARACTER	[^*][LENGTH]	[^][LENGTH]
NUMBER_N0	[0-9][LENGTH]	[0-9][LENGTH]
NUMBER_N1	[0-9][LENGTH]\R.[0-9]{0,1}	[0-9][LENGTH]\R.[0-9]{0,1}
NUMBER_N2	[0-9][LENGTH]\R.[0-9]{0,2}	[0-9][LENGTH]\R.[0-9]{0,2}
NUMBER_N3	[0-9][LENGTH]\R.[0-9]{0,3}	[0-9][LENGTH]\R.[0-9]{0,3}
NUMBER_N6	[0-9][LENGTH]\R.[0-9]{0,6}	[0-9][LENGTH]\R.[0-9]{0,6}
TIMESPAN_SHORT	[0-9][LENGTH]\R:[0-9]{2,2}	[0-9][LENGTH]\R:[0-9]{2,2}
TIMESPAN	[0-9][LENGTH]\R:[0-9]{2,2}\R:[0-9]{2,2}	[0-9][LENGTH]\R:[0-9]{2,2}\R:[0-9]{2,2}
ORDER	[0-9a-zA-Z.+][LENGTH]	[0-9a-zA-Z.+*][LENGTH]

The placeholder [LENGTH] is replaced with the defined length at runtime. If the defined length is '0', an '*' is entered.

Input/Output formats with calculation

If you specify the **output format** *mpdv_calc*, you can include calculations in the formatting. In this format, you can specify a divisor and multiplier and you can also specify if you want to invert the result. You can also specify the number of decimal places.

Example: "mpdv_calc;MULT=5;DIV=2;INVERSE=false;FORMAT=n3" (the value is multiplied by 5, divided by 2, subsequently inverted, and the result is displayed with 3 decimal places)

Overview of regular expressions

You can find a large amount of information on regular expressions using the search engines on the internet. In the following, the most important aspects are presented.

Meta characters

Represent a range of characters.

Character	Description
.	Matches any character.
[aeiou]	Matches any single character included in the specified set of characters.
[^aeiou]	Matches any single character, which is not included in the specified set of characters.

[0-9a-fA-F]	Use of a hyphen (–) allows specification of contiguous character ranges.
\R.	Matches the decimal separator specified by the System.Globalization.NumberFormatInfo.NumberDecimalSeparator property of the current culture.
\R:	Matches the time separator specified by the DateTimeFormatInfo.TimeSeparator property of the current culture.

Quantifier

Repetition, number of characters

Quantifier	Description	Samples
*	Specifies zero or more matches.	The "\w*" mask matches a string consisting of zero or more letter characters. It's equivalent to the "\w{0,}" mask.
+	Specifies one or more matches.	The "\w+" mask matches a string consisting of one or more letter characters. It's equivalent to the "\w{1,}" mask.
?	Specifies zero or one match.	The "\w?" mask matches zero or one letter character. It's equivalent to the "\w{0,1}" mask.
{n}	Specifies exactly <i>n</i> matches.	The "\d{4}" mask matches exactly four digits.
{n,}	Specifies at least <i>n</i> matches.	The "\d{2,}" mask matches two or more digits.
{n,m}	Specifies at least <i>n</i> , but no more than <i>m</i> , matches.	The "\d{1,3}" mask matches either one, or two, or three digits.

Special characters

Special characters

Character	Description	Samples
 	Alternation symbol. This can be used to implement a choice between two or more alternatives.	The "1 2 3" mask matches either "1" or "2" or "3". The "abc 123" mask matches either "abc" or "123". The "\d{2} \p{L}{2}" mask matches either two digits or two letters.
()	Grouping. You can use parentheses to create sub-expressions, or to limit the scope of the alternation.	The "(an ba)t" mask matches either "ant" or "bat". The "(net)+" mask matches "net", "netnet", "netnetnet", ... strings. Compare with the "net+" mask which matches the "net", "nett", "nettt", ... strings. The "(0 1)+" mask matches a string of indeterminate length, consisting of "0" and "1".

Examples:

Input 1..9999 => Input format for property : ([1-9][1-9][0-9][1-9][0-9][1-9][0-9][0-9])

Input 0..999 => Input format for property : ([0-9][1-9][0-9]100)

Best practice: input of long string fields

The client identifies the width of an input field using the attribute **Length**. In case of long string fields with more than approx. 20 characters, the layout can become confusing because these string fields use the complete width of the layout and are very long compared to other input fields. Very long string fields are cut off on the right-hand side, if the available space is not enough. To avoid this behavior, you can control the displayed field width regardless of the number of characters that you can enter.

- Use the attribute **Length** to specify the **width** of the input field.
- You can use a regular expression in the **InputFormat** to specify the **number of characters that you can enter**.

If you enter strings that are larger than the displayed field, the input field automatically scrolls horizontally.

Examples:

Attribute	Length	InputFormat	Effect
article.designation	50	.{0.250}	The field is displayed with a width of 50 characters. You can enter up to 250 characters.
operation.input_component_list	25	.{0.40}	The field is displayed with a width of 25 characters. You can enter up to 40 characters.

6.7.13 FillChar

Obsolete. This field must be left empty.

6.7.14 Calculation

Obsolete. This field must be left empty.

6.7.15 Further fields see ServiceParameterGui

For a description of the following fields, refer to the data types of the ServiceParameterGui:

ControlType, ControlTypeMode, ControlParameter, ControlDataSource, ControlDataSourceMode, ControlDataSourceParameter, ControlDataSourceResult, VisibleCondition and EditableCondition.

6.8 ControlDataSource

A ControlDataSource defines a data source that you can use to fill selection lists in controls, for example. These can be data logics (service requests) or reference values (see also ReferenceData).

Reference values are usually required to fill selection lists (and/or RadioGroups) with static contents.

You use data logics to request services that identify selection lists (or RadioGroups) dynamically. For example, these lists can include master data that are configured in the database.

The settings made in the columns **Parameter** and **Result** can be overwritten in a **Property** or **ServiceParameterGui**.

6.8.1 Name

Name of the ControlDataSource. The name should be composed of English terms clearly describing the data source. You usually use the camelCase notation.

6.8.2 Source

If the data source is a web service, this field contains the name of the client's data logic. You derive the data logic from the service name. To do so, remove the dot between domain and function and use a capital letter for the first letter of the function:

Service	Data logic
MDUser.list	MDUserList
MDUnits.list	MDUnitsList

In case of reference values, this field includes the **Type** of a **ReferenceData**.

6.8.3 Parameter

A list of parameters. The list does not include spaces, use semicolons to separate parameters. This field is only allowed in combination with web service data sources. A parameter can be allocated dynamically or permanently.

Permanent parameters appear as <acronym>=<value>, e.g.

"dialogconfiguration.type=AIPDEF;dialogconfiguration.type=AIPTNR".

Dynamic parameters are specified as a pair of <acronym1>=[<acronym2>], e.g.

"resource.id=[resource.id];pdvprocessparameter.evaluation_ts=[pdvsinglevalue.evaluation_ts]"

The acronym in square brackets is replaced with the acronym values from the ControlPanel.

6.8.4 Columns

A list of requested columns. The list does not include spaces. To separate columns, semicolons are used. This is only permissible for web service data sources.

6.8.5 Result

You can enter 1-n acronyms separated by semicolon. The sequence used specifies the importance.

- Position 1 (Value): Name of acronym whose value is entered in the input field.
- Position 2 (ControlValue): Name of acronym whose value is displayed in the selection list. If you do not specify position 2, the acronym of position 1 is displayed.
- Position 3 (LabelValue): If you specify position 3, the value of the acronym is entered into the label field of the input field and also displayed in the selection list.
- Position 4-n: Use these positions to define additional return values, which are then used to update "dependent" controls in the client ("lookup").

Only with web service data sources:

Optional return columns of the data source, separated by semicolons. Without spaces. The return has the format `<acronym>=<value>` - for acronym pairs, the second acronym is therefore replaced with the result value (e.g. if you enter "operation.resource.id=resource.id", this results in "operation.resource.id=4711").

6.9 ReferenceData

Reference values are usually required to fill selection lists (and/or RadioGroups) with static contents. In contrast to values provided by web services, reference values are fixed and do not change. For this reason, reference values can be entered once in a list and are delivered in this form.

6.9.1 ref_data_key

The **ref_data_key** must be unambiguous for each entry. In special cases, this key is used in the source code (at least in the server).

Usually, the **ref_data_key** is composed of **type** + : + **db_key**; this facilitates its allocation to type and key. An exception occurs if the **db_key** includes a German expression. The **ref_data_key** must then be formed differently. For example, *pwdexclusion:person.firstname* is a super **ref_data_key** for the type *pwdexclusion.pwd* and **db_key** *PNR.PVORNAME*.

6.9.2 Type

Use this field to summarize various ReferenceData entries to a list.

6.9.3 db_key

The **db_key** is the actual value that is selected in the list. This key identifies an entry unambiguously within a **Type**. You cannot freely select the key because the key is often transferred to services and can correspond to the content of a configuration identifier in the database, for example.

6.9.4 is_default

The entry with this key is preallocated as default.

6.9.5 Designation

Text displayed in the selection list. A language key is specified.

6.9.6 sort_key

Specifies the sequence that is used to display the entries in the selection list.

6.10 Authorization

You use the authorization mechanism

- to protect applications and functions against unauthorized use in the client,
- to hide fields or field groups in the GUI,
- to prevent these fields from being edited.

6.10.1 Authorization type

Controls the type of authorization. Possible values:

- Acronym: enables the authorization of individual fields (properties)
- AcronymGroups: enables the authorization to group fields
- Application: enables the authorization of applications
- Functions: enables the authorization of functions which are e.g. requested from the application toolbar.

6.10.2 Authorization Context

Context where the authorization is intended. If the field is left empty, authorization is always granted, irrespective of the context. You normally use this field to control the authorization of acronyms in the context of special services.

6.10.3 Authorization ID

Identifies the object to be authorized, i.e. the name of the acronym or the ID of an application.

6.10.4 Authorization key

The authorization key that is used to protect the object.

6.10.5 Authorization Designation

(Optional) text description of the authorization.

7 Using the MDS Repository as Development Tool

The following has to be respected for correct completion, when using the MDS Repository as development tool to create new services.

7.1 Use of Transformation Types for the Dynamic Conversion of DB or BAPI Values

7.1.1 Summary

The web services provided by the server are partially created dynamically by interpreters at runtime. This is both possible for HYDRA wrappers (service calling a HYDRA dialog) and in List Services (service directly accessing the database).

The runtime interpreters map particular "Special treatments" in their standard form (e.g. converting the string fields returned by HYDRA BAPIs/lists into the real data type according to repository). Other things may not be mapped generally, since their function is not always the same (e.g. conversion of Boolean values). These are partially presented by J/N in HYDRA, but sometimes also by 1/0.

So far, a user exit had to be created for the relevant list service or wrapper for each of these special treatments. This requires enormous effort for partially simple tasks (write user exit, check-in, compile, ...).

This effort is to be reduced by the transformation type. The service parameters in the repository provide a new column named Transformation Type. This column is used to enter the definition of the transformation in a key/value format in the form

KEY1=VALUE | KEY2=VALUE | . . .

It is compulsory that the value **FCT=...** allocating the function is always indicated. According to the function, other values must additionally be indicated to configure the function.

The transformation type supports four different types of transformation (four different points in the service progress), through which changes may be made.

- Wrapper
 - o Adjusting the dialog data string sent for the request to HYDRA (e.g. converting a Boolean input value in its string)
 - o Adjusting the result (e.g. converting the string value returned by HYDRA into a Boolean value according to the representation in HYDRA)
- List Services

- o Indication of special filters for the service parameter (e.g.: filtering to a Boolean field must be converted for the actual filter in SQL, since the field is, for instance, a string field with J/N values in the database)
- o Adjusting the result (e.g. converting the selected string value with J/N from the database into a Boolean value)

All transformation functions are programmed in Java and may support one, some or all of the above-indicated transformations.

The Wrapper Interpreter (if set) will activate the transformation function for the transformation type when HYDRA is activated and upon a HYDRA return conversion.

If the transformation function returns false upon the transformation of the HYDRA request, the field with the default behavior will be used. If it returns true, the interpreter will not address the field since it assumes that it will be handled by the transformation function.

When creating the result conversion function, the wrapper returns either a data table expression or null. If null is returned, the standard conversion of the field is performed according to the configured data type. If the return is not null, the provided expression is used and the standard conversion is not applied.

The list interpreter builds its data type map, its filter map and its ResultModification map in accordance with the default procedure. Subsequently, it activates the methods for transforming the list call and the list result modification. These will independently modify the maps.

7.1.2 Adding a New Transformation Function

The transformation functions are located in the JAVA project **WSData** in the **de.mpdv.common.transformationType** package.

A new class for transformation has to be created within this package. The class should be given an appropriate name (e.g. BooleanTransformer) and has to extend the class AServiceCallTransformer. Example:

```
public class BooleanTransformer extends AServiceCallTransformer
```

The class must overwrite the constructor of the abstract upper class. The configuration parameters for the transformation may (if necessary) be verified in the constructor. The only transfer parameter in the constructor is a map including the key/value pairs from the transformer type field in the repository. The map will allocate the value (map value) to the key (map key).

According to the transformation, the following methods have to be overwritten subsequently:

transformHydraCall

- Parameter
 - o **callHelper** Object composing the HYDRA call. Values may be set and modified on this object
 - o **paramValue** Value provided for the field by the client
 - o **type** Data type of the field according to the repository
 - o **hydraAcronym** HYDRA acronym for the field
 - o **transferEmptyValuesToHydra** Flag from the repository indicating whether the field is to be transferred to HYDRA even if it is blank/null
 - o **env** Request environment with environment information on the service call
- Return
 - o **boolean** True if parameter was transformed, otherwise false

transformHydraResultConversion

- Parameter
 - o **columnName** Name of column
 - o **destinationType** Data type for field according to repository
 - o **env** Request environment with environment information on the service call
- Return
 - o **IDataTableExpression** Not null, if parameter was transformed, otherwise null

transformListCall

- Parameter
 - o **filterMap** Map allocating special filter implementation of a column
 - o **columnName** Name of column
 - o **tableAlias** Alias for the data base table used in the data source
 - o **dbField** Name of data base field according to the repository Qualified with %1\$s.
 - o **env** Request environment with environment information on the service call
- Return
 - o **boolean** True if parameter was transformed, otherwise false

transformListResult

- Parameter
 - o **dataTypeMap** Map allocating the data type for the data base selection of a column
 - o **modificationMap** Map allocating the result conversions of a column
 - o **columnName** Name of column
 - o **destinationType** Data type for field according to repository
 - o **env** Request environment with environment information on the service call

- Return
 - o **boolean** True if parameter was transformed, otherwise false

The newly added transformation function must finally be registered in the **ServiceCallTransformationManager**. This takes place in the method **initTransformerMap**. In the function map initialized there, another entry must be added in the following form:

```
map.put("BOOLTRANSFORMATION", new IServiceCallTransformerGenerator()
{
    /**
     * @see
     de.mpdv.common.transformationType.ServiceCallTransformationManager.IServiceCa
     llTransformerGenerator#generateTransformer(java.util.Map)
     */
    public IServiceCallTransformer generateTransformer(Map<String,
String> keyValueMap)
    {
        return new BooleanTransformer(keyValueMap);
    }
});
```

One relevant aspect for the entry in the map is the key. This key will address the function by FCT=... in the transformation type field of the repository (BOOLTRANSFORMATION in the example). The second relevant aspect is, that the entity created upon access must be indicated (a new entity of the BooleanTransformer class in the example).

7.1.3 Standard Transformation Functions

HYCOLORTORGBTRANSFORMATION

This transformation is used to convert fields for the return containing a color in HYDRA color code (1-16) to the associated RGB representation as integer, and this both for wrappers and for list services.

FCT-Id

HYCOLORTORGBTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported Transformations

Transformation	Description
HYDRA Result	Conversion of HYDRA color code from the string field of HYDRA to RGB as integer
List Result	Selection of HYDRA color code as integer from the database and subsequently conversion to RGB as integer

DATETIMEFILTERTRANSFORMATION

This transformation is intended to deposit a filter for fields which are selected as datetime through the database function get_datetime, however consist of two separate fields with the date and the time part in the database, in List Services.

FCT-Id

DATETIMEFILTERTRANSFORMATION

Configuration parameters

Name	Description
DBFIELDDATE	Name of database field with date part (without alias)
DBFIELDTIME	Name of database field with time part (without alias)

Supported Transformations

Transformation	Description
List Call	Adding a SeparateDateAndTimeFilter for the field, configured with the two indicated database fields

BOOLTRANSFORMATION

This transformation is intended to treat Boolean fields for Wrappers and List Services. This process will consider both the implementation for the HYDRA call, the List Service filter, and the result conversion for the two service types.

FCT-Id

BOOLTRANSFORMATION

Configuration parameters

Name	Description
TRUEINVAL	(OPTIONAL) Value for Yes (ja) upon HYDRA call/ List Filter (default J and/or if REALDATATYPE=integer then 1)
FALSEINVAL	(OPTIONAL) Value for No upon HYDRA call/ List Filter (default N and/or if

	REALDATATYPE=integer then 0)
TRUERESVAL	(OPTIONAL) Value for Yes (ja) upon return from HYDRA / the selection from the database (default J and/or if REALDATATYPE=integer then 1)
FALSERESVAL	(OPTIONAL) Value for No upon return from HYDRA / the selection from the database (default N and/or if REALDATATYPE=integer then 0). Must include a value matching the indicated REALDATATYPE (J for REALDATATYPE=integer is an error)
REALDATATYPE	(OPTIONAL) Indication of real data type of field, integer and string supported (default string)
NULLHANDLING	(OPTIONAL) Indication of interpretation of null values. Possible values are none (ignore null), true (interpret null as true) and false (interpret null as false) (default none)
OTHERVALHANDLING	(OPTIONAL) Indication of interpretation of other values (than null, the value for true and the value for false). Possible values are none (ignore others), true (interpret others as true) and false (interpret others as false) (default none)

Supported Transformations

Transformation	Description
HYDRA Result	Conversion of string value provided by HYDRA to Bool
List Result	Conversion of selected value from DB (either integer or string) to Bool
HYDRA Call	Conversion of true/false from client call to configured true/false values
List Call	Adding a filter for the DB field to filter the SQL according to data type and configured true/false values.

Examples:

Real value of string type, true=Y and false=N; null is interpreted as null (NULLHANDLING, REALDATATYPE, FALSEINVAL and FALSERESVAL need not be indicated because default values are correct, each)

```
FCT=BOOLTRANSFORMATION | TRUEINVAL=Y | TRUERESVAL=Y |
```

Real value of integer type, true=1 and false=0; null is interpreted as null (NULLHANDLING, TRUEINVAL, TRUERESVAL, FALSEINVAL and FALSERESVAL need not be indicated because default values are correct, each)

```
FCT=BOOLTRANSFORMATION | REALDATATYPE=integer |
```

DECIMALPLACESTRANSFORMATION

Converts the decimal places of a wrapper, e.g. "2" in a format string, in this case "#####0.##"

FCT-Id

DECIMALPLACESTRANSFORMATION

Configuration parameters

Name	Description
(none)	(-)

Supported Transformations

Transformation	Description
HYDRA Call	Conversion of a number in a format string, "2" -> "#####0.##".

DECIMALPLACESNUMBERTRANSFORMATION

Converts a format string, e.g. "0.00" to an integer, in this case "2", for a List Service.

FCT-Id

DECIMALPLACESNUMBERTRANSFORMATION

Configuration parameters

Name	Description
(none)	(-)

Supported Transformations

Transformation	Description
List Result	Conversion of a format string, e.g. "0.00" in an integer, "0.00" -> "2".

CASEINSENSITIVEFILTERTRANSFORMATION

String filter on a DB field regardless of upper/lower case

FCT-Id

CASEINSENSITIVEFILTERTRANSFORMATION

Configuration parameters

Name	Description
(none)	(-)

Supported Transformations

Transformation	Description
List Call	String Filter Case insensitive

SHIFTENDFILTERTRANSFORMATION

This transformation is used to filter the shift end time stamp in List Services. For this purpose, the date field or the date field + 1 day has to be used, according to whether the shift end is larger or smaller than the shift start.

FCT-Id

SHIFTENDFILTERTRANSFORMATION

Configuration parameters

Name	Description
DBFIELDDATE	Name of database field with shift date (without alias)
DBFIELDBEGIN	Name of database field with shift start (without alias)
DBFIELDEND	Name of database field with shift end (without alias)

Supported Transformations

Transformation	Description
List Call	Adding a ShiftEndDateFilter on basis of shift date, start and end

DATATYPECONVERSIONTRANSFORMATION

This transformation is intended to effect a data type conversion between DB and Web service in List Services.

At present, the following are supported:

- string to integer
- string to decimal
- integer to string
- integer to decimal
- decimal to string
- decimal to integer

FCT-Id

DATATYPECONVERSIONTRANSFORMATION

Configuration parameters

Name	Description
DBTYPE	Data type in DB
WSTYPE	Data type for web service

Supported Transformations

Transformation	Description
List Call	If a filter is indicated in web service type, this is to be converted to filter in DB type
List Result	Conversion of DB type field in web service type in result.

TSPARTTRANSFORMATION

This transformation is used to identify the components of a time stamp in List Services. Components are, e.g. year, day, months, calendar week,

FCT-Id

TSPARTTRANSFORMATION

Configuration parameters

Name	Description
MODE	Component to be identified (see separate table)

Supported Transformations

Transformation	Description
List Result	Conversion of DB type field datetime in requested component (as integer) in result

Valid Values for MODE

Name	Description
DAY	Day of month
DOY	Day of year
DOW	Day of week (0=Sunday ... 6=Saturday)
MONTH	Month
YEAR	Year
CWJ	Calendar week acc. to JAVA standard (CW1 = first complete week in year)
CWD	Calendar week acc. to DIN 1355/ISO 8601 (CW1 = week including January 4th)
CWU	Calendar week acc. to USA (CW1 = week including January 1st)
QUART	Quarter
HR	Hour
MIN	Minute
SEC	Second
MIL	Millisecond
MONTHB	Month of business year. The first month of the business year is determined from CAQ Option 1018.
QUARTB	Quarter of business year. The first month of the business year is determined from CAQ Option 1018.

Examples:

Domain	Service Function	Acronym	Is Result	Web Service Type	Conversion Method	Data Object Name	DB Alias	DB Field
								
Click here to add a new row								
TsPartTest.list	a.ts	☒	datetime			TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)
TsPartTest.list	a.day	☒	integer	FCT=TSPARTTRANSFORMATION MODE=DAY		TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)
TsPartTest.list	a.doy	☒	integer	FCT=TSPARTTRANSFORMATION MODE=DOY		TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)
TsPartTest.list	a.month	☒	integer	FCT=TSPARTTRANSFORMATION MODE=MONTH		TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)
TsPartTest.list	a.year	☒	integer	FCT=TSPARTTRANSFORMATION MODE=YEAR		TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)
TsPartTest.list	a.cwj	☒	integer	FCT=TSPARTTRANSFORMATION MODE=CWJ		TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)
TsPartTest.list	a.hr	☒	integer	FCT=TSPARTTRANSFORMATION MODE=HR		TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)
TsPartTest.list	a.min	☒	integer	FCT=TSPARTTRANSFORMATION MODE=MIN		TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)
TsPartTest.list	a.sec	☒	integer	FCT=TSPARTTRANSFORMATION MODE=SEC		TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)
TsPartTest.list	a.mil	☒	integer	FCT=TSPARTTRANSFORMATION MODE=MIL		TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)
TsPartTest.list	a.dow	☒	integer	FCT=TSPARTTRANSFORMATION MODE=DOW		TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)
TsPartTest.list	a.quart	☒	integer	FCT=TSPARTTRANSFORMATION MODE=QUART		TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)
TsPartTest.list	a.cwd	☒	integer	FCT=TSPARTTRANSFORMATION MODE=CWD		TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)
TsPartTest.list	a.cwu	☒	integer	FCT=TSPARTTRANSFORMATION MODE=CWU		TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)
TsPartTest.list	a.monthb	☒	integer	FCT=TSPARTTRANSFORMATION MODE=MONTHB		TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)
TsPartTest.list	a.quartb	☒	integer	FCT=TSPARTTRANSFORMATION MODE=QUARTB		TsPartTest.list	ut	hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)

ALPHAPERSONIDTRANSFORMATION

This transformation is intended to convert an alphanumeric personnel number to numerical for the result in List Services, and/or to filter the alphanumeric field through the numerical number.

FCT-Id

ALPHAPERSONIDTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported Transformations

Transformation	Description
List Call	Adding a special filter in order to filter the alphanumerical field through the numerical person.id.
List Result	Conversion of DB type string field into numerical personnel number

FIXCUTOFFNUMWORKPLACEIDTRANSFORMATION

This transformation has been designed to fill up the result field once more with leading zeros. This is required on systems using numeric machine numbers and for wrappers that have already cut the leading zeros, since the client needs the complete machine number as it is included in the database.

FCT-Id

FIXCUTOFFNUMWORKPLACEIDTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported Transformations

Transformation	Description
HYDRA Result	Completes the relevant result field with leading zeros until 8 characters have been reached, provided that it is shorter and numeric machine numbers are in use.

CATEGORYLEDTRANSFORMATION

This transformation is used to convert fields for the return containing the name of a bitmap for an order category to the associated LED constant, and this both for wrappers and for list services.

The assignment is as follows:

Bitmap	LED constant
--------	--------------

fa.bmp	LED_FA
gk.bmp	LED_GK
kp.bmp	LED_KP
na.bmp	LED_NA
pj.bmp	LED_PJ
pm.bmp	LED_PM

FCT-Id

CATEGORYLEDTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported Transformations

Transformation	Description
HYDRA Result	Conversion of the bitmap name from the string field from HYDRA to the LED constant
List Result	Selection of the bitmap name as string from the database and subsequent conversion to LED constant

STATUSLEDTRANSFORMATION

This transformation is used to convert fields for the return containing the name of a status bitmap to the associated LED constant, and this both for wrappers and for list services.

The assignment is as follows:

Bitmap	LED constant
x.bmp	LED_RED
v.bmp	LED_GREY
u.bmp	LED_YELLOW
p.bmp	LED_BLUE
n.bmp	LED_PINK
l.bmp	LED_LIGHT_GREEN
f.bmp	LED_YELLOW_GREEN
e.bmp	LED_GREEN
a.bmp	LED_BLACK

FCT-Id

STATUSLEDTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported Transformations

Transformation	Description
HYDRA Result	Conversion of the bitmap name from the string field of HYDRA to the LED constant
List Result	Selection of the bitmap name as string from the database and subsequent conversion to LED constant

LEGACYFULLTSTRANSFORMATION

This transformation is used for mapping a complete time stamp to a single acronym of the dialog string for wrapper services. The default functions of the wrapper interpreter only allow for the date part to be mapped on an acronym or date and time each on separate acronyms.

The acronym is assigned to the values of the time stamp in the format MM/dd/yyyy HH:mm:ss.

FCT-Id

LEGACYFULLTSTRANSFORMATION

Configuration parameters

Name	Description
------	-------------

Supported Transformations

Transformation	Description
HYDRA Call	Setting the complete time stamp to an acronym

LEGACYARRAYPARAMETERTRANSFORMATION

This transformation is intended to support an "IN" and "BETWEEN" for Wrappers Services. The default functions of the wrapper interpreter only allow for single values to be mapped on HYDRA acronyms.

The list of values is converted to a string separated by separators. Each single value is also embraced by single inverted commas for the "string" web service type. The separator used between the single values can be configured and is a comma by default.

FCT-Id

LEGACYARRAYPARAMETERTRANSFORMATION

Configuration parameters

Name	Description
SEPARATOR	Optional: separator to be used, if not stated, comma is to be used

Supported Transformations

Transformation	Description
HYDRA Call	Conversion of value lists to a string separated by separators



The data types "string" and "integer" are supported only!

7.2 Checklist: Repository Data

The correct completion of the MDS repository is naturally a complex task, which is occasionally prone to errors, too. The following sections are intended to assist you in avoiding typical mistakes.

Term explanation: *Input parameter* is an acronym, if **isFilterParameter** or **isSpecialParameter** is set. *Output parameter* or *Return value* is an acronym, if **isResult** is set to Y.

ServiceParameter: An input parameter must define a minimum of one operator

Effect: Wrapper generator stops with the following error message: Input Parameter specified with no operator (with indication of acronym and services concerned)

Reason: An input parameter should define a minimum of one operator.

Solution: Set a minimum of one of the Can columns (usually CanEqual) to Y

ServiceParameter: Operator cannot exist without input parameters

Effect: Wrapper generator stops with the following error message: Operator specified for parameter that is no input parameter (with indication of acronym and services concerned)

Reason: An input parameter should define a minimum of one operator.

Solution: Set a minimum of one Can column (usually CanEqual) to Y.

ServiceParameter: Acronym must be input or output parameter

Description: it is not possible that an acronym is neither input nor output parameter, exception: acronym with fixed value for the wrapper.

Effect: Wrapper generator stops with the following error message: Neither input nor output parameter found (with indication of acronym concerned)

ServiceParameter: Acronym with fixed value for the wrapper

Description: Normally, an acronym is at least one of both: input or output parameter. In case that a wrapper has to transfer a fixed-value parameter upon a BAPI call and this is not known on the client side, all three columns for input and output parameters may remain unset.

Check: isFilterParameter, isSpecialParameter, isResult blank, DefaultValue must be set; HydraAcronym must be set.

Effect: if DefaultValue is not set, the wrapper generator stops with the following error message: Neither input nor output parameter found (with indication of acronym concerned)

For details please refer to section 6.1.

ServiceParameter: Acronym with fixed value for wrapper must have data type string

Description: WebServiceType for acronym with fixed value may only be string, even if DB column has a different data type.

Effect: Wrapper generator stops with the following message: java.lang.IllegalStateException: Fix value parameter with other datatype than string found: INTEGER

Solution: Declare data type in repository as string.

ServiceParameter: Reference field for *date, *time and datetime must be identical

Effect: Wrapper generator stops with the following error message: At least one part of date/time triple parameter is missing

Reason: The entries with the * types are determined for the wrapper service and must have the same reference value as the corresponding datetime entry. The reference value must be clear within a service.

Solution: equalize the three reference values.

ServiceParameter: Hydra acronym must be available for an input parameter of a wrapper service

Effect: Wrapper generator stops with the following error message: Input parameter with empty Hydra-Acronym found.

Reason: The wrapper service must be able to map an acronym to the HYDRA acronym of the BAPI

Solution: Add HYDRA acronym

ServiceParameter: A wrapper service should have a minimum of one parameter

Effect: Trouble will only occur when update, delete, lock, or unlock is called in the MOC application.

Reason: Functions as delete or lock typically require a key field

ServiceParameter: Wrapper services do not have any output parameters

Effect: Description: isResult must not be set

Solution: empty isResult

ServiceParameter: Wrapper services do not have any filter parameters

Description/Solution: Set isSpecialParameter to Y, empty FilterParameter

ServiceParameter: No double acronyms within the same service

Exception: multiple ResultSets

Effect: DataLogic generator stops with the following error message: ERROR: Acronym duplicate in non-multiple result set: <acronym>! Please ensure the services.xml export doesn't contain duplicate entries!

Solution: Check each service for clear acronyms

ServiceParameter: Indicate key field for list service, too

Effect (e.g.): Delete in the MOC application will cause the exception "MissingPrimaryKeyException". However, isKey is correctly available in repository, wrapper, servicex.xml and DataLogic of delete services. The reason is the lack of a key in the list service so that the data record may be identified in the grid.

Solution: Indicate isKey and isMandatory for the key fields of the list

ServiceParameter: Indicate mandatory fields for wrapper

Effect: Insert service of MOC application complains by displaying an error returned from BAPI.

Solution: Set isMandatory for appropriate mandatory fields (cf. associated SystemDesign document)

Service: Set service type correctly

Specific case: all services of the domain (list, insert, copy, update, delete, lock, ...) are set to Java service although wrappers are planned.

Effect: ProjectManager/SVN Working Copy/Commit does not suggest the newly "created" wrapper services for check-in.

Reason: The exported services.xml is created empty; wrapper generator indeed runs without error message, but does, quite correctly, not generate any source code.

Solution: Visual diagnosis: Frequently list is a Java service, however the other functions of the domain are wrappers.

All tables, all columns with specific value stock: Only particular specified values are permissible

Typical: V instead of Y. This is very difficult to recognize in a visual check.

It occurs upon insertion/pasting (Ctrl-V) in the repository

Solution: Check columns for impermissible values (by column filter selection in case of visual diagnosis).

7.3 Entry of Fixed Values for Wrappers

If fixed values (independent of client call) are to be transferred to HYDRA at a wrapper (e.g. MOD=E), these must be entered as follows:

- WebServiceType. Set to string
- DefaultValue (where the default value must be entered, i.e. E, for instance)
- HydraAcronym (where the acronym must be entered, i.e. MOD, for instance)

The following columns MUST be empty:

- InputAsArray
- IsSpecialParameter
- IsFilterParameter
- IsMandatory
- Can....

- IsResult

Example:

Service: MDPlanningProfileAssignment.deleteSingleEntry

Acronym: planningprofileassignment.deletionmode

8 Repository Client

You use the MPDV Repository Client MRC to display and edit repository data. It provides a user-friendly access.

8.1 Quick start

This section provides a quick overview of how to work with the Repository Client. The individual steps are only briefly described. For further information on the individual steps, refer to the sections in the following, if required.

Installation

Requirements

To use the Repository Client, you must have installed the Microsoft DotNet framework (at least version 4.5.2).

Program installation

To install the program, just copy the folder including the binary files into your system. An installation program is not required.

Installation of developer license

If the developer license is not available, you can only read the data. You cannot save or export the data.

The developer license is handed out once you have attended a respective Customizing Training. The developer license is provided as *.lic file. This file is included in the data medium that you have received during the training: Folder "Repository Client", subfolder "tools/licence", e.g.

x:\CUT-MOC_81_files\Tools\MPDVRepositoryClient\tools\licence\mpdvWrite.lic.

Copy the folder "licence" with its content into the folder of the Repository Client in the roaming directory of the Windows user, e.g.:

C:\Users\%User%\AppData\Roaming\MPDV\RepositoryClient\licence\mpdvWrite.lic

This folder is automatically created on the first start of the Repository Client.

Before you start

Before you start working with the Repository Client, you must make sure that the Repository Client has been installed according to the installation instructions and that the required license files are stored as described there.

In general, the repository is empty when you start work. However, if you do not want to start with an empty repository, it is recommended to make sure that the data you want to work with is available.

First steps

Start the Repository Client.

→ Start `.\\bin\\mrc.exe`

Now load or create a [Workset](#).

A workset specifies the sources of the repository that you want to edit.

→ Click the button *Load work set* in the file-based repository
-> select `workingset.work`

Click the button [Repository](#) → [Load Repository](#) to load the data from the sources specified in the workset.

How to proceed further depends on what you want to do via the Repository Client.

Tip: Working with perspectives

The Repository Client provides the possibility to use different perspectives for different tasks. Do not hesitate to close views you do not need or drag them to another open view in order to tab them and hence provide for more space and clarity. You can also use more than one view of the same type. Simply adapt your perspective to the requirements of your tasks.

Example:

If you create a service and you would like to check with an existing service how to populate the fields, you can simply open another service view. Thus, you do not need to destroy your current view and re-orient yourself later.

Default perspectives

The installation of the Repository Client provides default perspectives:

default

This is a good perspective to start with. It provides a combined view for server and client-related contents.

Select a domain on the top left. Via the included relations, the top right area shows the services, servicesGUI and properties of this domain. The bottom right area shows the ServiceParameters of the service selected above, the ServiceParameterGui of the ServiceGui selected above and the ControlDataSources, ReferenceData and Authorizations of the selected domain.

default client

Similar to the "default" perspective but limited to the contents concerning client development.

default server

Similar to the "default" perspective but limited to the contents concerning server development.

DB schema

Shows documentation of the database structure.

Validation

Use this perspective to validate contents.

Proceed as follows:

- Open perspective and load data from workset.
- Perform validation: tab *Repository* --> button *Validate*.
- A CSV file listing the identified irregularities is generated in the sub-directory "validation_logs" of the Repository Client's installation directory. The application that is linked to this file type in the operating system opens the CSV file.
- In the views for Domains, Properties, ServiceParameters, etc. the Repository Client only shows the entries with detected irregularities.

You should analyze and, if necessary, correct the detected irregularities. Not every irregularity leads to an error.

8.2 Start and exit Repository Client

Start the Repository Client via the Windows start menu, a link on the desktop or the command line. As soon as all required components have been loaded, the application window is shown.

You can start and run the Repository Client multiple times in parallel on a PC. You can access different repository data in each of the started instances of the Repository Client. For example, you can view several versions of the repository at the same time.

You can also start the client by opening one of the workset files. On start of the client, the system attempts to load the contents of the workset defined in this file. This option is available after the first start of the Repository Client.

Command line parameters

You may transfer parameters to the Repository Client upon the start. The following parameters are supported:

-perspective/-p <perspectivename> Use this parameter to start the Repository Client with a specific perspective. If you do not use this parameter, the last active perspective is started by default.

`-workset/-w <worksetfile>` Use this parameter to specify the workset to be loaded. If you do not specify the workset on start of the application, the last loaded workset is loaded.

`--autoload/--a` If you use this parameter, the Repository Client loads the repository defined in the last active workset or in the workset transferred via parameters. This repository is loaded directly on start of the application.

`--trim/--t` If you use this parameter, the Repository Client removes so-called leading and trailing "whitespaces" when loading the repository. Only select this option if needed, because the load time increases extremely.

Exit

To exit the application, click  in the title bar.



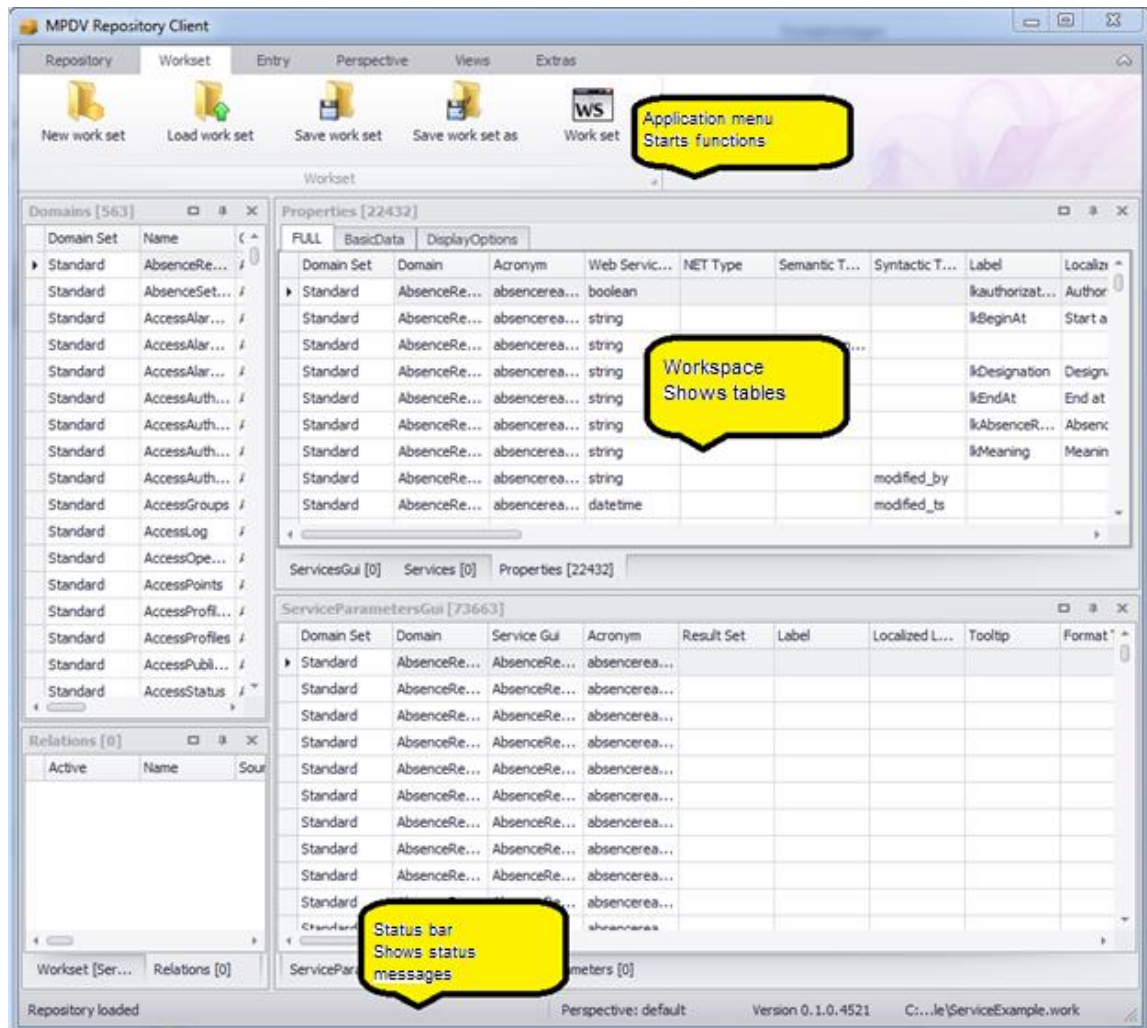
If the loaded repository includes active changes when you exit the application, a respective message is issued asking you to save the changes. If you do not want to save the changes, you can discard the changes or stop exiting the application.



Note: Changes to the workset and perspective are discarded when you exit the application, if you have not saved the changes.

8.3 The application window

The application window forms a framework for the display of different tables. It includes the application menu with control elements to call and control different functionalities. The menu is on top of the window. A status bar is at the bottom of the window. The status bar shows progress and event messages.



You can individually dock the grids/table views. To do so, click the title bar of a table view/grid and drag it out of the docking position. For orientation purposes, the system shows the docking positions where you can drop the table view. You can also drop a table view without docking it.

8.4 Grids/table views

Grids/table views are components to present data records in a table. You can change the tables in the Repository Client according to your requirements. For each grid/table view, the functions described below are available.



The settings, that you make in a table, are saved with the perspective. To undo changes, you can switch to the standard perspective (in the application menu: [Perspective](#) → [Change perspective](#)).

Sort table data

Click the table header to sort table data in descending order. If you click the table header once more, data is sorted in ascending order. The selected sorting option is shown.

You can sort data by several columns: Press the *Shift* key of your keyboard after sorting the first column. Then click the other column headings by which you want to sort.

You can also use the context menu of the table header to sort data.

Group data in the table

You can group table data if the grouping bar is shown. If the grouping bar is not shown, you can show the bar via the context menu of the table header. To group by a column, click the column header and drag it to the grouping pane. Multiple grouping is also supported.

Optimum column width (best fit)

Select the option "*Best fit*" in the context menu of the table header to adjust the column width of the selected column to the optimum width. In this case, "optimum" means that the column is as wide as the largest entry in the selected column.

Optimum column width (all columns) / Best fit (all columns)

Click this function to adjust all columns to the optimum width.

Change column width

You can change the column width using the mouse, i.e. shift the space between two cells to the left or right.

Show and/or hide columns and entire categories

Use the context menu function *Select columns* to show and/or hide individual columns and entire categories. For this purpose, select the function via the context menu and then drag the required columns and/or categories from the table to the pool or from the pool to the table.

Change the sequence of columns and categories

Also use the mouse to change the display order of columns and categories. To do so, drag & drop the column or category to be shifted to the requested location. The system will show the location to which the column and/or category is allocated when you release the left mouse button.

Freezing columns to prevent horizontal scrolling

You can freeze columns at the left and right-hand side to keep the columns in view while scrolling. These column settings are included in the perspective and can be saved with the perspective. Right-click the column header and press one of the below-mentioned shortcuts to freeze columns:

- CTRL + right click: freeze at the left-hand side.
- ALT + right click: freeze at the right-hand side.
- SHIFT + right click: Unfreeze.

Filter table data

Click the filter icon  in the column where you intend to set the filter and select the required filter option from the list; i.e. you select one of the values available in the table or compose a combination of values in the *user-defined* filter.

You can also set several filters in different columns. The table footer indicates that the table has been filtered and also shows the filter criteria. Use the function *Edit filter* on the right of the footer to open the filter editor. Use the filter editor to create complex filter criteria across all columns. You may also open the filter editor via the context menu of the table header.

Search box

Use the context menu of the table header to access the option *Show search box*. This option provides a search box within the table. Use this box to quickly search and/or filter the requested data. Simply start typing in this box and the system will only show those rows matching the data you typed. The more characters of the required combination you enter, the more reduced and detailed the result will be.

Filter row

Use the context menu of the table header to open the option *Show filter row*. This option provides an additional row shown below the table header. You may enter a search term in any column, and the system will narrow down the displayed rows appropriately. You can also use wildcards. You can also combine values in several columns to narrow down the search.

Edit rows

Double-click a row or press the "Enter" button to switch to the editing mode and edit the respective row. When you have finished editing and leave the row, the editing mode is terminated. Edited rows are highlighted in color.

8.5 The application menu

You can use the application ribbon menu of the Repository Client to control various functions of the tool. It includes several tabs that are described in the following.

Workset

Includes functions to administer worksets. A workset specifies the sources included in the repository that you want to edit. To display a workset, use the workset panel which consists of a grid/table view.



- **New workset:** This function creates a new workset. If a workset is loaded that has been modified, a dialog pops up asking you to save the changes.
Click "Yes" to save the changes, "No" will discard them. In both cases, a new workset is created. Click "Cancel" to cancel the process of creating a new workset.
- **Load workset:** This functions loads a workset from an existing file. You can select the workset to be loaded via a file dialog. If the currently loaded workset has been modified, you can save the changes as described above.
- **Save workset:** This function saves the current workset in the file from which it was loaded. If the current workset is new, a file dialog pops up asking you to select the file in which you want to save the workset.
- **Save workset as:** Use this function to save the currently loaded workset in a file. You can select the files using a file dialog.
- **Workset:** Use this button to show and/or hide the grid/table view presenting the currently loaded workset. For details on the workset table, please refer to section Workset.

Repository

Includes functions to load and save data in the repository. In addition, you may display repository-specific data here.

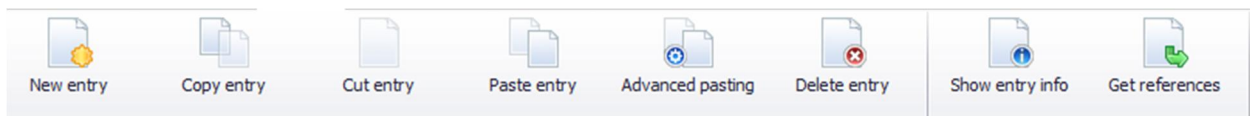


- **Load repository:** Use this function to load the repository. The currently loaded workset specifies the data sources that are used to load the repository. If a repository is loaded that has been modified, a dialog pops up asking you to save the changes. Click "Yes" to save the changes, "No" will discard them. In both cases, the repository will be reloaded subsequently. Click "Cancel" to cancel the process of loading a repository.
- **Save repository:** ***only available if used in development mode**
Use this function to save changes in the repository. If no changes have been made, an appropriate note will be displayed.
- **Export repository:** ***only available if used in development mode**
In contrast to saving the repository, you can use this function to export parts of the repository. For details on this function, please refer to section **Fehler! Verweisquelle konnte nicht gefunden werden.**
- **Validate:** ***only available if used in development mode**
You can use this function to validate your data records manually. (See section "Validation").
- **Value list:** Use this menu entry to show and/or hide the value list. The list includes permissible entries for specific fields of the repository.

- **References:** Use this entry to show and/or hide the table with repository references. For details on references, please refer to section References.
- **Changes: *only available if used in development mode**
Use this button to show and/or hide the change view. This view shows the current modifications in the loaded repository.

Entry

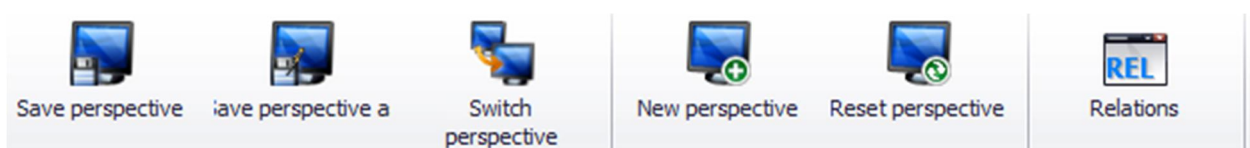
The *Entry* tab summarizes the functions that you can use to edit the loaded repository. The entries refer to the currently focused table view/grid.



- **New entry:** Use this function to create a new entry. For details on this function, please refer to [Context menu → New](#).
- **Copy entry:** Use this function to copy selected table entries. For details on this function, please refer to [Context menu → Copy](#).
- **Cut entry:** Use this function to cut selected table entries. For details on this function, please refer to [Context menu → Cut](#).
- **Paste entry:** Use this function to insert (paste) entries from the cache/clipboard. For details on this function, please refer to [Context menu → Insert](#).
- **Advanced pasting:** Use this function to edit entries in the clipboard prior to inserting them. For details on this function, please refer to [Context menu → Advanced pasting](#).
- **Delete entry:** Use this function to delete the selected entries. For details on this function, please refer to [Context menu → Delete](#).
- **Show entry info:** Use this function to open a dialog showing information on the currently selected entry. For details on this function, please refer to [Context menu → Info](#).
- **Get references:** Use this function to open a new grid/table view showing the currently selected data record including referenced values. For details on this function, please refer to [Context menu → Get references](#).

Perspective

These entries of the menu refer to the administration of perspectives. A perspective is a layout of table views/grids and includes also the associated relations between table views/grids.



- **Save perspective:** Use this function to save the currently shown arrangement of tables.
- **Save perspective as:** Use this function to save the currently shown perspective under a different name. You can enter the new name in the displayed dialog.
- **Switch perspective:** Use this function to change the perspective. A dialog with all available perspectives is shown.
- **New perspective:** Use this function to create a new perspective. Similar to the "Save perspective as" function, you can select the name of the perspective in a dialog. After entering the name, you can immediately switch to the new perspective.
- **Reset perspective:** Use this function to reset the current perspective to the status saved last.
- **Relations:** Use this menu entry to show/hide the grid/table view showing the relations between the grids/table views. For details on relations, please refer to section Relations.



Note: Changes to the perspective are discarded when you exit the application, if you have not saved the changes explicitly.

Views

Use these entries to open table views/grids. The entries will open a new grid/table view each showing the relevant data records of the repository.



For clear identification of the data records shown in the tables, the *Parent* column is included in each of the tables. The *Parent* column includes the identifier for the father node in the repository tree. The other columns of these tables are defined by the repository documentation.

The *View* area additionally includes a group with entries for the remaining grids/table views of the application.

8.6 Workset

Worksets define a set of data sources that make up the repository that you want to edit. Use the workset management function to organize your work on different projects and create an appropriate workset for each of your projects. You can show/hide the workset table via the application menu ([Workset](#) → [Workset](#)).



Note: The workset loaded last will be loaded on start of the Repository Client.

Workset [example.work]							
Name	Server Source	Client Source	Prio...	Is Writeable	Active	Overrides	M
ZIP	D:\DevRepo\ServerDomains.zip	D:\DevRepo\ClientDomains.zip	1	☐	☐		
Runtime	\\servername\hydra1\jhydradir\MOC\1	C:\Program Files (x86)\MPDV\HYDRA 8\MOC	2	☐	☒		
▶ Dev	d:\DevSrc\Repository\Data\server	d:\DevSrc\Repository\Data\client	3	☒	☒		

The workset table includes the following columns:

Name

You can specify the domain set that you want to use to load the data source. The repository data include the information on the domain set that was used to load the data. You can copy data from a domain set into another domain set. Example: Copy existing applications from the domain set "Runtime" (read only) into your development directory, e.g. the domain set "Dev" (writable) where you can make changes.

Client Source

You can specify the data source that you want to use to load client data. The following options are available:

- Load data from the runtime structure of the client reference in the server
In the server, the client configurations are stored as client reference with runtime structure. You can load the repository data from this structure.

Example:

HYDRA: x:\jhydradir\MaintenanceManager\rt\client\MOC

MIP: x:\wsp_config\MaintenanceManager\rt\client\MOC

The access is read-only.

- Load data from the runtime structure of a local MOC client
If you enter a path to an MOC installation directory, the respective client data are directly loaded from the MOC runtime installation.

Example:

C:\Program Files (x86)\MPDV\HYDRA 8\MOC

The access is read-only.

- Load data from a local directory with domain structure
You use local directories with domain structure for the administration of your own developments. Using the Repository Client, you can read data in this directory and you can also save data into this directory.

Example: d:\DevSrc\Repository\Data\client

- Load data from a ZIP archive
You can enter a ZIP file (including path) that includes the data in domain structure. MPDV provides the ZIP archives as part of the trainings or on the support portal.
The access is read-only.

Server Source

You can specify the server source that you want to use to load the server-specific data. The following options are available:

- Load data from the runtime structure in the server

You can read the configurations for the server in a server installation. To this end, the configuration directory of the web service provider (WSP) up to the instance number is specified.

Example:

```
HYDRA:      \\<servername>\<install_dir>\jhydradir\MOC\1
MIP:        \\<servername>\<install_dir>\wsp_config\MOC\1
```

By default, the configuration is loaded from the standard scope. As an alternative, you can also load the configuration from the *custom* scope, if you enter the value "custom" in the field "name". The access is read-only.

- Load data from a local directory with domain structure

You use local directories with domain structure for the administration of your own developments. Using the Repository Client, you can read data in this directory and you can also save data into this directory.

Example: d:\DevSrc\Repository\Data\server

- Load data from a ZIP archive

You can enter a ZIP file (including path) that includes the data in domain structure. MPDV provides the ZIP archives as part of the trainings or on the support portal.

The access is read-only.

Priority

The priority of a data record specifies the loading sequence of sources that are allocated to the same data record. In the above example, data is first read from the local development directory "d:\DevSrc\Repository" and then from the runtime installations "Server" and "Client". Data records with low priority are overridden and consequently not loaded.

Is Writeable

In this column you can specify if a data source grants write access. You only require write access in workset entries where you want to make local developments.



Please note that ZIP archives do not grant write access. For this reasons, you must not enable "is Writeable" in case of a ZIP data source.



Please note that it is not supported to save data in the runtime structure (client directory or server runtime directory). You must therefore not enable "Is Writable" for data sources with runtime structure.

Overrides

You can specify in this column which domain set is overridden by the current one. This affects the resolving of references (for details please refer to section References).



An entry in the "Overrides" column does not have any effect on the loading of the data sources. See column "Priority".

Active

Use this option to enable or disable an entry.

8.7 Relations

Relations are a property of perspectives and define table filters. Tables that include relations are dynamically adapted to the selected values of another table by setting a filter. For example: Using relations, you can specify that only the service parameters of the service currently selected in another service view are displayed in a service parameter view. The *Relations* table lists the relations of the current perspective. You can call this table via the application menu (*Perspective* → *Relations*).

Active	Name	Source	Target	Filter	Var0	Var1	Var2
☒	Authorizations for Domain	Domains	Authorizations	[Parent] = 'id->Authorizations'			
☒	ControlDataSources for Domain	Domains	ControlDataSources	[Parent] = 'id->ControlDataSources'			
☒	DataObject for Service	Services	Dataobjects	[DomainSet] = '\$0' and [Domain] = '\$1' AND [Name] = '\$2'	DomainSet	Domain	Name
☒	Properties for Domain	Domains	Properties	[Parent] = 'id->Properties'			
☒	ReferenceData for Domain	Domains	ReferenceData	[Parent] = 'id->ReferenceData'			
☒	SchemaColumns for ServiceParameter	ServiceParameters	SchemaColumns	(([TableName] = '\$0' AND [ColumnName] = '\$1' AND [TableNam...	DBTabelle	DBField	HydraAcronym
☐	ServiceParameter for ServiceParameterGui	ServiceParametersGui	ServiceParameters	[Domain] = '\$0' AND [Service] = '\$1' AND [Acronym] = '\$2'	Domain	ServiceGui	Acronym
☒	ServiceParameters for Service	Services	ServiceParameters	[Parent] = 'id'			
☒	ServiceParametersGui for Service	Services	ServiceParametersGui	[DomainSet] = '\$0' and [Domain] = '\$1' AND [ServiceGui] = '\$2'	DomainSet	Domain	Name
☐	ServiceParametersGui for ServiceGui	ServicesGui	ServiceParametersGui	[Parent] = 'id'			
☒	Services for Domain	Domains	Services	[Parent] = 'id->Services'			
☐	Service forServiceGui	ServicesGui	Services	[Name] = '\$0'	Name		
☒	ServicesGui for Domain	Domains	ServicesGui	[Parent] = 'id->ServicesGui'			

The following columns are displayed:

Active

This checkbox specifies if the relation is used.

Name

The name of the relation – free choice.

Source

The table and its selection that are used to set the filters. If you edit an entry in this column, the currently possible assignments, i.e. all currently existing views are presented in a selection box.

Target

The table where the filter is applied. If you edit an entry in this column, the currently possible assignments, i.e. all currently existing views are presented in a selection box.

Filter

You can store the filter expression here that you want to apply to the target table. Variables ranging between \$0 and \$9 are supported. These variables are dynamically filled with the values of the columns Var[0-9].

Var[0-9]

In these columns, you can specify the columns of the source table that are used to adapt the filters dynamically.

As soon as a correct (and activated) relation is entered in this table, it is applied. If you close one of the referenced views of a relation, the relevant view is removed from the relation. If this results in a double entry in the *Relations* table, this entry is removed. You can therefore use this view to administer the relations between concrete table instances and to administer unbound relations that can serve as template for relations.

8.8 References

References show the inherent connections between data records of the repository. They are defined by the repository structure and cannot be edited in the Repository Client. For example: A value in the column "Syntactic Type" of the *Property* table references another data record in the *Property* table. The *References* table lists the defined references and may be activated via the application menu ([Repository](#) → [References](#)).

Name	Source	Source Column	Dependency	Condition	Target	Filter	Priority
ControlDataSource1	Property	ControlDataSource	ControlDataSourceMode	Lookup	ControlDataSource	[Name] = \$value and [Domainset] = \$domset	-1
ControlDataSource2	Property	ControlDataSource	ControlDataSourceMode	Reference	ReferenceData	[type] = \$value and [Domainset] = \$domset	-1
SyntacticType	Property	SyntacticType			Property	[Acronym] = \$value	0
SemanticType	Property	SemanticType			Property	[Acronym] = \$value	1
Acronym1	ServiceParameter	Acronym			Property	[Acronym] = \$value	3
Acronym2	GuiServiceParameter	Acronym			Property	[Acronym] = \$value	3

The following columns are displayed:

- **Name:** Name of the reference
- **Source:** The repository object type that can include this reference.
- **SourceColumn:** The source type property that can include the reference.
- **Dependency:** Source type property specifying the reference target.
- **Condition:** Value that the property specified under *Dependency* must have. Only then, the reference is pursued. For example: The value of ControlDataSourceMode (lookup, reference) specifies the target of the reference (ControlDataSource or ReferenceData) which is specified in the ControlDataSource property.
- **Target:** The repository object type that is referenced.

- **Filter:** Filter that selects the referenced data from the overall quantity of this type of data. Such a filter can include the variable \$value (value of column *Value* in the current row) and \$parent (value of column *Parent* in the current row).
- **Priority:** Specifies the priority of the reference. You can find further details in section "Get references".

References provide two general functions:

Show reference: You can use this function to display the referenced data in a new table. You can call this function via the context menu ([Context menu](#) → [Show reference](#)).



Note: This function is only available in the context menu of cells which can include references.

Get references: Use this function to complete missing values of a data record with values of referenced data records. For example: You can use this function to show the inherited values of a property of the *SemanticType* or the *SyntacticType*.

You can call this function via the context menu ([Context menu](#) → [Get references](#)) of the table views/grids. A new data record is generated and shown in a new panel. The generated data record is a copy of the currently selected data record. The values that are not filled are filled by those in the referenced data records. The reference priority specifies the filling sequence.

9 Using the MDS Repository Client as development tool

The Repository Client of the MES Development Suite is not only used as service documentation. You can also use it to edit the MDS Repository data to create new services.

9.1 How to create new contents

The data structure of the repository is hierarchical. This means that a service is always part of a domain, a service parameter is always part of a service and properties are always the children of a domain.

This again means that you always work in a "top-down" view when working in the repository. For example, if you want to create a new service, you must ensure that the domain where you want to create the service does actually exist. If you want to create a service parameter for a new service, this one should also exist at this time, etc.

If you keep this basic information in mind and design your workflow on basis of this structure, you will spare a lot of unnecessary work and frustration.

If you would like to change the repository, it is recommended to create another domain set within the work set to manage your modifications. Do not forget to check the "IsWriteable" option – otherwise you will not be able to save any changes later on.

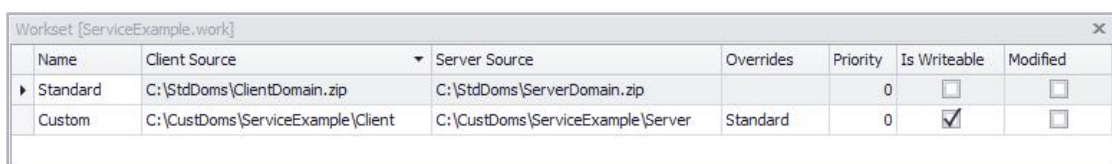
Please also note that the workset is not part of the repository data model and changes in the workset will only become effective upon re-loading the repository. We therefore recommend that the workset is defined for the imminent task, first.

Prior to starting your work, it is reasonable to make yourself familiar with the [Relations](#).

Example: Creating new services

The use case in the following illustrates the workflows that are involved in the generation of services.

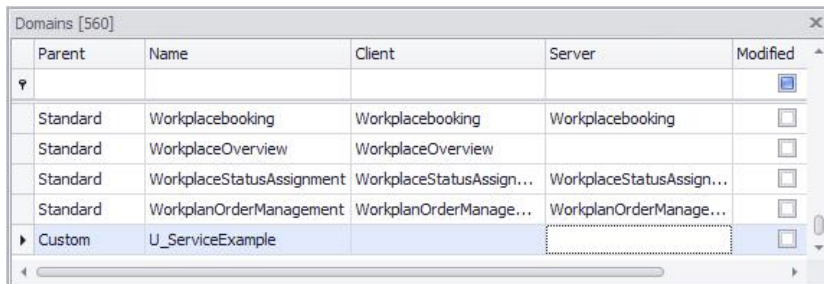
1. Import the latest repository version.
2. Open the repository client.
3. Create a new [Workset](#) with two domain sets (standard, custom).



Name	Client Source	Server Source	Overrides	Priority	Is Writeable	Modified
Standard	C:\StdDoms\ClientDomain.zip	C:\StdDoms\ServerDomain.zip		0	☐	☐
Custom	C:\CustDoms\ServiceExample\Client	C:\CustDoms\ServiceExample\Server	Standard	0	☒	☐

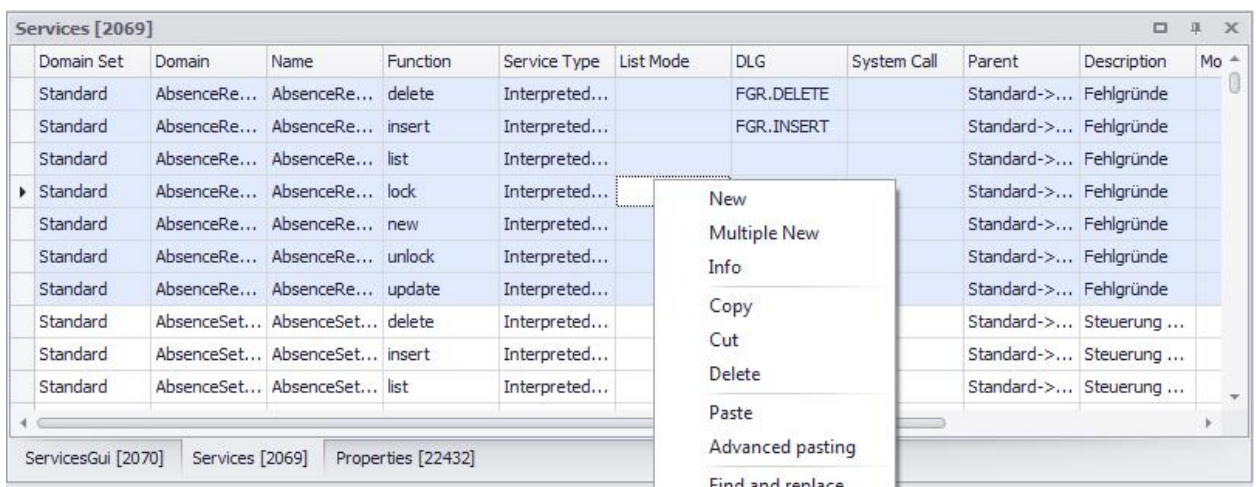
4. Save the new workset.

5. Load the repository.
6. Create a new domain via the context menu (right click the domain view→New) and edit the domain data (Name = "U_ServiceExample").



Parent	Name	Client	Server	Modified
Standard	Workplacebooking	Workplacebooking	Workplacebooking	
Standard	WorkplaceOverview	WorkplaceOverview		
Standard	WorkplaceStatusAssignment	WorkplaceStatusAssign...	WorkplaceStatusAssign...	
Standard	WorkplanOrderManagement	WorkplanOrderManage...	WorkplanOrderManage...	
Custom	U_ServiceExample			

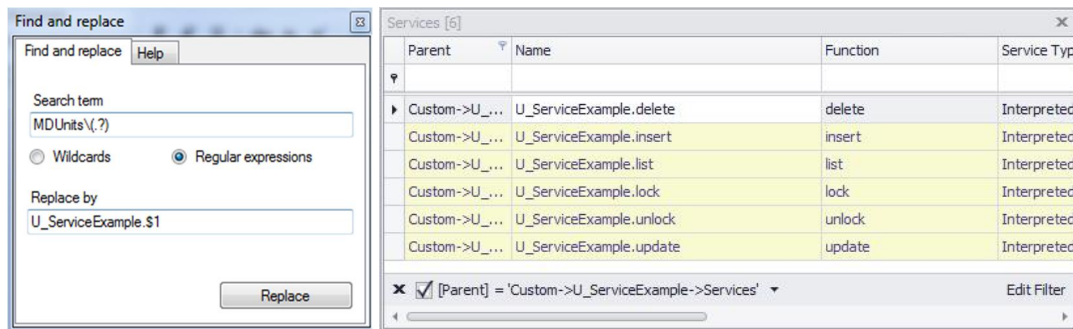
7. Copy other services to be used as model via the context menu:



Domain Set	Domain	Name	Function	Service Type	List Mode	DLG	System Call	Parent	Description	Mo
Standard	AbsenceRe...	AbsenceRe...	delete	Interpreted...		FGR.DELETE		Standard->...	Fehlgründe	
Standard	AbsenceRe...	AbsenceRe...	insert	Interpreted...		FGR.INSERT		Standard->...	Fehlgründe	
Standard	AbsenceRe...	AbsenceRe...	list	Interpreted...				Standard->...	Fehlgründe	
Standard	AbsenceRe...	AbsenceRe...	lock	Interpreted...				Standard->...	Fehlgründe	
Standard	AbsenceRe...	AbsenceRe...	new	Interpreted...				Standard->...	Fehlgründe	
Standard	AbsenceRe...	AbsenceRe...	unlock	Interpreted...				Standard->...	Fehlgründe	
Standard	AbsenceRe...	AbsenceRe...	update	Interpreted...				Standard->...	Fehlgründe	
Standard	AbsenceSet...	AbsenceSet...	delete	Interpreted...				Standard->...	Steuerung ...	
Standard	AbsenceSet...	AbsenceSet...	insert	Interpreted...				Standard->...	Steuerung ...	
Standard	AbsenceSet...	AbsenceSet...	list	Interpreted...				Standard->...	Steuerung ...	

8. Select the U_ServiceExample domain in the domain view. The selected domain becomes the active domain which is used to filter the services. (This is only possible with an active relation from domain to service).
9. The services can be inserted using the context menu in the service view. The active filter (set before) defines which of the copied services can be added to the new domain. All included service parameters are automatically copied at the same time.
Of course you can also use proven key combinations, e.g. Ctrl+C for copying, Ctrl+X for cutting, as well as Ctrl+V for pasting/inserting.

10. At this point, an adjustment of the service names is required. For example, you can change the names using the *Find and replace* dialog that is also available via the context menu:



11. Adjust / remove / add service parameters in known manner.
 12. Save.

The files have been written to the specified location in the hard disk.

13. *Optional*: Export

You can directly write into a structure, which you can use to directly test your changes.

This example could well have been extended. But to directly start work with the client, the example used illustrates the major steps to get a first idea.

At this point, you might ask how you have to proceed with the GUI part of the services. Of course you could also copy them into the new domain and adjust them. Other option: right-click the domain to create the GUI part of the services automatically and to add potentially missing properties from the created services.

It might be easier to copy the complete domain and to simply delete the elements that are not required.

One level further down, the properties of the structure become even more evident. If only a few service parameters of a service are required for a new one, it might be easier to copy the complete service and to delete the excessive parameters. This spares the entire "Creation" of a new service.

9.2 Context menu of the table view/grid

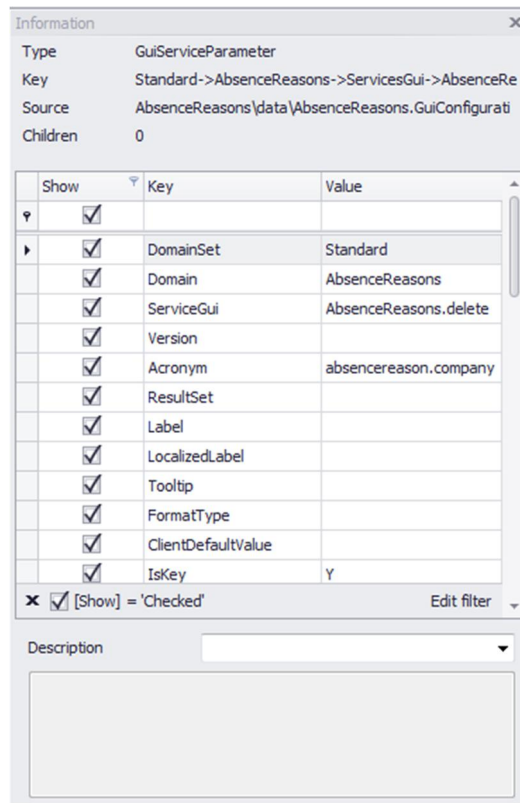
A context menu opens if you right-click the tables. The menu includes different entries depending on the type and status of the table view.

New

Use this function to add a new row to the table view. In the columns with set filters, the values will be set according to the filters in the new row. If, for example, a filter is set to "LIKE Test%" or "= Test", the cell value is set to "Test". Advanced filters are not supported.

Info

Click *Info* to open an InfoPanel. The panel is bound to the source table and shows information on the selected data record in the source table. In addition to a clear identification of the data record, the data source from which it was loaded is shown. The entry *Children* shows the number of data records allocated. In the given example, the service parameter has 2 children service parameters. In addition, the service attribute values are listed in a table. In the bottom area of the dialog, the description stored for the data record is shown.



Copy

Deposits selected rows into the clipboard. This option is only offered if the view contains data.

Cut

Deposits selected rows into the clipboard and subsequently deletes them. This option is only offered if the view contains data.

Delete

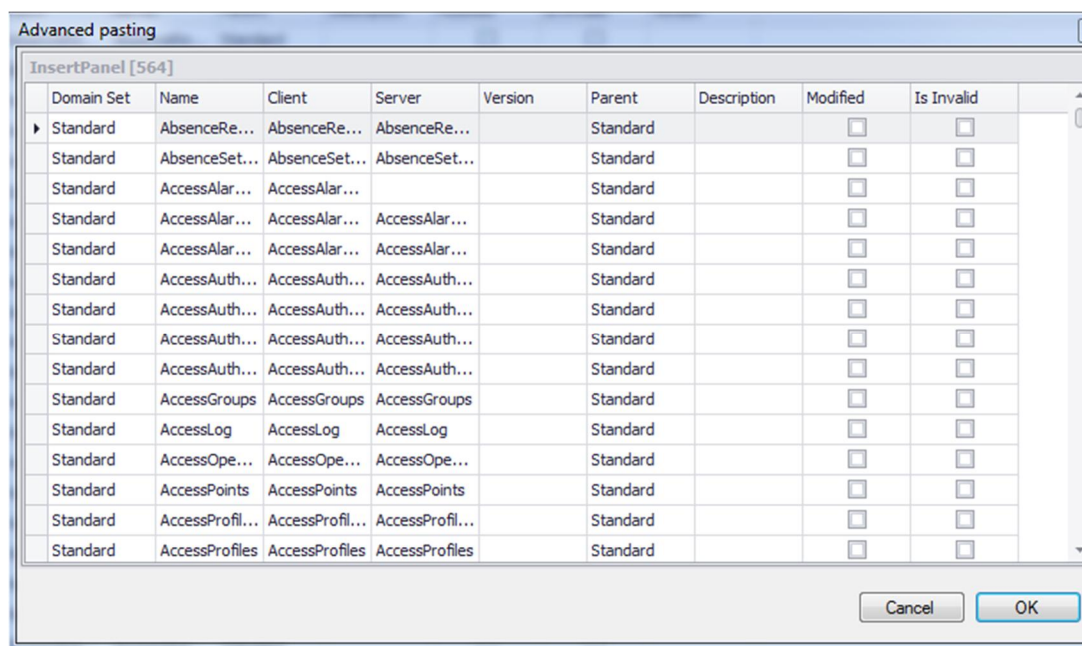
If you select this function, a dialog listing the data records to be deleted is shown. Click "Yes" to delete them; "No" will cancel the deletion process.

Insert

This function adds rows from the clipboard to the grid. This option is only offered if the cache/clipboard contains data which may be inserted in the currently shown table.

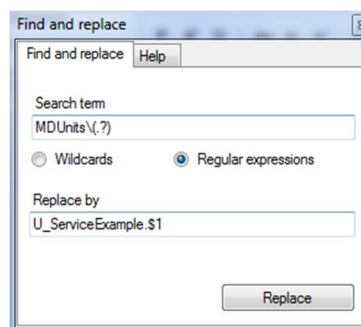
Advanced pasting

Contrary to 'Insert', this function opens a dialog that allows to edit the entries in the cache/clipboard before you insert them. It is possible to allocate new values to individual cells and to all cells of a column. You can cancel the *Insert* process. You can only select this option if the cache/clipboard contains data which may be inserted in the currently shown table.



Find and replace

Use this function to find and replace values within a column. If you select this function, the following dialog opens:



Specify the search term and the term that should replace the search term and confirm by "Replace". For example, use this dialog to replace prefixes. In addition, this function supports regular terms.

Relations

This entry leads to a list with identifiers of relations that are defined in the relations table. If you select one of these identifiers, a list of the currently shown tables opens. Select one of these tables to instantiate this relation. A new relation of this type is automatically created; the source is set on the current table and the target on the selected table, respectively. For details on the semantics of relations, please refer to section Relations.

Show reference

You can use this function to open a table view listing the data to which the entry of the current cell refers. For details on the semantics of references, please refer to section References.

Get references

This entry opens a new table which will fill the current data record with values from the referenced data records. For details on the semantics of this function, please refer to section References.

Create GUI configuration

This entry is only shown in the context menu of the domain panel. Use this function to create the ServicesGUI according to the services of the selected domain.

Create properties

Also this entry is only available in the domain panel. Use this entry to create properties according to the ServiceParameterGui.

Create service based on SQL

Also this entry is only available in the domain panel. You can use this function to generate services. You use an SQL statement to extract information on fields and tables.

- A "select" statement generates a service of type InterpretedJavaService.
- A "create table" statement generates services of type InterpretedBapiService to edit data and a list service of type InterpretedJavaService to show the respective data.

The information included in existing parameters in the repository is added to the information on the individual fields, if possible (the allocation is based on the table and the field name).

Note: This function only helps to create services. It is up to the user to ensure the correctness.

Example 1 ("select" statement)

```
select m.masch_nr,  
       m.bez_lang,  
       k.bezeichnung,  
       k.sap_logical_system  
from   maschinen m  
       left outer join kostenstellen k  
           on k.kostenstelle = m.kostenstelle
```

This statement is used to generate a list service. No existing acronym for the column `k.sap_logical_system` can be found. For this reason, the column is marked in the "acronym" with "<TODO>" and the acronym must be defined manually (delete <TODO>). Then you can run the list service.

Example 2 ("create table" statement)

```
create table u_test_table  
(  
    test_string char(20),  
    test_date   date,  
    test_integer integer,  
    test_decimal decimal(18, 6),  
    test_serial serial  
);
```

This statement requires more manual rework:

- Check acronyms
- If you use the database type "serial": with database type "serial", you must assign `WebServiceType=integer`. For the services delete, lock, unlock and update, you must include the constraint "SERIAL|" in the serial column and specify it as mandatory parameter (`IsMandatory=Y`). For the service insert, make the settings `IsMandatory=N`, `IsResult=Y` and `IsSpecialParameter=N`.
- For the editing services, you can define key columns (except for serials) if required (Constraint "KEY=n|") and define them as mandatory parameters (`IsMandatory=Y`)
- The services delete, lock and unlock only require the key columns (constraint "KEY=n" or "SERIAL|"). Delete the columns that are not used.
- Check `WebServiceType` with all service parameters and complete, if required.
- Also respect the further notes on the generation of services in this document.

Then you can run the service.

9.3 Export

Use the application menu ([Repository](#) → [Export repository](#)) to activate the export dialog. Here, you can make a detailed selection of the data records that you want to export. Settings in this dialog are displayed again when re-opening the dialog.

In the "Domain filter" area you can specify the domain set that you want to export. If you do not make any entry here, all domain sets are exported. In addition, you can set a filter for the domains that you want to export. If you do not set any filter, all domains of the relevant domain set are exported. In the "File filter" area, you can specify which data types are to be exported.

In the "Export paths" area, you can store and select up to three paths for export. For each path, you can also specify the export structure that you want to use.

- **Client Domain:** Data in this structure can be read by the Repository Client.
- **Server Domain:** Data in this structure can be read by the Repository Client.
- **Client Runtime:** Data in this structure can be read and processed by the client.
- **Server runtime:** Data in this structure can be read and processed by the server.

Start the data export via "Export". When the export is completed, a dialog opens showing the number of exported data records.

9.4 Validation

The Repository Client provides an integrated validation function checking the syntax of the columns (or property) to be edited and the syntax of a data record itself. The validation function also checks the consistency between several data records (data types), in particular master-detail relations of individual domains, and it provides a multiple validation and a validation subject to a function type or data type (e.g. Service --> ServiceType). This function is performed when you edit data or when you click the validation button.

10 Interpreted Java Service2

1.1 Introduction

Interpreted Java services version 2 are web services that are converted by an interpreter to SELECT SQL statements. The result is converted to a web service result, once the SQL statement has been executed.

The definition for the interpreter is created in XML files using the repository client.

10.1 Availability

As of SP7

10.2 Definition

An interpreted Java service is defined for the repository (further information on the required values and their meaning can be found in the section entitled "repository data").

The service domain can be exported as XML file, once the definition has been completed. The resulting files (the ones relevant to the interpreter) are **<Domain>.Configuration.xml** and **<Domain>.do.xml**.

10.3 Storage in a server

Both files are stored on a server in

JHYDRADIR\MOC\<MANDANT>\listInterpreter\<Scope>. The scope can have one of the following values: standard, custom or local.

10.4 Available Special Parameters

Each interpreted Java service includes special parameters that are always available and can always be used. Subject to how the interpreted Java service is customized in the file <Domain>.do.xml, the parameters affect processing or are ignored.

These parameters are available:

Name	Data type	Operators	Description
checkresponsibilityarea	boolean	EQUAL	Is only effective if checkRespAreaMode and checkRespAreaField are configured. If false, the responsibility area will not be checked. If true, it

			will be checked. The default value is true.
checkresponsibilityareafunctions	String[]	EQUAL, IN	Is only effective, if checkRespAreaMode and checkRespAreaField are configured and checkresponsibilityarea == true. This parameter controls which functions are checked by the responsibility area. The default value is select. The following functions can be indicated: create (vab_tab.anlegen='J') delete (vab_tab.loeschen='J') select (vab_tab.anzeigen='J') update (vab_tab.aendern='J') use (vab_tab.verwenden='J')
longtermdata	boolean	EQUAL	Is only effective if tableClauseLongterm is configured. If false, no long-term data will be used. If true, it will be checked. The default value is false.

10.5 Repository data

10.5.1 Tab Services

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Name	The complete service name, i.e. Domain.Function (e.g. BOPerson.list)	
Service Function	The function of the service (e.g. list)	
Service Type	The type - for interpreted Java services: fixed InterpretedJavaService2	
Description (German)	Brief (internal) description of the service	

10.5.2 Tab ServiceParameter

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Service Function	The function of the service (e.g. list)	
Service	The complete service name, i.e. Domain.Function (e.g. BOPerson.list)	
Acronym	Acronyms (e.g. person.id) have to be unique for service and result set.	
Web Service Type	The data type of the parameter (decimal, integer, string, boolean, datetime)	
DB table	The table that is used to select the value for the acronym	
DB Field	The field from which the value for the acronym is to be selected This is either just the field name or the expression (if it is a calculated field) including placeholder for the table alias (e.g. <code>hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time)</code> or <code>{fn substring(%1\$s.field,2,1)}</code>). The placeholder for the table alias is always <code>"%1\$s"</code> .	
DB Alias	The table alias for the table that is used to select the value for the acronym	
Conversion Method	Here you can specify transformations for input and result parameters (e.g. conversion bool to J/N and vice versa or the correct filtering for datetime fields that consist of two fields in the database). Possible transformations are described elsewhere.	X
Can ...	Have to be set for filter parameters (for Boolean only Can Equal; for string all and for the others, everything except Can Like and Can Like or null)	X (if only Result)
IsFilterParameter	Specifies whether or not the field is a filter field. A filter parameter including its operator is directly converted into an SQL fragment	X (if only Result)

IsResult	Specifies whether or not it is a Result	X (if only Filter)
IsSpecialParameter	Specifies whether or not a field is a special parameter. At the moment, standard processing only supports the above-mentioned special parameters. Further parameters can be used in exits. Special parameters are "options" that cannot directly be converted into an SQL fragment of the type <DB field> <Operator> <Value>.	X
IsMandatory	Specifies whether or not it is a mandatory field. If TRUE and the parameter is missing, an error message is generated at runtime. Is checked with special parameters and filter parameters.	X
InputAsArray	Specifies whether the field also supports arrays as input parameters. (e.g. the operators IN or BETWEEN require an array as input parameter)	
ConditionalFieldKey	This field specifies if a DB field is only conditionally available. The condition as to whether the field is available is checked using the Configuration Manager (see Configuration Manager in the section "server"). The feature key of the Configuration Manager (feature set) has to be entered in this field within the repository for checking.	X
DBFieldAlternative	Only relevant if it is a conditional field. Here you can enter the alternative value. This value can be a figure, null, 'string', {fn ...}, or even another field / subselect. Default value is zero if nothing else is entered. Note: This field has a different behavior than "DB Field". The alias is not automatically put in front. If you want to use the alias of the field "DB Alias", you must put %1\$. in front of the field name.	X
Constraints	If the value "SKIP_INTERPRETER " is entered here, the interpreter ignores this acronym. This is useful, if you want to edit or add acronyms in an exit.	X

10.5.3 Tab Dataobjects

Name	Meaning	Optional
Name	Name of the data source. Must be identical to the complete service name, i.e. domain.function (e.g. BOPerson.list).	
orderBy	Specifies sorting (with the real alias and not %1\$s. in front of the field name) Please note: Do not use if "groupByCols" is applied. This might lead to SQL errors if sorting is based on a field that is not included in the "group by" clause (in case the client has not requested it).	X
groupByCols	Specifies the "group by" fields (in the order) for the SQL statement. The value includes a list of acronyms including their database field (with the real alias and not %1\$s. in front of the field name). The interpreter only adds the fields requested by the client (including their acronyms) to the group by clause. For example: Acronym1=alias.field1 Acronym2=alias.field2	X
filterBy	Specifies fixed filters (with the real alias and not %1\$s. in front of the field name)	X
checkRespAreaMode	Checks the responsibility area of the current user. Modes: "none": no check direct: directly checked by a field of the data source (joined to vab_tab). Use of --DEFAULT-- if empty or null directnotempty: check directly via a field of the data source (joined to vab_tab) "person": check via VAB of person "machine": check via VAB of machine	X
checkRespAreaField	Specifies the field including the responsibility area for	X

	direct/directnotempty. Join field if person or machine (with the real alias and not %1\$s. in front of the field name)	
checkRespAreaDefaultValue	Specifies the default value for checking the responsibility area (if the parameter has not been specified by the client). The default value is true. Valid values are true and false.	X
dataTabLabel	Name of the resulting result set	
tableClause	Specifies the table clause without the key word FROM (only relevant if several tables are used).	
tableClauseLongterm	Specifies the table clause for long-term data (only relevant if several tables are used or if long-term data exist). If nothing is entered the tableClause will be used as tableClauseLongterm. If this one is neither indicated, the first table and its alias found for a service parameter will be used.	X
conditionalLongtermKey	This value specifies if the long-term data tables are only conditionally available. The condition as to whether the tables are available is checked by the Configuration Manager. The feature key of the Configuration Manager has to be entered here for checking.	X
mergeOnAttributeLevel	Attribute available as of SP11: This attribute controls how individual DataObjects are merged over several scopes (a single DataObject is identified by the attribute "name"). If the attribute mergeOnAttributeLevel does not exist or the value is not equal "Y", the behavior is the same than before SP11 (backward compatibility). This means that the whole configuration of the DataObjects (not the whole file, but only the entire row) is completely overwritten by a higher scope. For example, if a filterBy is introduced in the custom scope, the complete DataObject in the standard scope is replaced. If the standard is then extended, the standard extension is not applied in the	

	custom scope. If the attribute is set to "Y", the merge behavior changes and one attribute is merged after the other and not the complete DataObject at a time. Refer to the following subchapter for details. (This setting is the default setting for new DataObject configurations as of SP11; existing older configurations are not changed). You can find details and examples in the next subchapter.	
--	---	--

10.5.3.1 Rules for the merge of SQL attributes on attribute level

Only "name" is a mandatory attribute if you merge DataObjects on attribute level. The other values are taken over from the lower scope.

The following applies for most attributes:

- If the attribute is empty, the value of the lower scope **is taken over**.
- If the attribute is populated, the value of the lower scope **is overwritten**.

These SQL attributes are an exception:

- tableClauseLongterm
- tableClause
- filterBy
- groupByCols
- orderBy

The following applies with these SQL attributes:

- If the attribute is empty, the value of the lower scope **is taken over**.
- If the attribute is populated, the value of the lower scope **is written in front** (the value of the higher scope is written behind the value of the lower scope). A specific separator is used for the separation:
 - o tableClauseLongterm: a space character
 - o tableClause: a space character
 - o filterBy: an "AND"; in addition the lower scope is put in parentheses. If the higher scope does not include a key word, also the higher scope is put in parentheses.
 - o orderBy: a comma (",")
 - o groupByCols: a pipe ("|")
- If the attribute includes the key word \$NO_LOWER_SCOPE\$, the value of the lower scope is completely replaced by the value of the attribute.

If the attribute includes the key word \$LOWER_SCOPE_VALUE\$, the lower scope is copied to exactly this position (with the specific separator).

10.5.3.2 Examples of the merge of SQL attributes on attribute level

As of SP11

The following examples only refer to the attributes tableClauseLongterm, tableClause, filterBy, orderBy and groupByCols. The other attributes follow a very simple pattern: if the attribute in a higher scope is populated, the lower scope is replaced. If the attribute in the higher scope is not populated, the lower scope is used.

In the following, you can find examples of the behavior, if you merge on attribute level (mergeOnAttributeLevel=Y). We used "filterBy" in our examples. The first row is the lower scope (e.g. standard), the second row is the higher scope (e.g. custom) and the third row is the result of the merge.

```
"xy.field_x = '42'",
"foo.field_y = '24'",
"(xy.field_x = '42') and (foo.field_y = '24')"
```

```
"xy.field_x = '42'",
"",
"xy.field_x = '42'"
```

```
"",
"foo.field_y = '24'",
"foo.field_y = '24'"
```

```
"xy.field_x = '42'",
"foo.field_y$LANG = 'foo' $LOWER_SCOPE_VALUE$",
"foo.field_y$LANG = 'foo' and (xy.field_x = '42')"
```

```
"xy.field_x = '42'",
"foo.field_y = '24' $LOWER_SCOPE_VALUE$",
"foo.field_y = '24' and (xy.field_x = '42')"
```

```
"",
"foo.field_y = '24' $LOWER_SCOPE_VALUE$",
"foo.field_y = '24'"
```

```
"xy.field_x = '42'",
"foo.field_y = '24' and $LOWER_SCOPE_VALUE$ bar.field_z = 'world'",
"foo.field_y = '24' and (xy.field_x = '42') and bar.field_z = 'world'"
```

```
"",
"foo.field_y = '24' and $LOWER_SCOPE_VALUE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"
```

```
"xy.field_x = '42'",
"$LOWER_SCOPE_VALUE$ foo.field_y = '24'",
"(xy.field_x = '42') and foo.field_y = '24'"
```

```
"",
"$LOWER_SCOPE_VALUE$ foo.field_y = '24'",
"foo.field_y = '24'"
```

```
"xy.field_x = '42'",
"foo.field_y = '24' $NO_LOWER_SCOPE$",
"foo.field_y = '24'"
```

```

""
"foo.field_y = '24' $NO_LOWER_SCOPE$",
" foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y = '24' and $NO_LOWER_SCOPE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"

""
"foo.field_y = '24' and $NO_LOWER_SCOPE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"

"xy.field_x = '42'",
"$NO_LOWER_SCOPE$ foo.field_y = '24'",
" foo.field_y = '24'"

""
"$NO_LOWER_SCOPE$ foo.field_y = '24'",
" foo.field_y = '24'"

"satz_art = 'U'",
"$NO_LOWER_SCOPE$ masch_nr = '4711' $LOWER_SCOPE_VALUE$",
" masch_nr = '4711' and (satz_art = 'U')"
```

```

"satz_art = 'U'",
"masch_nr = '4711' $NO_LOWER_SCOPE$LOWER_SCOPE_VALUE$",
"masch_nr = '4711' LOWER_SCOPE_VALUE$" => invalid SQL
```

```

"satz_art = 'U'",
"masch_nr = '4711' $LOWER_SCOPE_VALUE$NO_LOWER_SCOPE$",
"masch_nr = '4711' (satz_art = 'U') andNO_LOWER_SCOPE$" => invalid SQL
```

```

"satz_art = 'U'",
"masch_nr = '4711' $LOWER_SCOPE_VALUE$ $LOWER_SCOPE_VALUE$",
"masch_nr = '4711' (satz_art = 'U') and $LOWER_SCOPE_VALUE$" => invalid SQL
```

```

"satz_art = 'U'",
"$LOWER_SCOPE_VALUE$ masch_nr = '4711' $LOWER_SCOPE_VALUE$",
"(satz_art = 'U') and masch_nr = '4711' $LOWER_SCOPE_VALUE$" => invalid SQL
```

10.6 Exits

Exits provide the entry points to enable changes to the defined behavior by programming.

10.6.1 Available user exits

User exits provide entry points that are not modified by releases. In case of releases, the interfaces are respected and preserved by MPDV in a backwards compatible manner.

10.6.1.1 `sdilInterpretedSqlModifyRequest`

The application developer can change the parameters and the column configurator using this exit. You can also create temporary tables as this exit includes access to the DB session which is also used by the main SQL.

10.6.1.2 sdiAddResultTransformationCallbacks

Using this exit, the application developer can modify the service result after having executed the SQL statement. Rows can be deleted, added or changed. The application developer registers a callback to this end, which is called for each row.

10.6.1.3 sdiInterpretedSqlCleanup

Using this exit, the application developer can undertake cleanup actions. This exit is always executed, whether or not errors occurred. Here you can e.g. clean up temporary tables as you can access the DB session of the main SQL.

10.6.2 Available program exits

Program exits offer extended functionality that is not backwards compatible. Changes can be made with every update. Users of program exits must test if their modifications still work after an update.

Therefore, program exits are of limited value with modifications.

10.6.2.1 sdiAugmentSql

With this user exit, the interpreted Java service provides the application developer with an option to extend the generated SQL shortly before it is executed.

10.6.3 Specifications for the implementation class of the exit

Package name: the class has to be included in a package with the domain name (in lower case letters). Further sub-packages are **not** allowed.

Example: The service is requested "MDUserAccountRules.list", consequently the package is called "mduseraccountrules"

Class

name:

The class must have a name of the following structure: domain name in lower case letters, whereas the first letter is written with a capital letter, the name of the service function follows and is written in lower case letters. The first letter is again written in upper case.

Example: The service is requested "MDUserAccountRules.list", consequently the class is called "MduseraccountrulesList"



The following definition applies for customized class names:
Customized names include "_" (see naming conventions)

After "_" the first letter is changed to upper case and the underscore is skipped

Example: U_CUST_Units_sample.list => UCustUnitsSampleList

Implemented interfaces / methods: no specifications

Other: The class must have a default constructor without parameters

Compilation: The Jar files *MpdvDomCoreSdiCompileLib.jar* and *MpdvDomCoreUserExitCompileLib.jar* must be included in the class path for the compilation process.

Deployment: The class file of the exit must be included in the directory
<JHYDRADIR>/MOC/<instance>/userexit/<scope> including package directory structure.

Example: Exit sdiAugmentSql for service "MDUserAccountRules.list":

```
package mduseraccountrules;

import de.mpdv.customization.userExit.IUserExitParam;

/**
 * Sample user exit
 *
 */
public class MduseraccountrulesList
{
    public void sdiAugmentSql(final IUserExitParam param)
    {
        // TODO implementation
    }
}
```

Directory structure in the server (instance 1, scope custom):

<JHYDRADIR>/MOC/1/userexit/custom/mduseraccountrules/MduseraccountrulesList.class

10.6.4 Interfaces

10.6.4.1 Class: InterpretedJavaServiceUeContext

de.mpdv.sdi.data.ue.InterpretedJavaServiceUeContext
-hydraNow: Calendar
-userId: String
+getHydraNow(): Calendar
+getUserId(): String

This context class provides data that is generally useful for interpreted Java services as regards to exits.

Field	Description
hydraNow	The property "hydraNow" is a time stamp created at the beginning of web service processing and, as a result, can be used as reference time stamp for the current web service call.
userId	Includes the user logged on to the client.

10.6.4.2 User exit: `SdiInterpretedSqlModifyRequest`

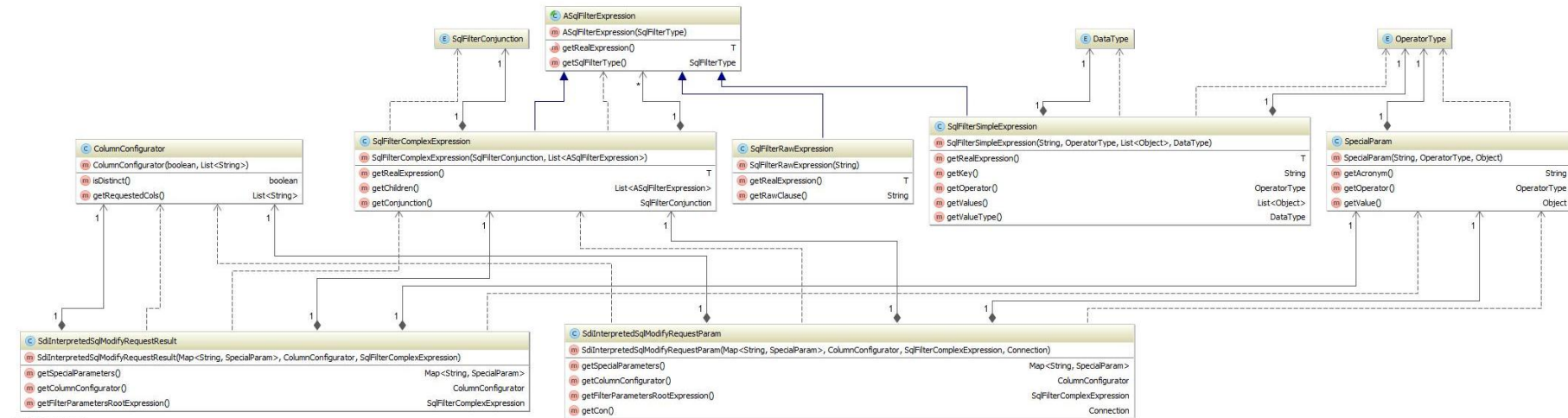
Parameter key in `IUserExitParam`:

Key name	Type	Description
param	<code>SdiInterpretedSqlModifyRequestParam</code>	Parameter structure for the user exit
context	<code>InterpretedJavaServiceUeContext</code>	Context structure for all exits of the interpreted Java Services
factory	<code>ISystemUtilFactory</code>	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in `IUserExitParam`:

Key name	Type	Description
result	<code>SdiInterpretedSqlModifyRequestResult</code>	Result structure for the user exit

Class diagram of parameter and result structures:



Powered by yFiles

10.6.4.3 Class *SdiInterpretedSqlModifyRequestParam*

Field	Type	Description
columnConfigurator	ColumnConfigurator	Includes the columns requested by the client
specialParameters	Map<String, SpecialParam>	Assignment of acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"
filterParametersRootExpression	SqlFilterComplexExpression	Root node of the SQL filter tree. All parameters of type "isFilterParameter" are included in this tree.
con	Connection	DB session that is also used by the main SQL.

10.6.4.4 Class *SdiInterpretedSqlModifyRequestResult*

Field	Type	Description
columnConfigurator	ColumnConfigurator	Includes the modified column configuration
specialParameters	Map<String, SpecialParam>	Includes the modified SpecialParameters
filterParametersRootExpression	SqlFilterComplexExpression	Includes the modified FilterParameter

10.6.4.5 User exit: *sdiAddResultTransformationCallbacks*

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiAddResultTransformationCallbackParam	Parameter structure for the

		user exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiAddResultTransformationCallbackResult	Result structure for the user exit: Note: If you only want to change the input row, you need not generate a result of the class "SdiModifyResultRowResult" and therefore also the key "result" is not necessary in IUserExitParam. A result of the class "SdiModifyResultRowResult" is generated, if rows are deleted or added.

10.6.4.6 Class SdiAddResultTransformationCallbacksParam

Field	Type	Description
specialParameters	Map<String, SpecialParam>	Assignment of acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"
columnConfigurator	ColumnConfigurator	Includes the columns requested by the client

dataRowBuilder	ISdiDataRowBuilder	Using this builder, you can generate the instances ISdiDataRow. A direct implementation of ISdiDataRow is not allowed!
----------------	--------------------	---

10.6.4.7 Class SdiAddResultTransformationCallbacksResult

Field	Type	Description
callbackList	List<ISdiResultTransformationCallback>	Callback list for the result transformation

10.6.4.8 Interface ISdiResultTransformationCallback

Method	Description
transform(ISdiDataRow dataRow, boolean isLastRow): ISdiDataRowStream	Callback method for the result transformation Return type: ISdiDataRowStream must not be NULL: Includes the rows as stream (to support streaming) once the input row has been edited in the user exit. The service then returns the stream rows as result to the client. You can use the class SdiEagerDataRowStream to modify the current row and to return few result rows. If you want to return large amounts of data, you must implement a stream that takes data rows from an external data source. If you must create instances of ISdiDataRow, you must use ISdiDataRowFactory from SdiAddResultTransformationCallbacksParam. A direct implementation of ISdiDataRow is not allowed! Input: ISdiDataRow dataRow: Includes a result row that the interpreter creates as service result.

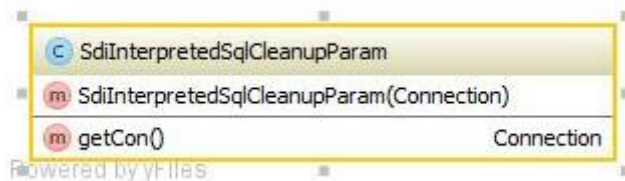
	isLastRow boolean: TRUE, if this row is the last row, otherwise FALSE
--	---

10.6.4.9 User exit: sdiInterpretedSqlCleanup

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiInterpretedSqlCleanupParam	Parameter structure for the user exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Class diagram of the parameter structure:



10.6.4.10 Class SdiInterpretedSqlCleanupParam

Field	Type	Description
con	Connection	DB session that is also used by the main SQL.

10.6.4.11 Program exit: sdiAugmentSql

Parameter key in IUserExitParam:

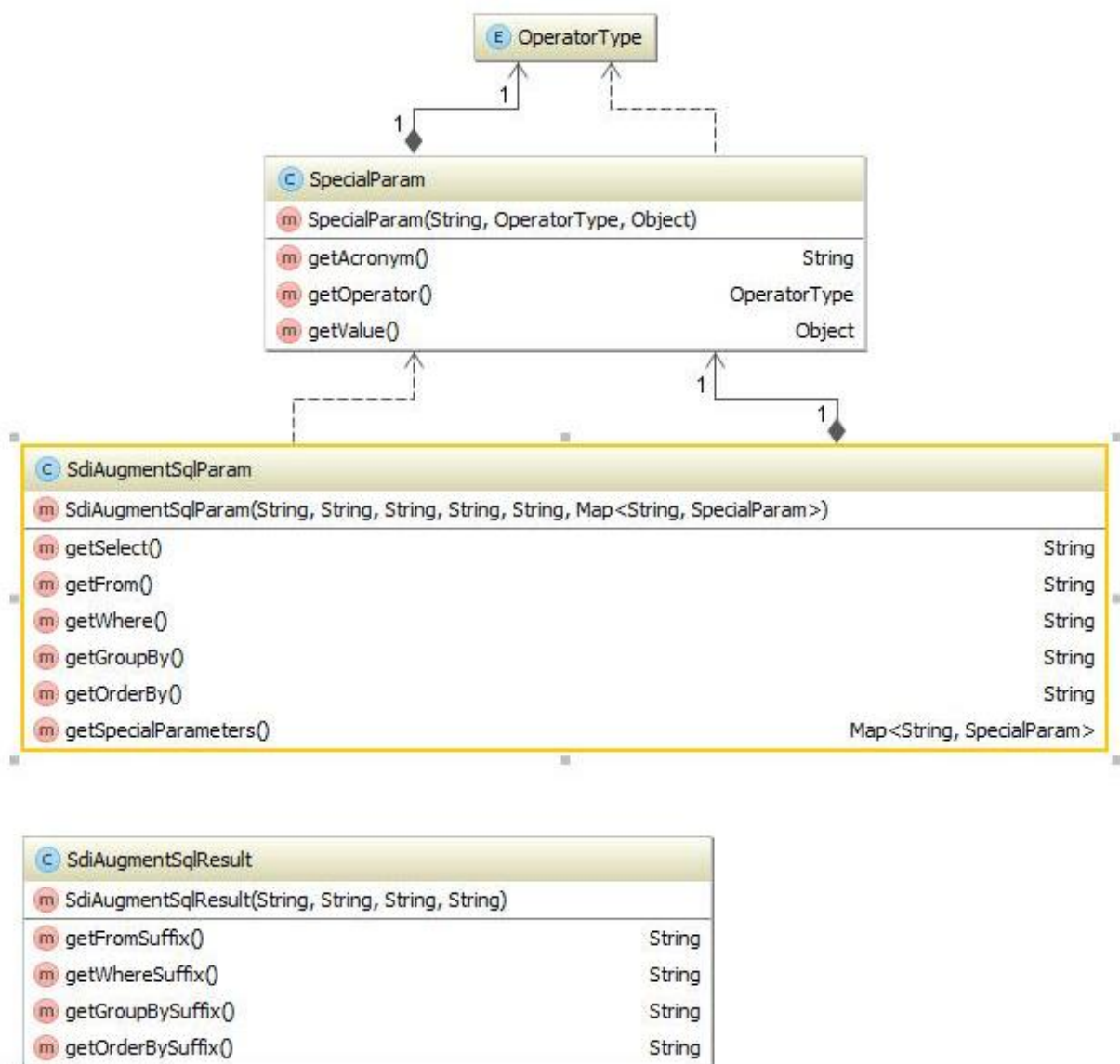
Key name	Type	Description
param	SdiAugmentSqlParam	Parameter structure for the program exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of

		the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiAugmentSqlResult	Result structure for the program exit

Class diagram of parameter and result structures:



Powered by yFiles.

10.6.4.12 Class SdiAugmentSqlParam

Field	Type	Description
select	String	SELECT clause created based on the configuration (only includes the column part up to FROM, only without the key word SELECT)
from	String	FROM clause created based on the configuration (without the key word FROM)
WHERE	String	WHERE clause created based on the configuration (without the key word WHERE)
groupBy	String	GROUP BY clause created based on the configuration
orderBy	String	ORDER BY clause created based on the configuration
specialParameters	Map<String, SpecialParam>	Assignment of acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

10.6.4.13 Class SdiAugmentSqlResult

Field	Type	Description
fromSuffix	String	Suffix for the FROM clause created in the program exit or NULL
whereSuffix	String	Suffix for the WHERE clause created in the program exit or NULL
groupBySuffix	String	Suffix for the GROUP BY clause created in the program exit or NULL
orderBySuffix	String	Suffix for the ORDER BY clause

		created in the program exit or NULL
--	--	-------------------------------------

11 Interpreted Java Service

11.1 Introduction

Interpreted Java services are web services that are converted by an interpreter to SELECT SQL statements. The result is converted to a web service result, once the SQL statement has been executed.

The definition for the interpreter is created in XML files using the repository client.

If you create new services, use the type `InterpretedJavaService2` instead of the `InterpretedJavaService`.

The `InterpretedJavaService2` type is prepared for the future streaming of data and offers more options for Java user exits.



As long as no Java user exits are used, it is still easy to convert the `InterpretedJavaService` type into the `InterpretedJavaService2` type by simply changing the service type. There are the following differences for services without Java user exits.

- The column `DataObjectName` for `InterpretedJavaService2` is not required in the repository data of service and should be set to empty.
- The column `parameterReference` for `InterpretedJavaService2` is not required anymore for the repository data of the `DataObjects` and should be set to empty.

11.2 Definition

An interpreted Java service is defined for the repository (further information on the required values and their meaning can be found in the section entitled "repository data").

The service domain can be exported as XML file, once the definition has been completed. The resulting files (the ones relevant to the interpreter) are **<Domain>.Configuration.xml** and **<Domain>.do.xml**.

11.3 Storage in a server

You can find both files on the server in

```
jhydradir\MOC\<MANDANT>\listInterpreter\<Scope>    bzw.  
wsp_config\MOC\<MANDANT>\listInterpreter\<Scope>
```

There are the following value: standard, custom or local.

11.4 Available Special Parameters

Each interpreted Java service includes special parameters that are always available and can always be used. Subject to how the interpreted Java service is customized in the file <Domain>.do.xml, the parameters affect processing or are ignored.

These parameters are available:

Name	Data type	Operators	Description
checkresponsibilityarea	boolean	EQUAL	Is only effective if checkRespAreaMode and checkRespAreaField are configured. If false, the responsibility area will not be checked. If true, it will be checked. The default value is true.
checkresponsibilityareafunctions	String[]	EQUAL, IN	Is only effective, if checkRespAreaMode and checkRespAreaField are configured and checkresponsibilityarea == true. This parameter controls which functions are checked by the responsibility area. The default value is select. The following functions can be indicated: create (vab_tab.anlegen='J') delete (vab_tab.loeschen='J') select (vab_tab.anzeigen='J') update (vab_tab.aendern='J') use (vab_tab.verwenden='J')
longtermdata	boolean	EQUAL	Is only effective if tableClauseLongterm is configured. If false, no long-term data will be used. If true, it will be checked. The default value is false.

11.5 Repository data

11.5.1 Tab Services

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Name	The complete service name, i.e. Domain.Function (e.g. BOPerson.list)	
Service Function	The function of the service (e.g. list)	
Service Type	The type - for interpreted Java services: fixed InterpretedJavaService	
Description (German)	Brief (internal) description of the service	

11.5.2 Tab ServiceParameter

Name	Meaning	Optional
Domain	The domain of the service (e.g. BOPerson)	
Service Function	The function of the service (e.g. list)	
Service	The complete service name, i.e. Domain.Function (e.g. BOPerson.list)	
Acronym	Acronyms (e.g. person.id) have to be unique for service and result set.	
Web Service Type	The data type of the parameter (decimal, integer, string, boolean, datetime)	
DB table	The table that is used to select the value for the acronym	
DB Field	The field from which the value for the acronym is to be selected This is either just the field name or the expression (if it is a calculated field) including placeholder for the table alias (e.g. hydadm.get_datetime(%1\$s.bearb_date,%1\$s.bearb_time) or {fn	

	substring(%1\$s.field,2,1)). The placeholder for the table alias is always "%1\$s".	
DB Alias	The table alias for the table that is used to select the value for the acronym	
Conversion Method	Here you can specify transformations for input and result parameters (e.g. conversion bool to J/N and vice versa or the correct filtering for datetime fields that consist of two fields in the database). Possible transformations are described elsewhere.	X
Can ...	Have to be set for filter parameters (for Boolean only Can Equal; for string all and for the others, everything except Can Like and Can Like or null)	X (if only Result)
IsFilterParameter	Specifies whether or not the field is a filter field. A filter parameter including its operator is directly converted into an SQL fragment	X (if only Result)
IsResult	Specifies whether or not it is a Result	X (if only Filter)
IsSpecialParameter	Specifies whether or not a field is a special parameter. At the moment, standard processing only supports the above-mentioned special parameters. Further parameters can be used in exits. Special parameters are "options" that cannot directly be converted into an SQL fragment of the type <DB field> <Operator> <Value>.	X
IsMandatory	Specifies whether or not it is a mandatory field. If this is true and the parameter is missing, an error message is generated at runtime. Is currently only checked for special parameters.	X
InputAsArray	Specifies whether the field also supports arrays as input parameters. (e.g. the operators IN or BETWEEN require an array as input parameter)	
DataObjectName	Name of the interpreted Java Service. Used as reference for the ...do.xml configuration	
ConditionalFieldKey	This field specifies if a DB field is only conditionally available. The condition as to whether the field is available is checked using the	X

	Configuration Manager (see Configuration Manager in the section "server"). The feature key of the Configuration Manager (feature set) has to be entered in this field within the repository for checking.	
DBFieldAlternative	Only relevant if it is a conditional field. Here you can enter the alternative value. This value can be a figure, null, 'string', {fn ...}, or even another field / subselect. "%1\$s." NEEDS to be entered for the alias if it is another field or subselect! The default value is null if nothing is entered.	X

11.5.3 Tab Dataobjects

Name	Meaning	Optional
Name	Name of the data source. References to the field DataObjectName for the repository (to connect the ServiceParameter)	
parameterReference	References to the service name in order for the correct parameters to be determined	
orderBy	Specifies sorting (with the real alias and not %1\$s. in front of the field name) Please note: Do not use if "groupByCols" is applied. This might lead to SQL errors if sorting is based on a field that is not included in the "group by" clause (in case the client has not requested it).	X
groupByCols	Specifies the "group by" fields (in the order) for the SQL statement. The value includes a list of acronyms including their database field (with the real alias and not %1\$s. in front of the field name). The interpreter only adds the fields requested by the client (including their acronyms) to the group by clause.	X

	For example: Acronym1=alias.field1 Acronym2=alias.field2	
filterBy	Specifies fixed filters (with the real alias and not %1\$s. in front of the field name)	X
checkRespAreaMode	Checks the responsibility area of the current user. Modes: "none": no check direct: directly checked by a field of the data source (joined to vab_tab). The value --DEFAULT-- is used if empty or null directnotempty: directly checked by a field of the data source (joined to vab_tab) "person": checked by the person's responsibility area "machine": checked by the responsibility area of the machine	X
checkRespAreaField	Specifies the field including the responsibility area for direct/directnotempty. Join field if person or machine (with the real alias and not %1\$s. in front of the field name)	X
checkRespAreaDefaultValue	Specifies the default value for checking the responsibility area (if the parameter has not been specified by the client). The default value is true. Valid values are true and false.	X
dataTabLabel	Name of the resulting result set	
tableClause	Specifies the table clause without the key word FROM (only relevant if several tables are used). If nothing is entered here, the first table and its alias found for a service parameter will be used as the tableClause.	X
tableClauseLongterm	Specifies the table clause for long-term data (only relevant if several tables are used or if long-term data exist). If nothing is entered the tableClause will be used as tableClauseLongterm. If this one is neither indicated, the first table and its alias found for a service parameter will be used.	X

conditionalLongtermKey	This value specifies if the long-term data tables are only conditionally available. The condition as to whether the tables are available is checked by the Configuration Manager. The feature key of the Configuration Manager has to be entered here for checking.	X
mergeOnAttributeLevel	Attribute available as of SP11: This attribute controls how individual DataObjects are merged over several scopes (a single DataObject is identified by the attribute "name"). - If the attribute mergeOnAttributeLevel does not exist or the value is not equal "Y", the behavior is the same than before SP11 (backward compatibility). This means that the whole configuration of the DataObjects (not the whole file, but only the entire row) is completely overwritten by a higher scope. For example, if a filterBy is introduced in the custom scope, the complete DataObject in the standard scope is replaced. If the standard is then extended, the standard extension is not applied in the custom scope. - If the attribute is set to "Y", the merge behavior changes and one attribute is merged after the other and not the complete DataObject at a time. Refer to the following subchapter for details. (This setting is the default setting for new DataObject configurations as of SP11; existing older configurations are not changed). You can find details and examples in the next subchapter.	

11.5.3.1 Rules for the merge of SQL attributes on attribute level

Only "name" and "parameterReference" are mandatory attributes if you merge DataObjects on attribute level. The other values are taken over from the lower scope.

The following applies for most attributes:

- If the attribute is empty, the value of the lower scope **is taken over**.
- If the attribute is populated, the value of the lower scope **is overwritten**.

These SQL attributes are an exception:

- tableClauseLongterm
- tableClause
- filterBy
- groupByCols
- orderBy

The following applies with these SQL attributes:

- If the attribute is empty, the value of the lower scope **is taken over**.
- If the attribute is populated, the value of the lower scope **is written in front** (the value of the higher scope is written behind the value of the lower scope). A specific separator is used for the separation:
 - o tableClauseLongterm: a space character
 - o tableClause: a space character
 - o filterBy: an "AND"; in addition the lower scope is put in parentheses. If the higher scope does not include a key word, also the higher scope is put in parentheses.
 - o orderBy: a comma (",")
 - o groupByCols: a pipe ("|")
- If the attribute includes the key word \$NO_LOWER_SCOPE\$, the value of the lower scope is completely replaced by the value of the attribute.

If the attribute includes the key word \$LOWER_SCOPE_VALUE\$, the lower scope is copied to exactly this position (with the specific separator).

11.5.3.2 Examples of the merge of SQL attributes on attribute level

As of SP11

The following examples only refer to the attributes tableClauseLongterm, tableClause, filterBy, orderBy and groupByCols. The other attributes follow a very simple pattern: if the attribute in a higher scope is populated, the lower scope is replaced. If the attribute in the higher scope is not populated, the lower scope is used.

In the following, you can find examples of the behavior, if you merge on attribute level (mergeOnAttributeLevel=Y). We used "filterBy" in our examples. The first row is the lower scope (e.g. standard), the second row is the higher scope (e.g. custom) and the third row is the result of the merge.

```
"xy.field_x = '42'",  
"foo.field_y = '24'",  
"(xy.field_x = '42') and (foo.field_y = '24')"
```

```

"xy.field_x = '42'",
"",
"xy.field_x = '42'"

"",
"foo.field_y = '24'",
" foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y$LANG = 'foo' $LOWER_SCOPE_VALUE$",
"foo.field_y$LANG = 'foo' and (xy.field_x = '42')"

"xy.field_x = '42'",
"foo.field_y = '24' $LOWER_SCOPE_VALUE$",
"foo.field_y = '24' and (xy.field_x = '42')"

"",
"foo.field_y = '24' $LOWER_SCOPE_VALUE$",
" foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y = '24' and $LOWER_SCOPE_VALUE$ bar.field_z = 'world'",
"foo.field_y = '24' and (xy.field_x = '42') and bar.field_z = 'world'"

"",
"foo.field_y = '24' and $LOWER_SCOPE_VALUE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"

"xy.field_x = '42'",
"$LOWER_SCOPE_VALUE$ foo.field_y = '24'",
"(xy.field_x = '42') and foo.field_y = '24'"

"",
"$LOWER_SCOPE_VALUE$ foo.field_y = '24'",
" foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y = '24' $NO_LOWER_SCOPE$",
" foo.field_y = '24'"

"",
"foo.field_y = '24' $NO_LOWER_SCOPE$",
" foo.field_y = '24'"

"xy.field_x = '42'",
"foo.field_y = '24' and $NO_LOWER_SCOPE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"

"",
"foo.field_y = '24' and $NO_LOWER_SCOPE$ bar.field_z = 'world'",
"foo.field_y = '24' and bar.field_z = 'world'"

"xy.field_x = '42'",
"$NO_LOWER_SCOPE$ foo.field_y = '24'",
" foo.field_y = '24'"

"",
"$NO_LOWER_SCOPE$ foo.field_y = '24'",
" foo.field_y = '24'"

"satz_art = 'U'",
"$NO_LOWER_SCOPE$ masch_nr = '4711' $LOWER_SCOPE_VALUE$",
" masch_nr = '4711' and (satz_art = 'U')"

"satz_art = 'U'",
"masch_nr = '4711' $NO_LOWER_SCOPE$LOWER_SCOPE_VALUE$",
"masch_nr = '4711' LOWER_SCOPE_VALUE$" => invalid SQL

"satz_art = 'U'",
"masch_nr = '4711' $LOWER_SCOPE_VALUE$NO_LOWER_SCOPE$",
"masch_nr = '4711' (satz_art = 'U') andNO_LOWER_SCOPE$" => invalid SQL

"satz_art = 'U'",
"masch_nr = '4711' $LOWER_SCOPE_VALUE$ $LOWER_SCOPE_VALUE$",

```

"masch_nr = '4711' (satz_art = 'U') and \$LOWER_SCOPE_VALUE\$" => invalid SQL

```
"satz_art = 'U'",  
"$LOWER_SCOPE_VALUE$ masch_nr = '4711' $LOWER_SCOPE_VALUE$",  
"(satz_art = 'U') and masch_nr = '4711' $LOWER_SCOPE_VALUE$" => invalid SQL
```

11.6 Exits

Exits provide the entry points to enable changes to the defined behavior by programming.

11.6.1 Available user exits

User exits provide entry points that are not modified by releases. In case of releases, the interfaces are respected and preserved by MPDV in a backwards compatible manner.

11.6.1.1 sdiModifyColumnConfigurator

With this user exit, the interpreted Java service provides the application developer with an option to modify/extend the column configurator before it is executed.

11.6.1.2 sdiModifyResultList

With this user exit, the interpreted Java service provides the application developer with an option to modify/extend the service result after it is executed.

11.6.2 Available program exits

Program exits offer extended functionality that is not backwards compatible. Changes can be made with every update. Users of program exits must test if their modifications still work after an update.

Therefore, program exits are of limited value with modifications.

11.6.2.1 sdiModifyColumnMap

With this program exit, the interpreted Java service provides the application developer with an option to modify/extend standard assignment of acronym to DB table before it is executed.

11.6.2.2 sdiAugmentSql

With this program exit, the interpreted Java service provides the application developer with an option to modify the generated SQL shortly before it is executed.

11.6.2.3 sdiModifySql

As of SP8: with this program exit, the SQL from the generator or of lower scopes can be overwritten explicitly.

11.6.3 Specifications for the implementation class

Package name: the class has to be included in a package with the domain name (in lower case letters). Further sub-packages are **not** allowed.

Example: The service is requested "MDUserAccountRules.list", consequently the package is called "mduseraccountrules"

Class

name:

The class must have a name of the following structure: domain name in lower case letters, whereas the first letter is written with a capital letter, the name of the service function follows and is written in lower case letters. The first letter is again written in upper case.

Example: The service is requested "MDUserAccountRules.list", consequently the class is called "MduseraccountrulesList"



The following definition applies for customized class names:
Customized names include "_" (see naming conventions)
After "_" the first letter is changed to upper case and the underscore is skipped

Example: U_CUST_Units_sample.list => UCustUnitsSampleList

Implemented interfaces / methods: no specifications

Other: The class must have a default constructor without parameters

Compilation: The Jar files *MpdvDomCoreSdiCompileLib.jar* and *MpdvDomCoreUserExitCompileLib.jar* must be included in the class path for the compilation process.

Deployment: The class file of the exit must be included in the directory <JHYDRADIR>/MOC/<instance>/userexit/<scope> including package directory structure.

Example: Exit sdiAugmentSql for service "MDUserAccountRules.list":

```
package mduseraccountrules;
```

```
import de.mpdv.customization.userExit.IUserExitParam;
```

```
/**
```



```

* Sample user exit
*
*
*/
public class MduseraccountrulesList
{
    public void sdiAugmentSql(final IUserExitParam param)
    {
        // TODO implementation
    }
}

```

Directory structure in the server (instance 1, scope custom):

<JHYDRADIR>/MOC/1/userexit/custom/mduseraccountrules/MduseraccountrulesList.class

11.6.4 Interfaces

11.6.4.1 Class: InterpretedJavaServiceUeContext

de.mpdv.sdi.data.ue.InterpretedJavaServiceUeContext
-hydraNow: Calendar
-userId: String
+getHydraNow(): Calendar
+getUserId(): String

This context class provides data that is generally useful for interpreted Java services as regards to exits.

Field	Description
hydraNow	The property "hydraNow" is a time stamp created at the beginning of web service processing and, as a result, can be used as reference time stamp for the current web service request.
userId	Includes the user logged on to the client.

11.6.4.2 Program exit: sdiModifyColumnConfigurator

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiModifyColumnConfiguratorParam	Parameter structure for the user exit
context	InterpretedJavaServiceUeContext	Context structure for all exits

		of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiModifyColumnConfiguratorResult	Result structure for the user exit

Class diagram of parameter and result structures:

```
de.mpdv.sdi.data.ue.SdiModifyColumnConfiguratorParam
+getColumnConfigurator():ColumnConfigurator
+getSpecialParameters():Map<String, SpecialParam>
```

```
de.mpdv.sdi.data.ue.SdiModifyColumnConfiguratorResult
+getColumnConfigurator():ColumnConfigurator
```

11.6.4.3 Class SdiModifyColumnConfiguratorParam

Field	Type	Description
columnConfigurator	ColumnConfigurator	Includes the columns requested by the client
specialParameters	Map<String, SpecialParam>	Assignment of acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

11.6.4.4 Class SdiModifyColumnConfiguratorResult

Field	Type	Description
columnConfigurator	ColumnConfigurator	Includes the requested columns after processing in user exit

11.6.4.5 Program exit: sdiModifyColumnMap

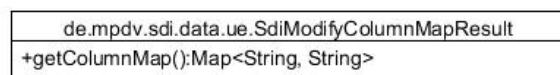
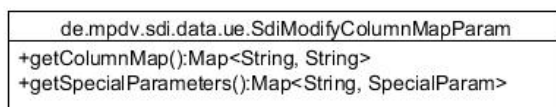
Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiModifyColumnMapParam	Parameter structure for the program exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiModifyColumnMapResult	Result structure for the program exit

Class diagram of parameter and result structures:



11.6.4.6 Class SdiModifyColumnMapParam

Field	Type	Description
columnMap	Map<String, String>	Includes the assignment of acronym => table column (including alias)
specialParameters	Map<String, SpecialParam>	Assignment of acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

11.6.4.7 Class SdiModifyColumnMapResult

Field	Type	Description
columnMap	Map<String, String>	Includes the assignment of acronym => table column (including alias)

11.6.4.8 Program exit: sdiAugmentSql

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiAugmentSqlParam	Parameter structure for the program exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiAugmentSqlResult	Result structure for the program exit

Class diagram of parameter and result structures:

de.mpdv.sdi.data.ue.SdiAugmentSqlParam
+getSelect():String +getFrom():String +getWhere():String +getGroupBy():String +getOrderBy():String +getSpecialParameters():Map<String, SpecialParam>

de.mpdv.sdi.data.ue.SdiAugmentSqlResult
+getFromSuffix():String +getWhereSuffix():String +getGroupBySuffix():String +getOrderBySuffix():String

11.6.4.9 Class SdiAugmentSqlParam

Field	Type	Description
select	String	SELECT clause created based on the configuration (only includes the column part up to FROM, only without the key word SELECT)
from	String	FROM clause created based on the configuration (without the key word FROM)
WHERE	String	WHERE clause created based on the configuration (without the key word

		WHERE)
groupBy	String	GROUP BY clause created based on the configuration
orderBy	String	ORDER BY clause created based on the configuration
specialParameters	Map<String, SpecialParam>	Assignment of acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

11.6.4.10 Class SdiAugmentSqlResult

Field	Type	Description
fromSuffix	String	Suffix for the FROM clause created in the program exit or NULL
whereSuffix	String	Suffix for the WHERE clause created in the program exit or NULL
groupBySuffix	String	Suffix for the GROUP BY clause created in the program exit or NULL
orderBySuffix	String	Suffix for the ORDER BY clause created in the program exit or NULL

11.6.4.11 User exit: sdiModifyResultList

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiModifyResultListParam	Parameter structure for the user exit
context	InterpretedJavaServiceUeContext	Context structure for all user exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system

		utilities, such as logger or DB connection in user exits.
--	--	---

Return key in IUserExitParam:

Key name	Type	Description
result	SdiModifyResultListResult	Result structure for the user exit

Class diagram of parameter and result structures:

de.mpdv.sdi.data.ue.SdiModifyResultListParam
+getColumnConfigurator():ColumnConfigurator
+getDataTables():List<IDataTable>
+getSpecialParameters():Map<String, SpecialParam>

de.mpdv.sdi.data.ue.SdiModifyResultListResult
+getDataTables():List<IDataTable>

11.6.4.12 Class SdiModifyResultListParam

Field	Type	Description
columnConfigurator	ColumnConfigurator	Includes the columns requested by the client
dataTables	List<IDataTable>	Includes the data table(s) generated as service result by the interpreter
specialParameters	Map<String, SpecialParam>	Assignment of acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

11.6.4.13 Class SdiModifyResultListResult

Field	Type	Description
dataTables	List<IDataTable>	Includes the data table(s) after processing in the user exit that the service supplies as result to the client

11.6.4.14 Program exit: sdiModifySql

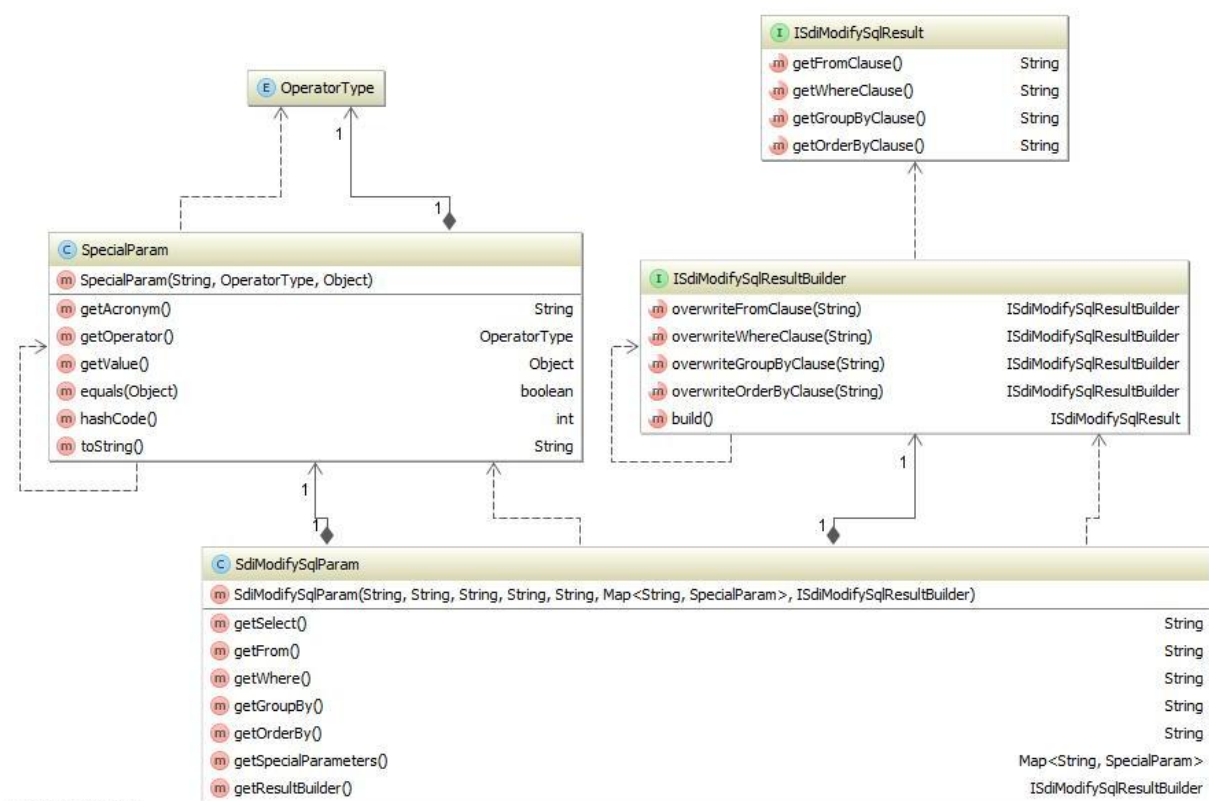
Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiModifySqlParam	Parameter structure for the program exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	ISdiModifySqlResult	Result structure of the program exit

Class diagram of parameter and result structures:



11.6.4.15 Class SdiModifySqlParam

Field	Type	Description
select	String	SELECT clause created based on the configuration (only includes the

		column part up to FROM, only without the key word SELECT)
from	String	FROM clause created based on the configuration (without the key word FROM)
WHERE	String	WHERE clause created based on the configuration (without the key word WHERE)
groupBy	String	GROUP BY clause created based on the configuration
orderBy	String	ORDER BY clause created based on the configuration
specialParameters	Map<String, SpecialParam>	Assignment of acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

11.6.4.16 Interface ISdiModifySqlResultBuilder

Method	Description
overwriteFromClause():ISdiModifySqlResultBuilder	Overwrites FROM clause. Only call this method if you really want to overwrite the FROM clause!
overwriteWhereClause():ISdiModifySqlResultBuilder	Overwrites WHERE clause. Only call this method if you really want to overwrite the WHERE clause!
overwriteGroupByClause():ISdiModifySqlResultBuilder	Overwrites GROUP BY clause. Only call this method if you really want to overwrite the GROUP BY clause!
overwriteOrderByClause():ISdiModifySqlResultBuilder	Overwrites ORDER BY clause.

	Only call this method if you really want to overwrite the Order BY clause!
build():ISdiModifySqlResult	Creates the result structure of the program exit.

11.6.4.17 Interface ISdiModifySqlResult

Method	Description
getFromClause(): String	FROM clause of the program exit if the clause is overwritten there. Otherwise it is the original clause.
getWhereClause(): String	WHERE clause of the program exit if the clause is overwritten there. Otherwise it is the original clause.
getGroupByClause(): String	GROUP BY clause from the program exit if it was overwritten there, otherwise the original clause.
getOrderByClause(): String	ORDER BY clause of the program exit if the clause is overwritten there. Otherwise it is the original clause.